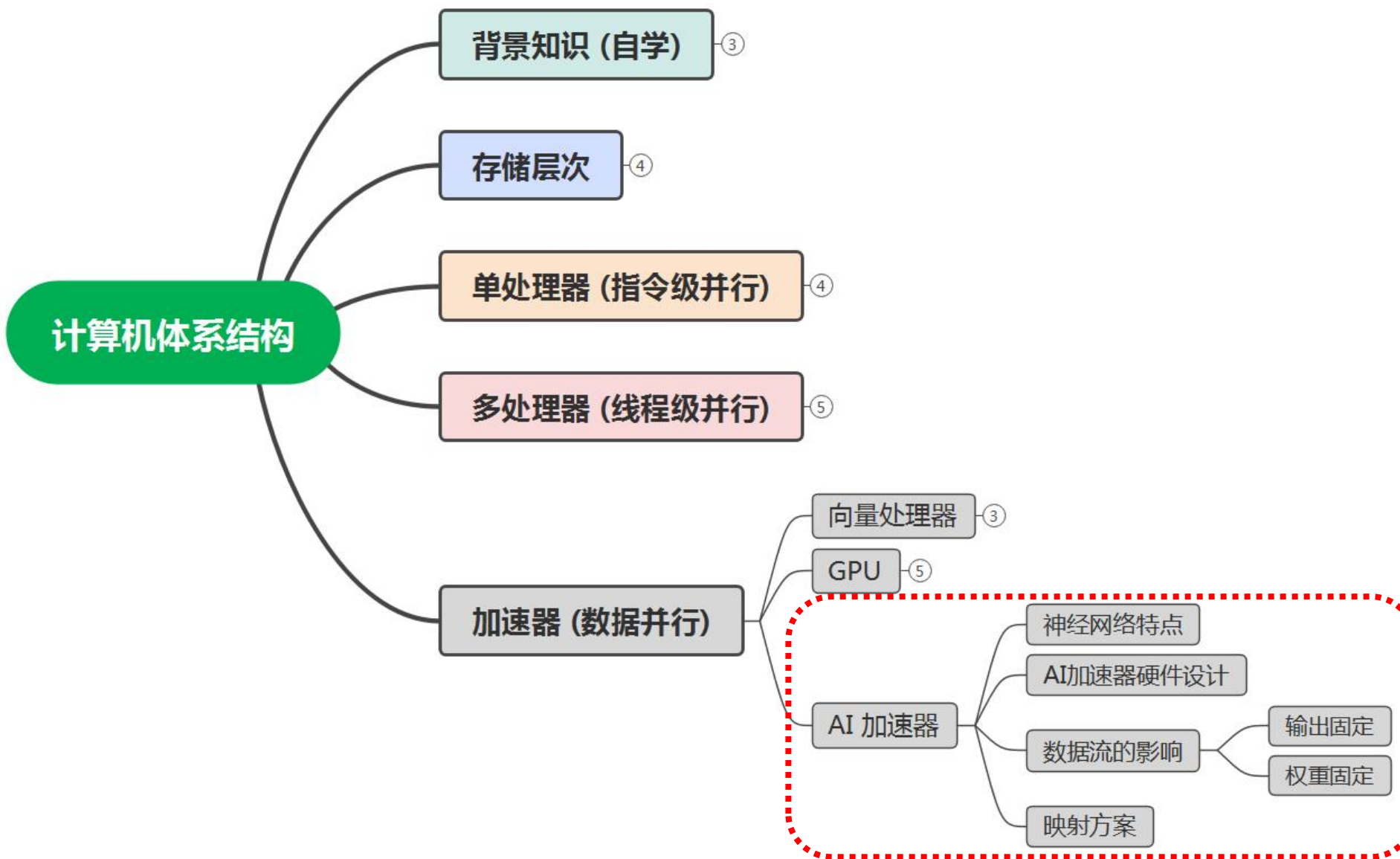


计算机体系结构

L09 AI 加速器



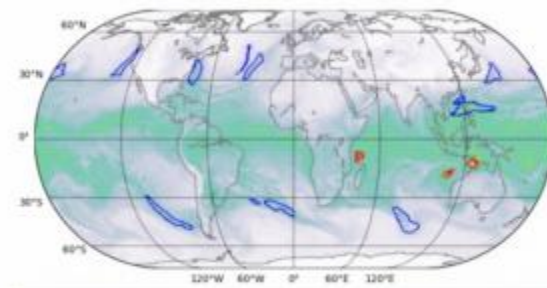
神经网络特点

AI 的兴起

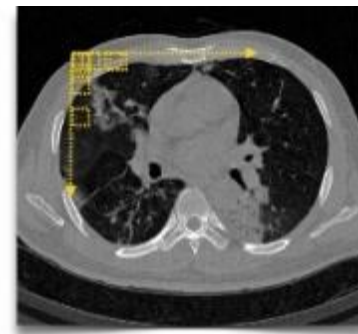
目标检测



图像分割



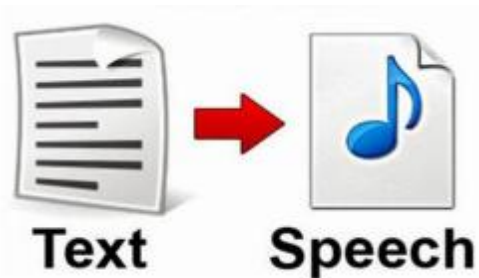
医学图像



语音识别



文字转换



推荐系统



游戏



AI 算力需求指数级增长

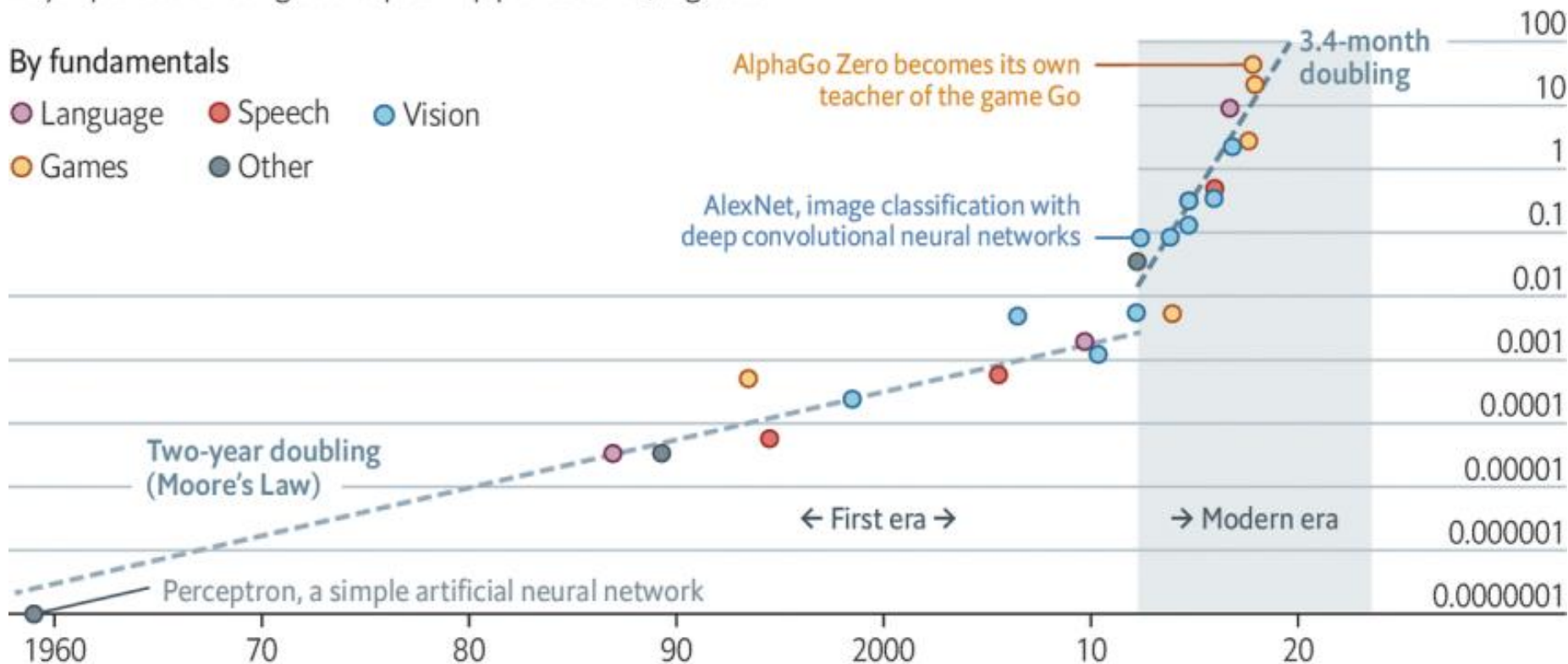
Deep and steep

Computing power used in training AI systems

Days spent calculating at one petaflop per second*, log scale

By fundamentals

- Language
- Speech
- Vision
- Games
- Other



Source: OpenAI

The Economist

*1 petaflop=10¹⁵ calculations

为何需要DNN加速器?

– 上亿的计算量

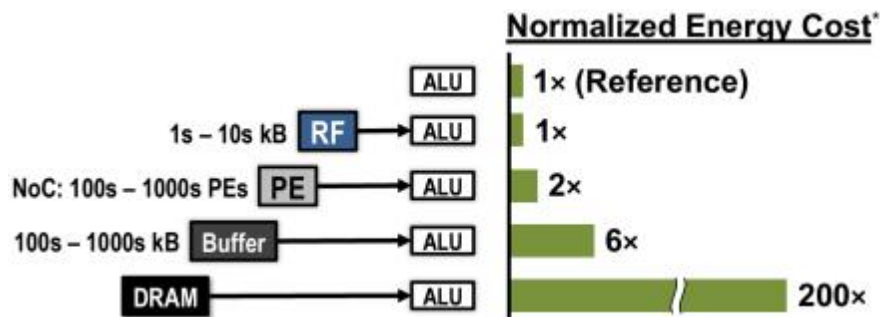
DNN Topology	Number of Weights
AlexNet (2012)	3.98M
VGGnet-16 (2014)	28.25M
GoogleNet (2015)	6.77M
Resnet-50 (2016)	23M
DLRM (2019)	540M
Megatron (2019)	8.3B



需要大量并行计算

CPU难以应对

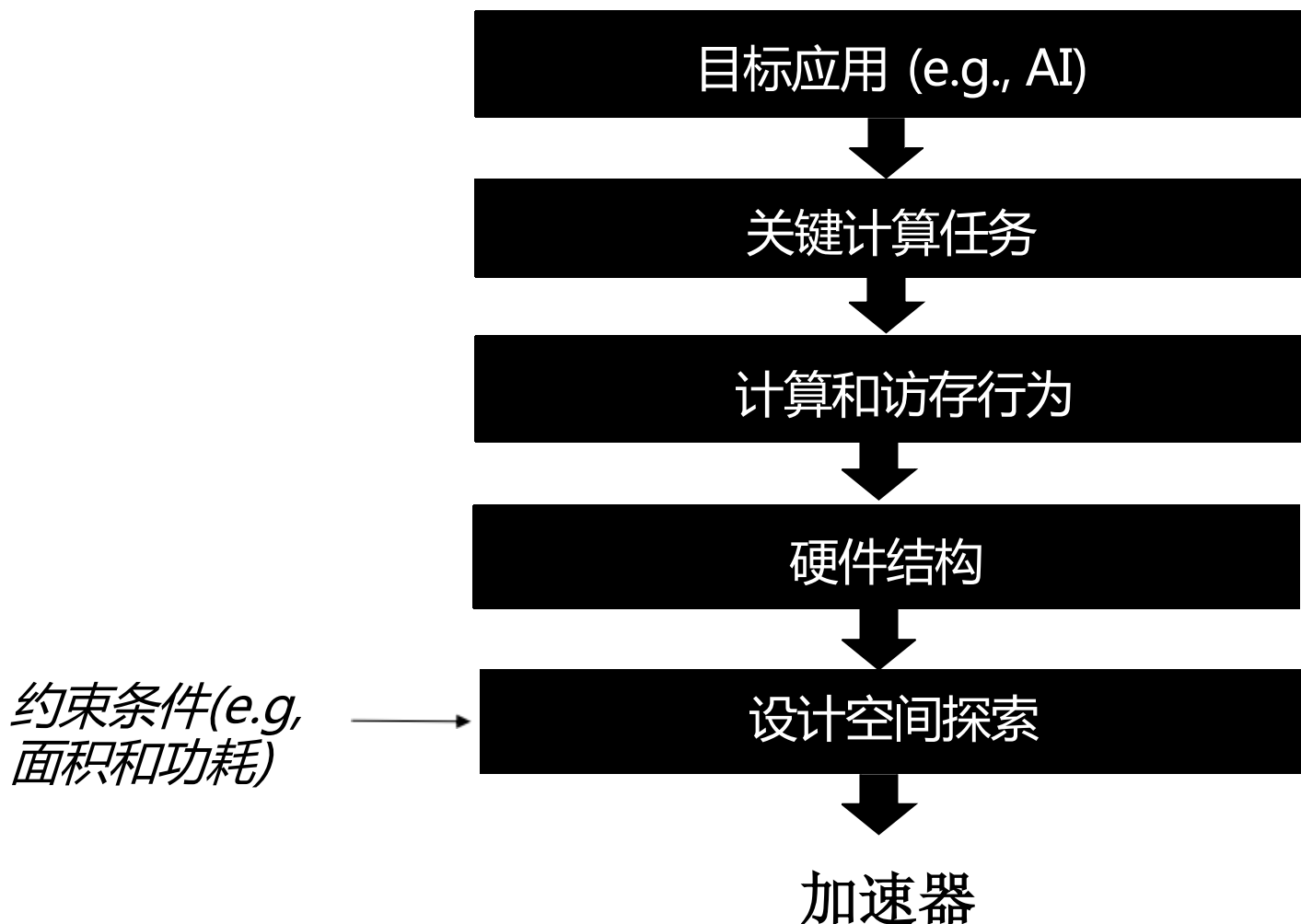
– 大量数据移动



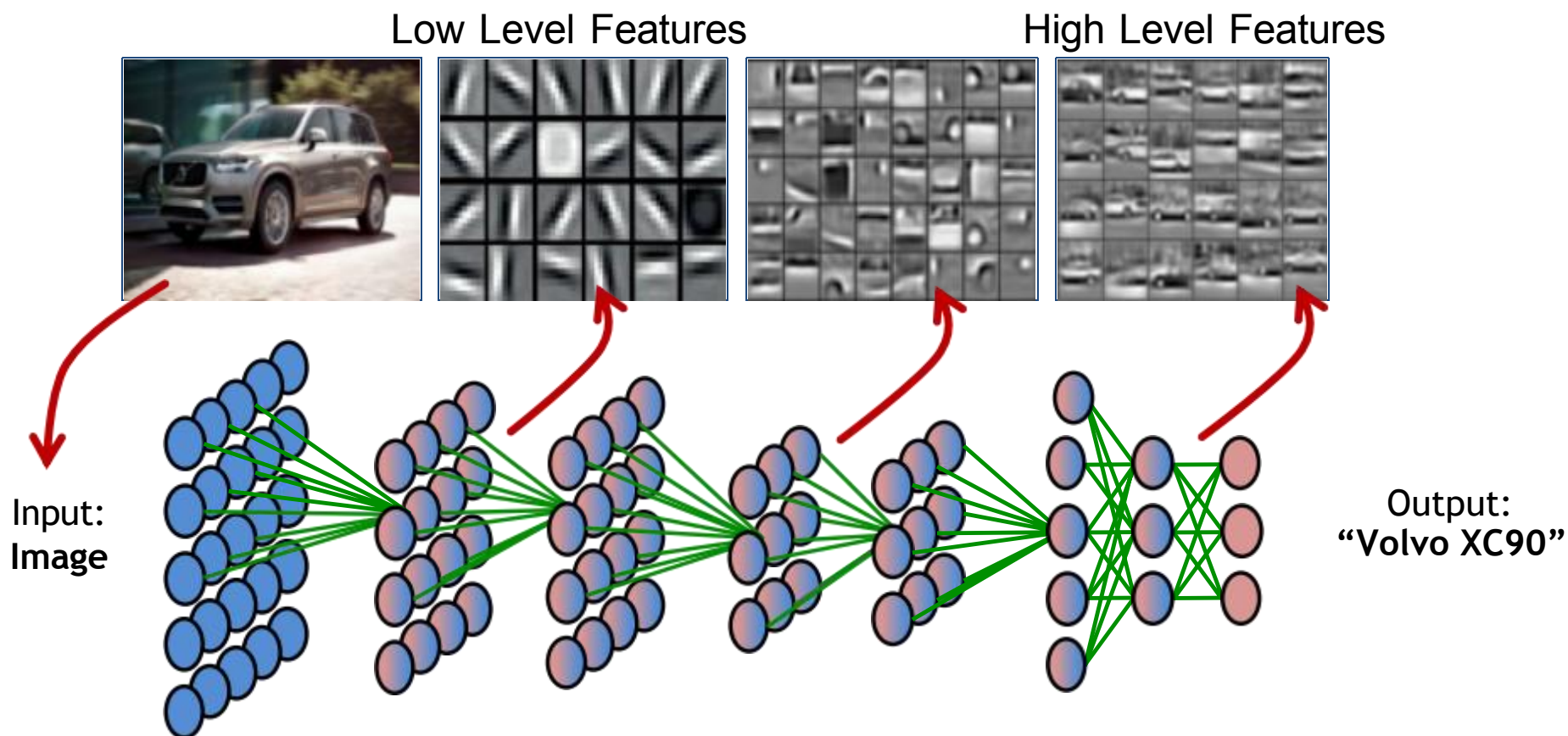
需要降低能耗

GPU难以应对

软硬件协同设计

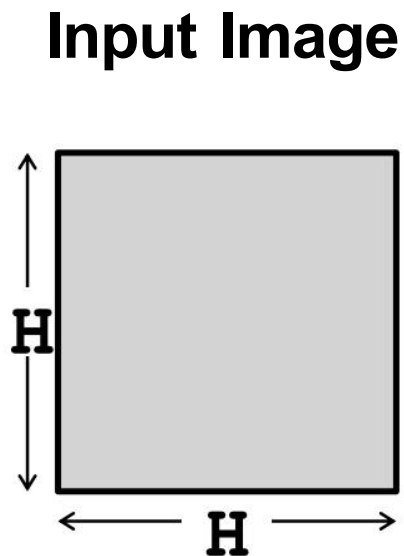
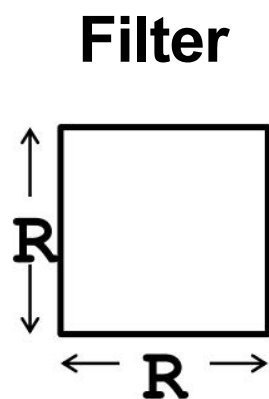


理解深度神经网络

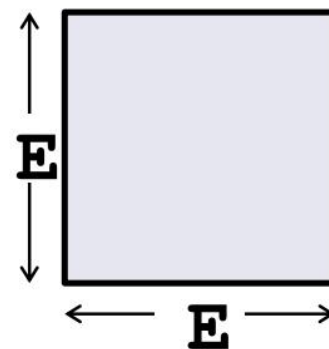


Modified Image Source: [Lee, CACM 2011]

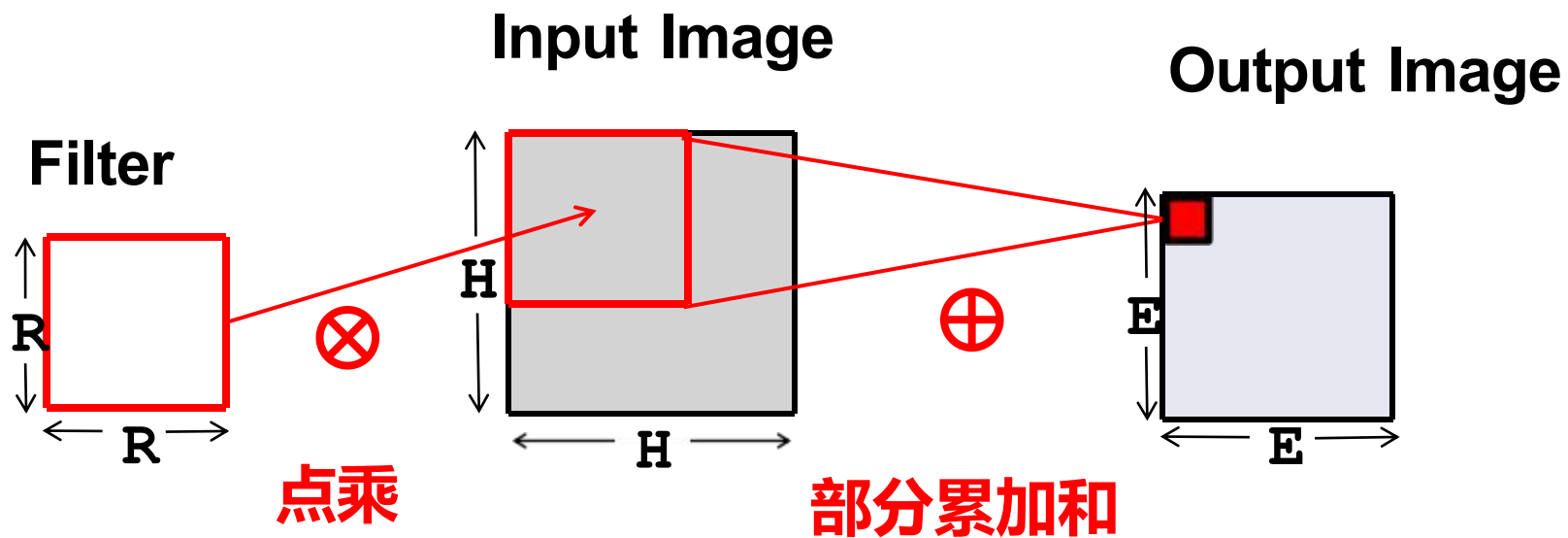
卷积操作



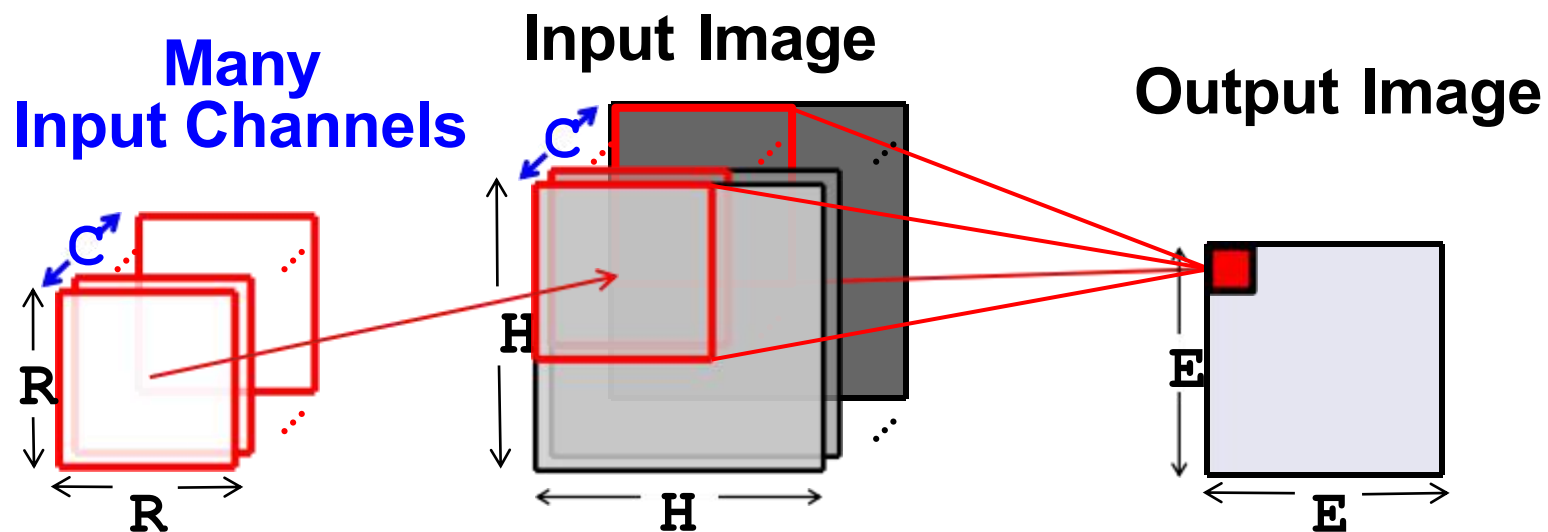
Output Image



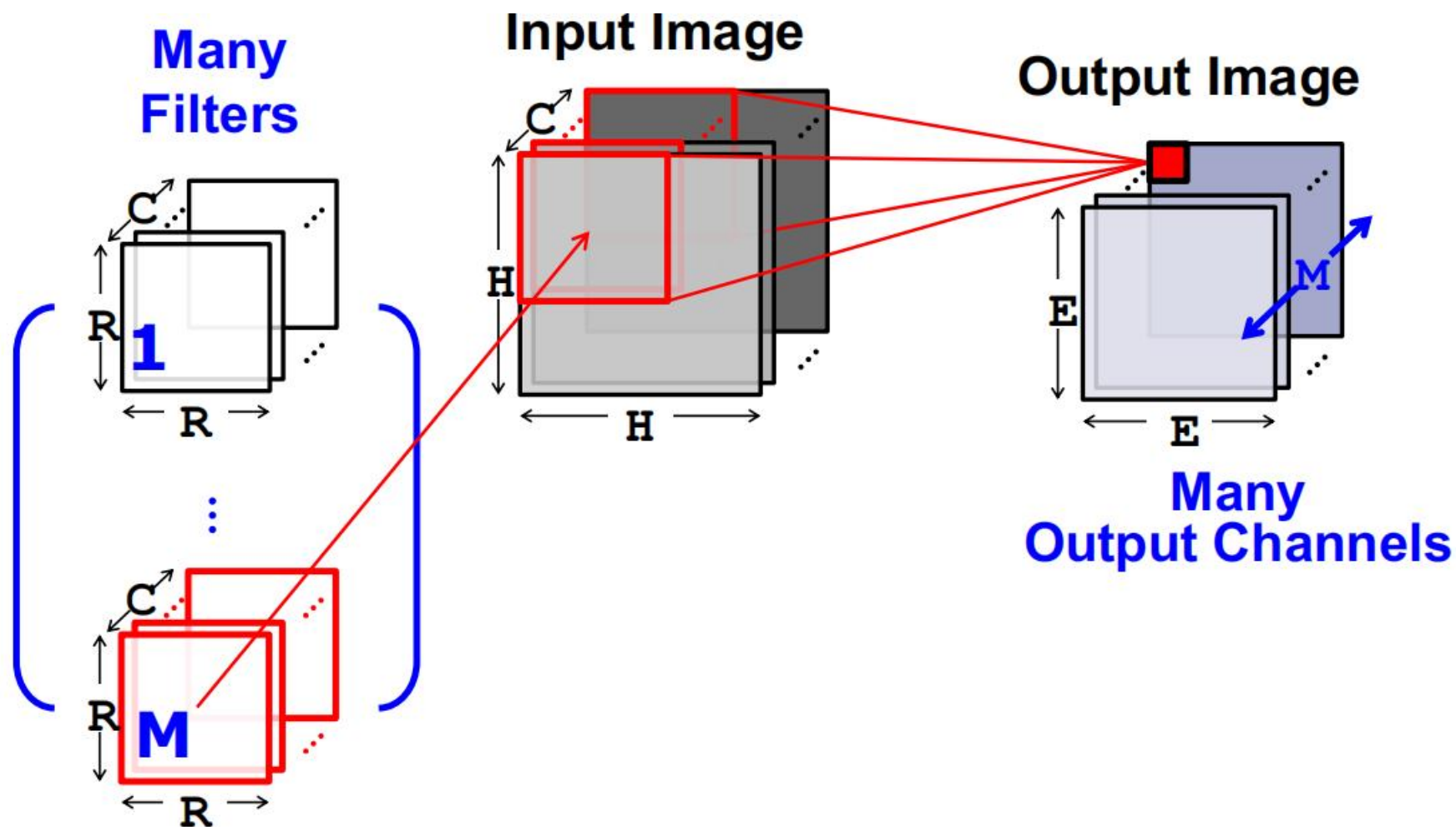
卷积操作



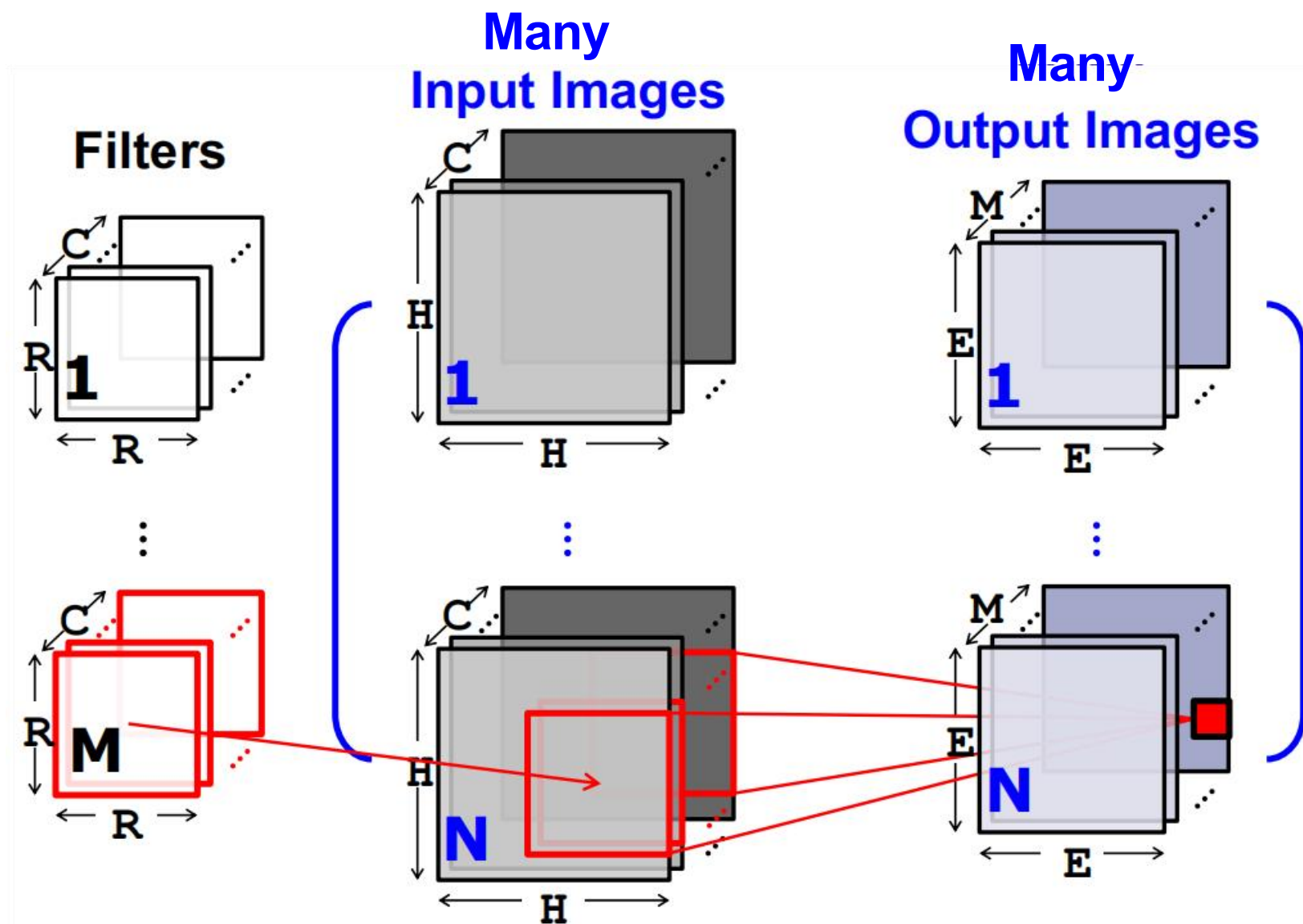
卷积操作



卷积操作



卷积操作



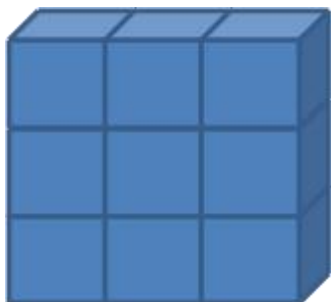
0阶张量 - Scalar



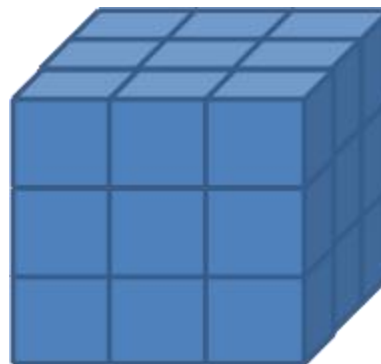
1阶张量 - Vector



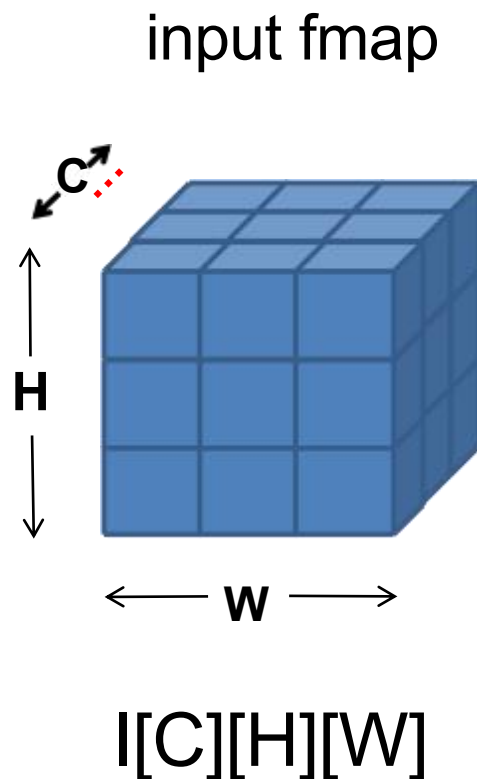
2阶张量 - Matrix



3阶张量 - Cube



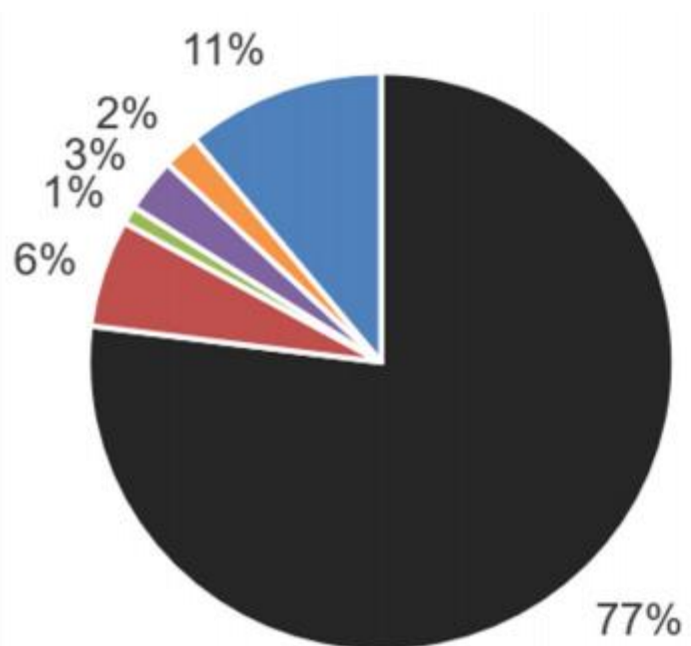
Example: 输入特征张量



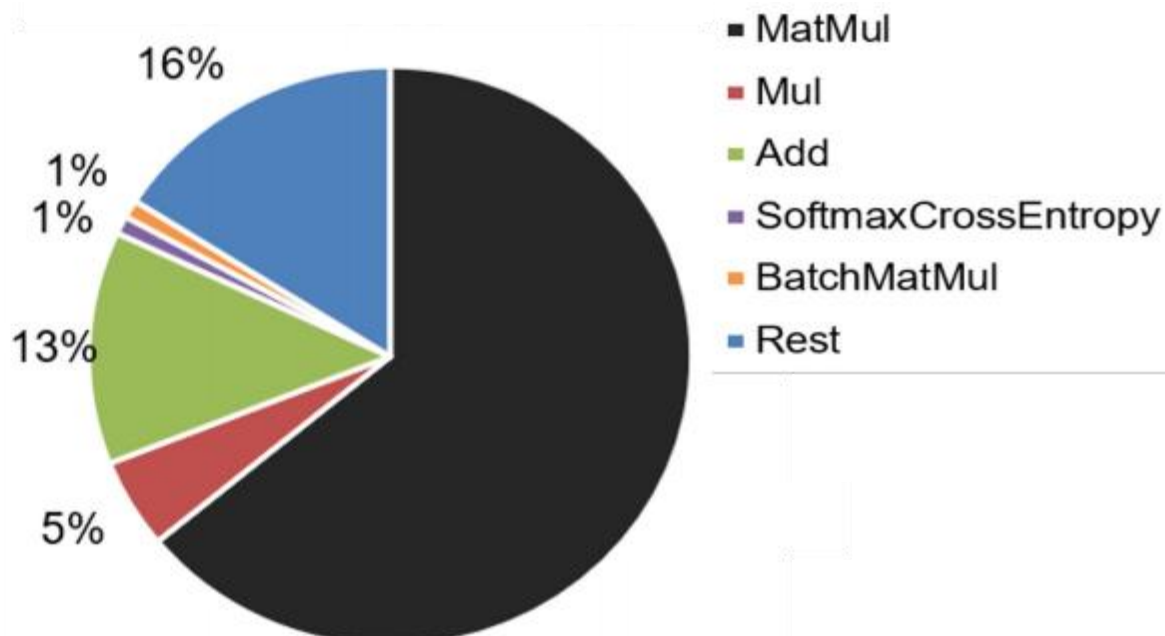
编译器将高阶张量“降维”
到适当的硬件结构中

AI 加速器硬件设计

加速哪些操作?



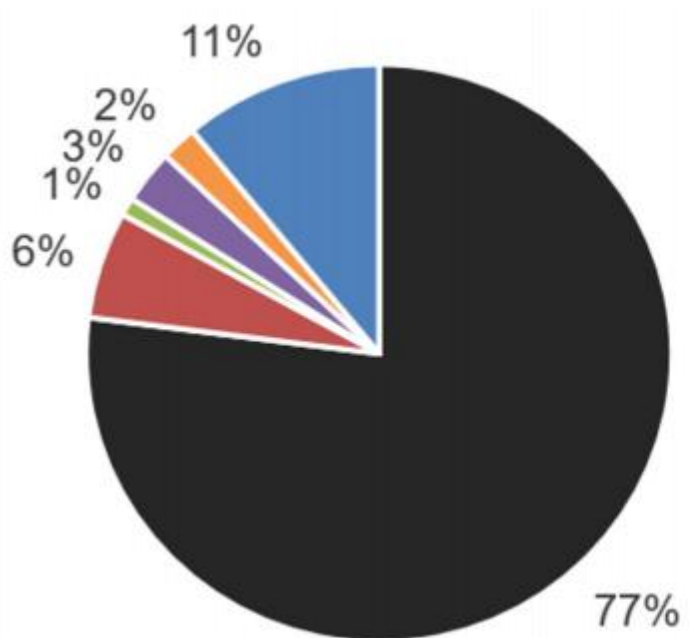
Transformer
(Language Understanding)



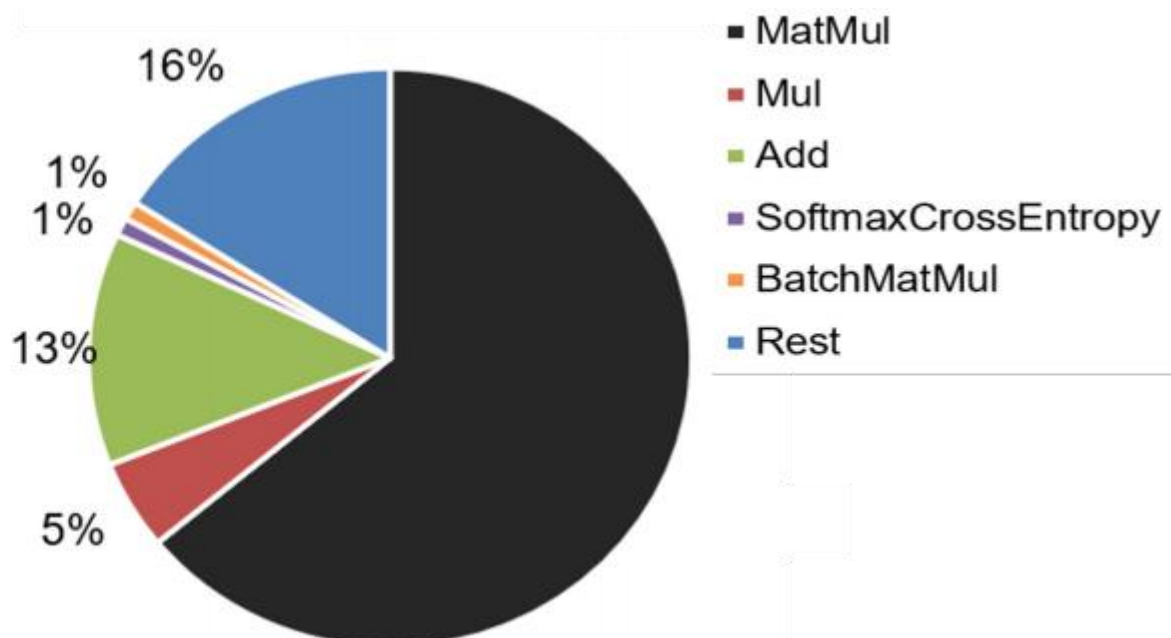
GNMT
(Machine Translation)

Runtime breakdown on V100 GPU

加速哪些操作?



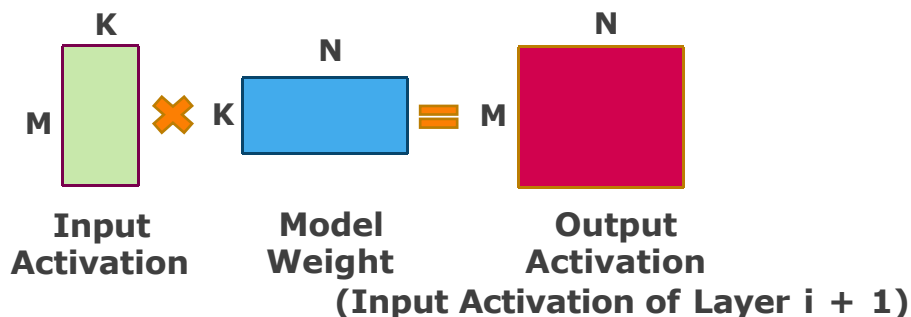
Transformer
(Language Understanding)



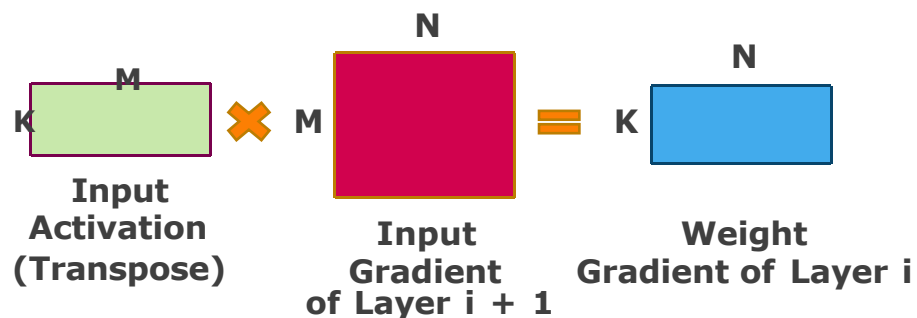
GNMT
(Machine Translation)

矩阵乘法在现代深度学习工作负载中占据大约70%的总运行时间!

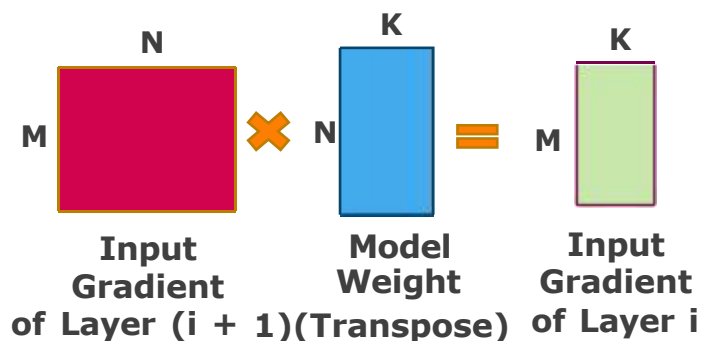
深度学习中的矩阵乘



前向传播



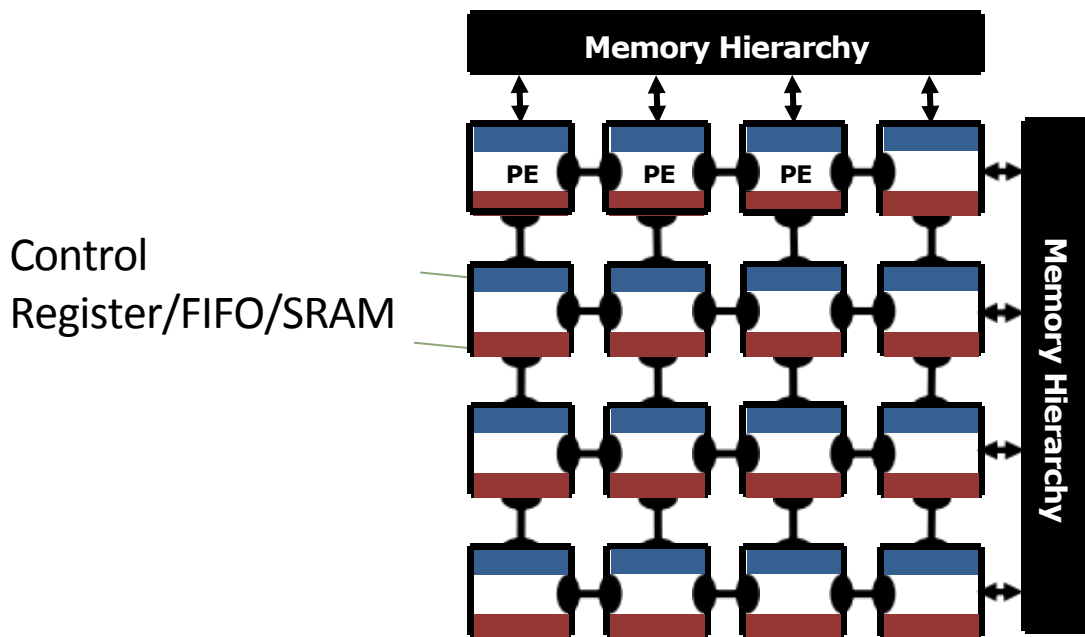
后向传播



空间（或数据流）加速器

- 上亿参数(i.e., weights)

- 十亿以上计算量
- 大量数据移动



特征

- 上千个运算单元 (PEs)
- 定制的Memory Hierarchy(通常无 cache)
- 定制的互连网络

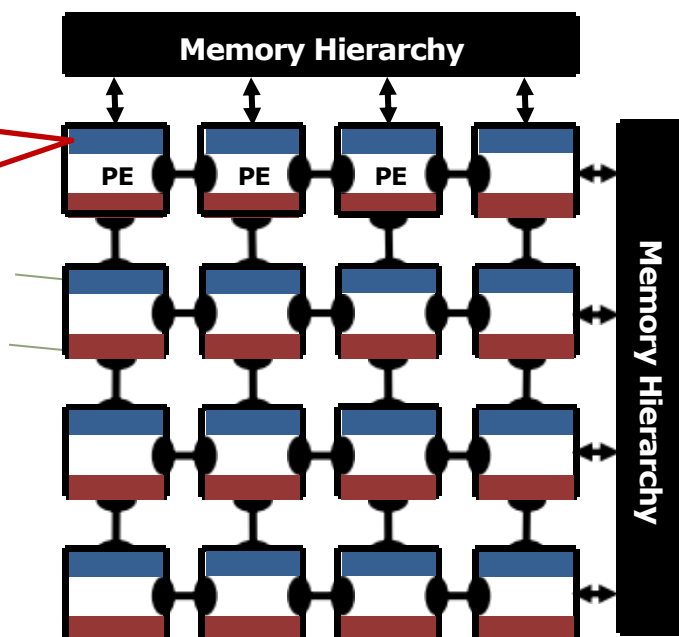
空间（或数据流）加速器

- 上亿参数(i.e., weights)

- 十亿以上计算量
- 大量数据移动

将计算分布到成千上万个处理单元(i.e., 并行性)

Control
Register/FIFO/SRAM



特征

- 上千个运算单元 (PEs)
- 定制的Memory Hierarchy(通常无cache)
- 定制的互连网络

空间（或数据流）加速器

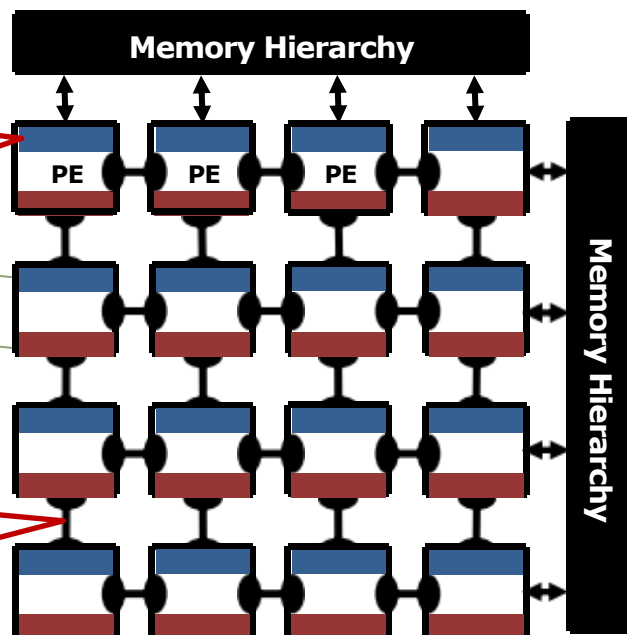
- 上亿参数(i.e., weights)

- 十亿以上计算量
- 大量数据移动

将计算分布到成千上万个处理单元(i.e., 并行性)

Control
Register/FIFO/SRAM

通过数据流实现阵列内的数据重用



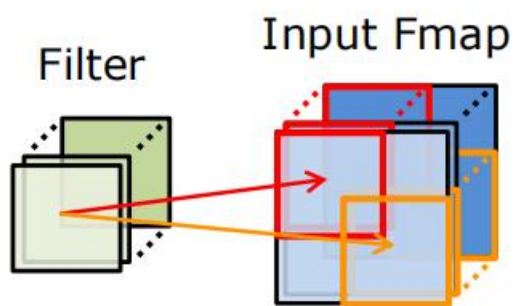
特征

- 上千个运算单元 (PEs)
- 定制的Memory Hierarchy(通常无 cache)
- 定制的互联网络

DNN中的数据重用

卷积重用

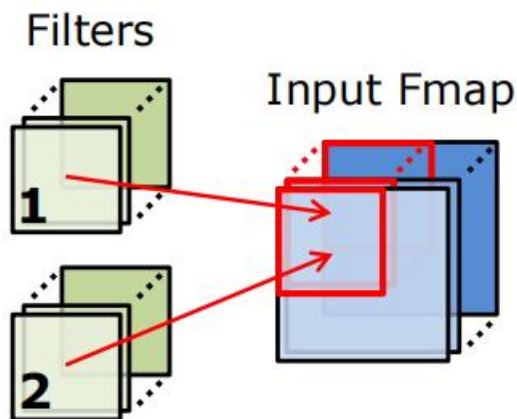
(滑动窗口计算卷积)



重用: 输入特征图
卷积核

特征图重用

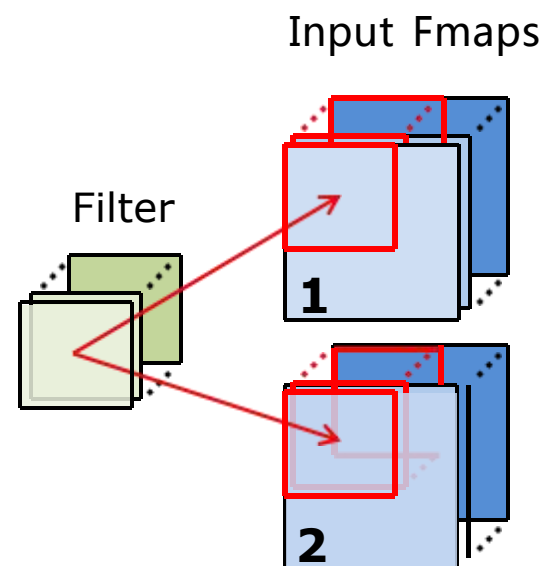
(同一特征图, 多个卷积核)



重用: 输入特征图

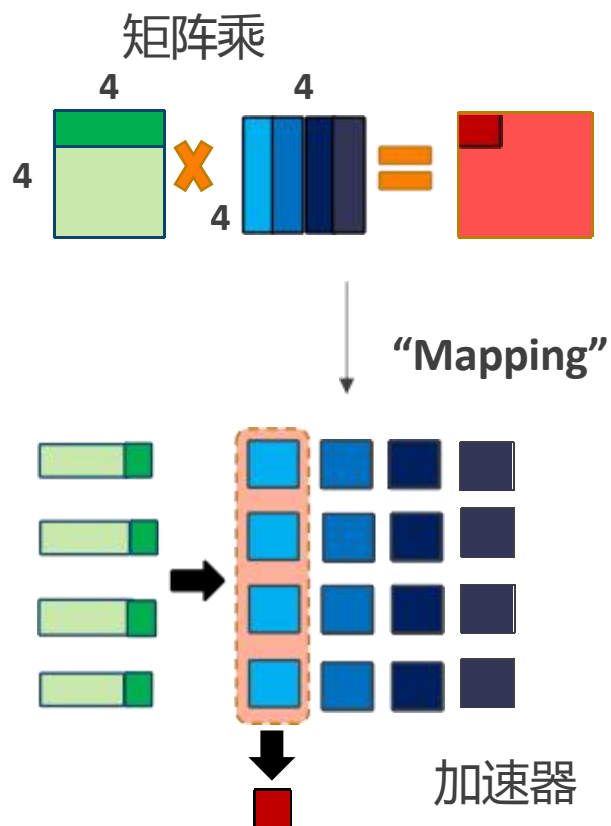
卷积核重用

(同一卷积核, 多个特征图)

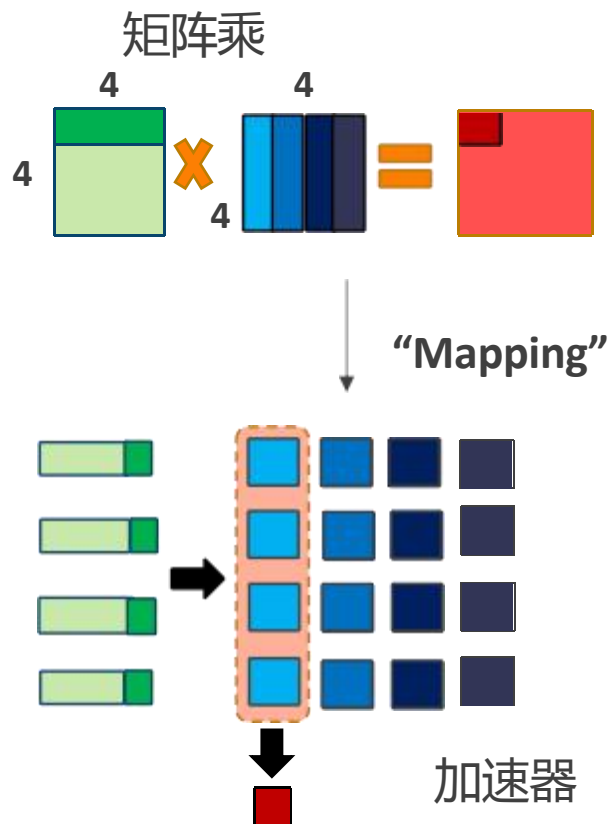


重用: 卷积核

如何利用数据重用?

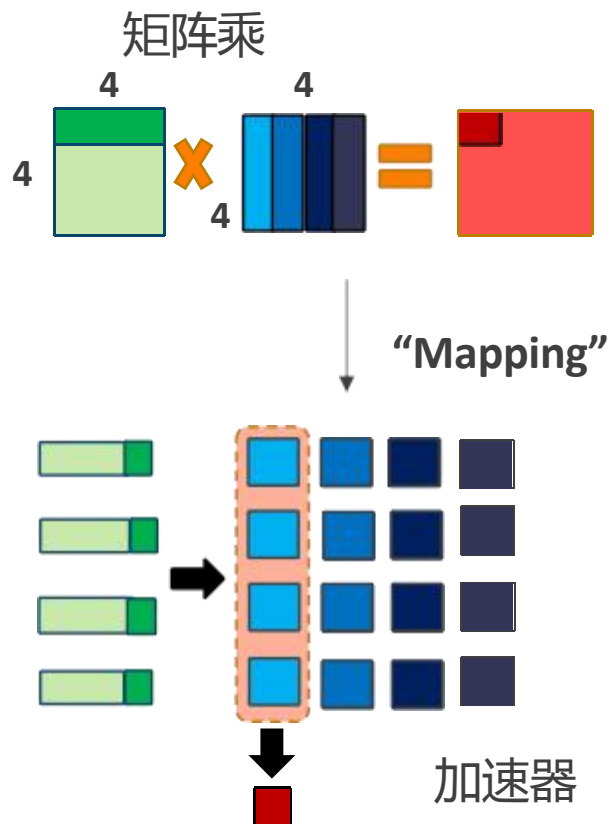


如何利用数据重用?



权重矩阵(蓝色)**重用**, 输入特征**流动**(称为 “权重静态映射”)

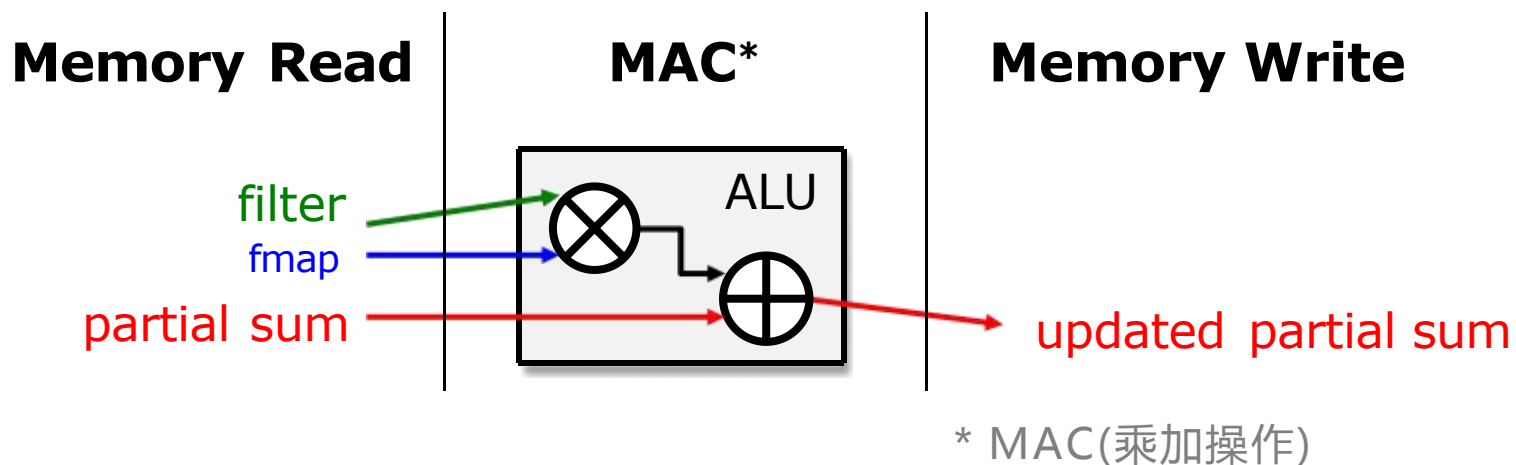
如何利用数据重用?



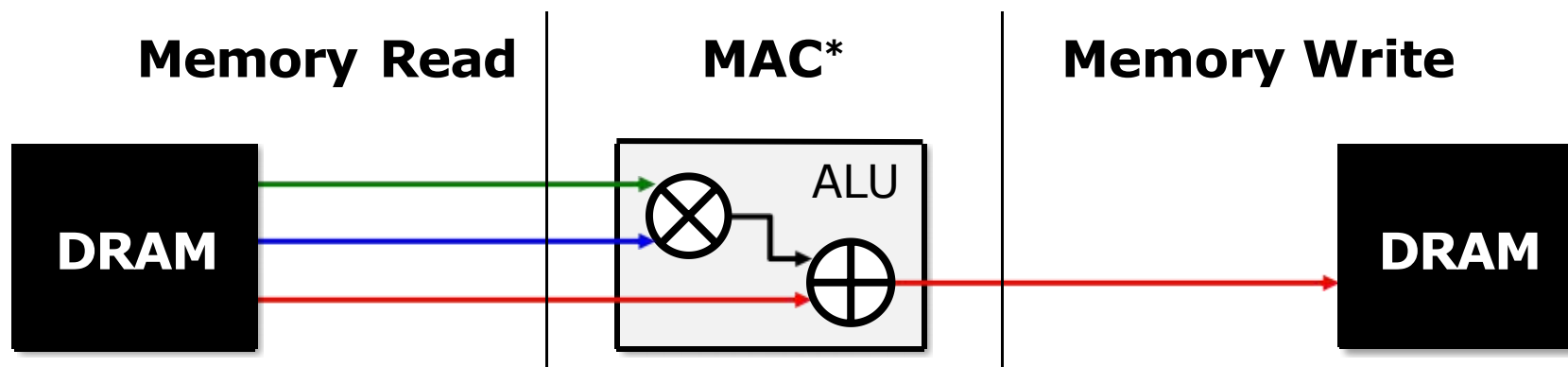
权重矩阵(蓝色)**重用**, **输入特征流动**(称为 “权重静态映射”)

需要什么样的硬件结构?

设计DNN加速器



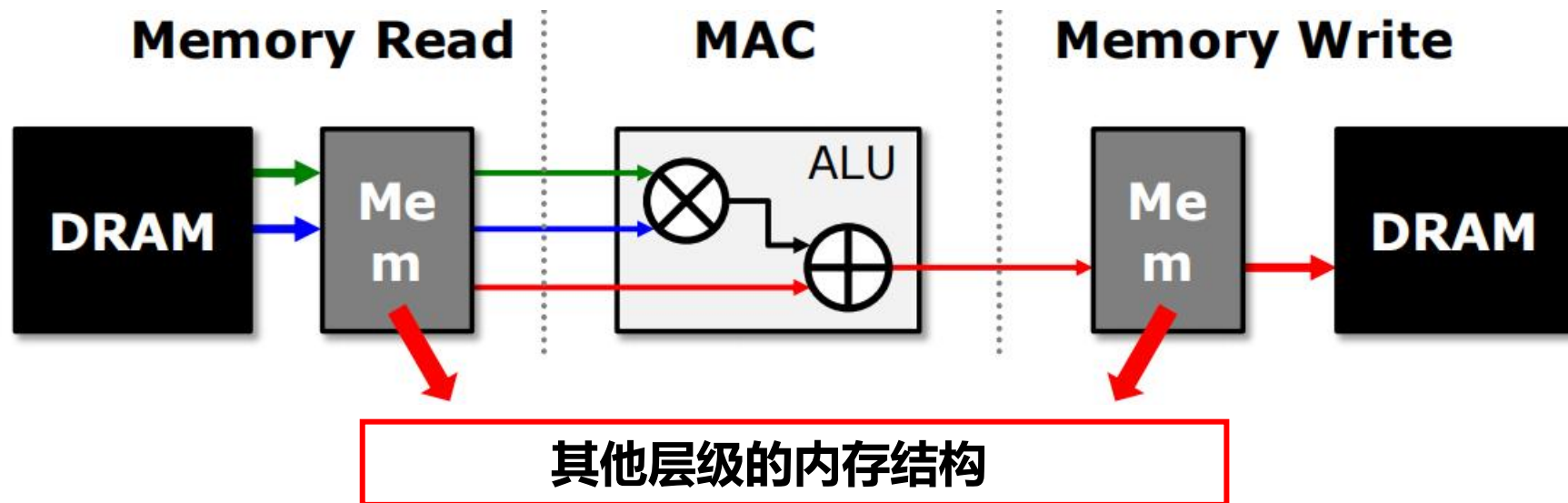
设计DNN加速器



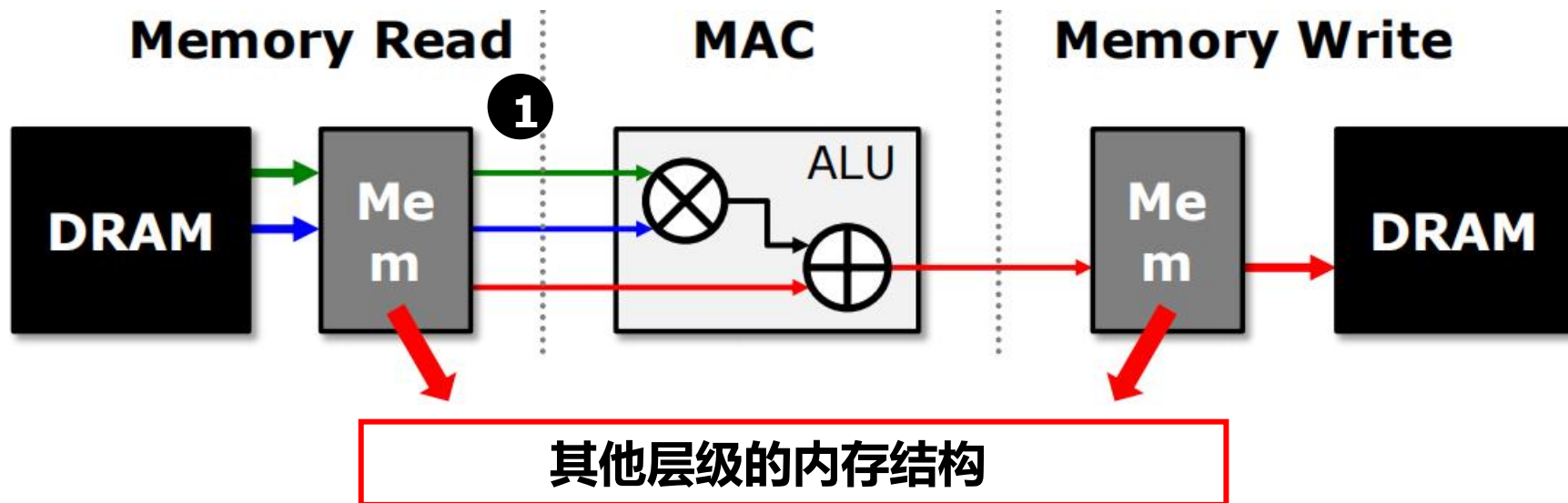
最坏情况: 所有内存读/写都访问**DRAM**

Example: AlexNet [NeurIPS 2012] 有724M 个乘加操作,
需要2896M 次DRAM访问

设计DNN加速器

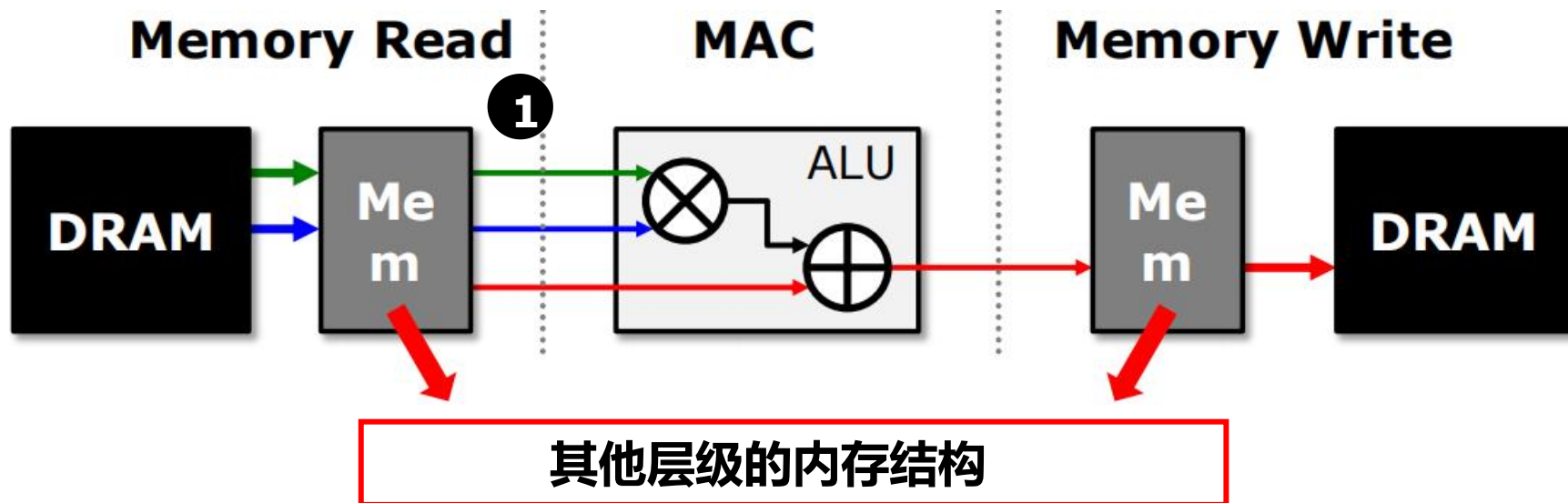


设计DNN加速器



Opportunities: **1** 数据重用

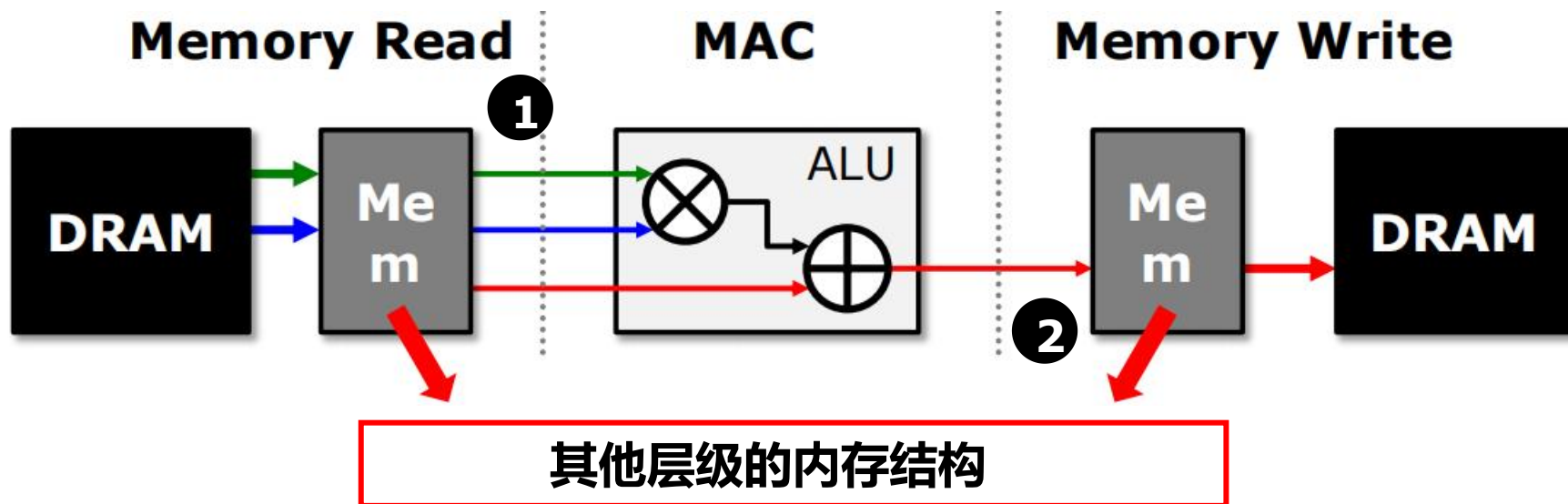
设计DNN加速器



Opportunities: ❶ 数据重用

❶ 可以减少高达500倍的DRAM访问 (AlexNet CONV layers)

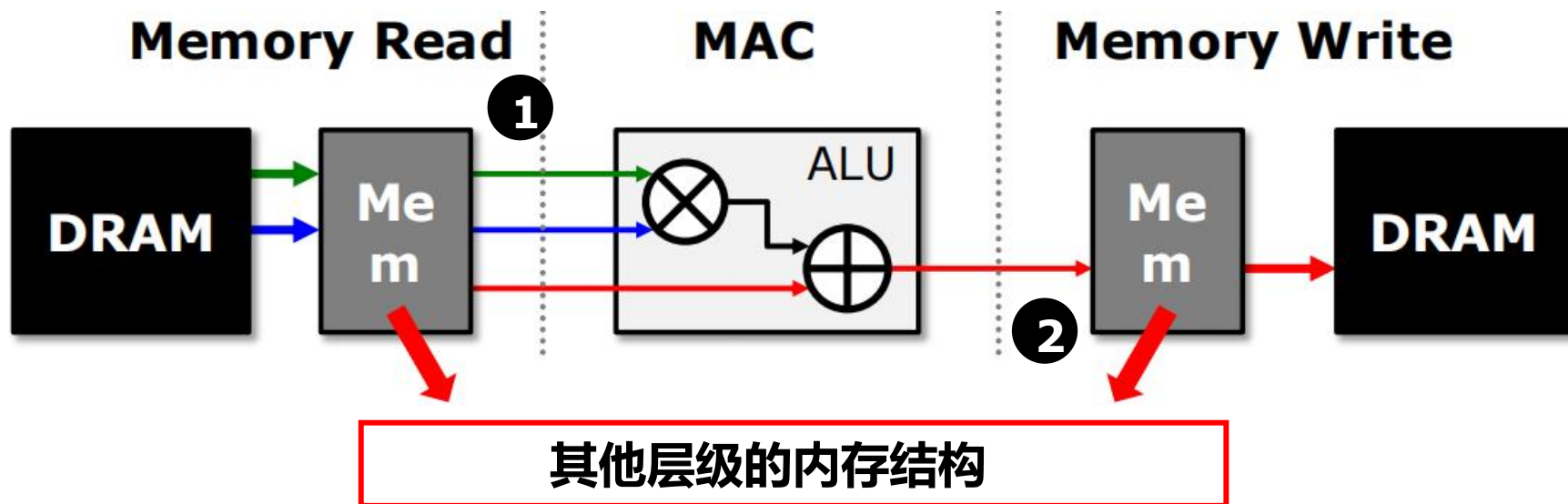
设计DNN加速器



Opportunities: ❶ 数据重用 ❷ 本地累加

- ❶ 可以减少高达500倍的DRAM访问 (AlexNet CONV layers)
- ❷ 部分和累加不需要访问DRAM

设计DNN加速器



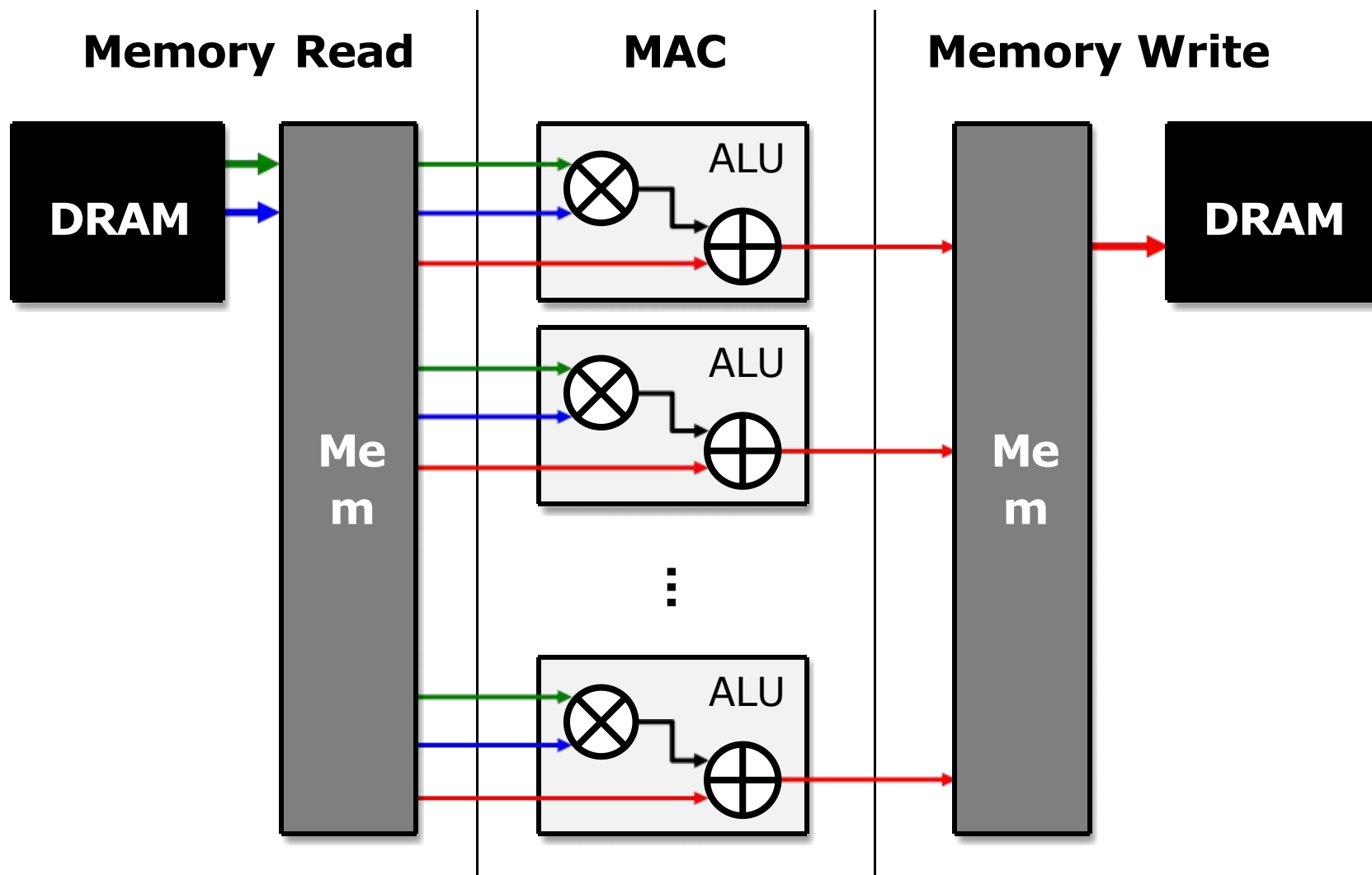
Opportunities: ① 数据重用 ② 本地累加

① 可以减少高达500倍的DRAM访问 (AlexNet CONV layers)

② 部分和累加不需要访问DRAM

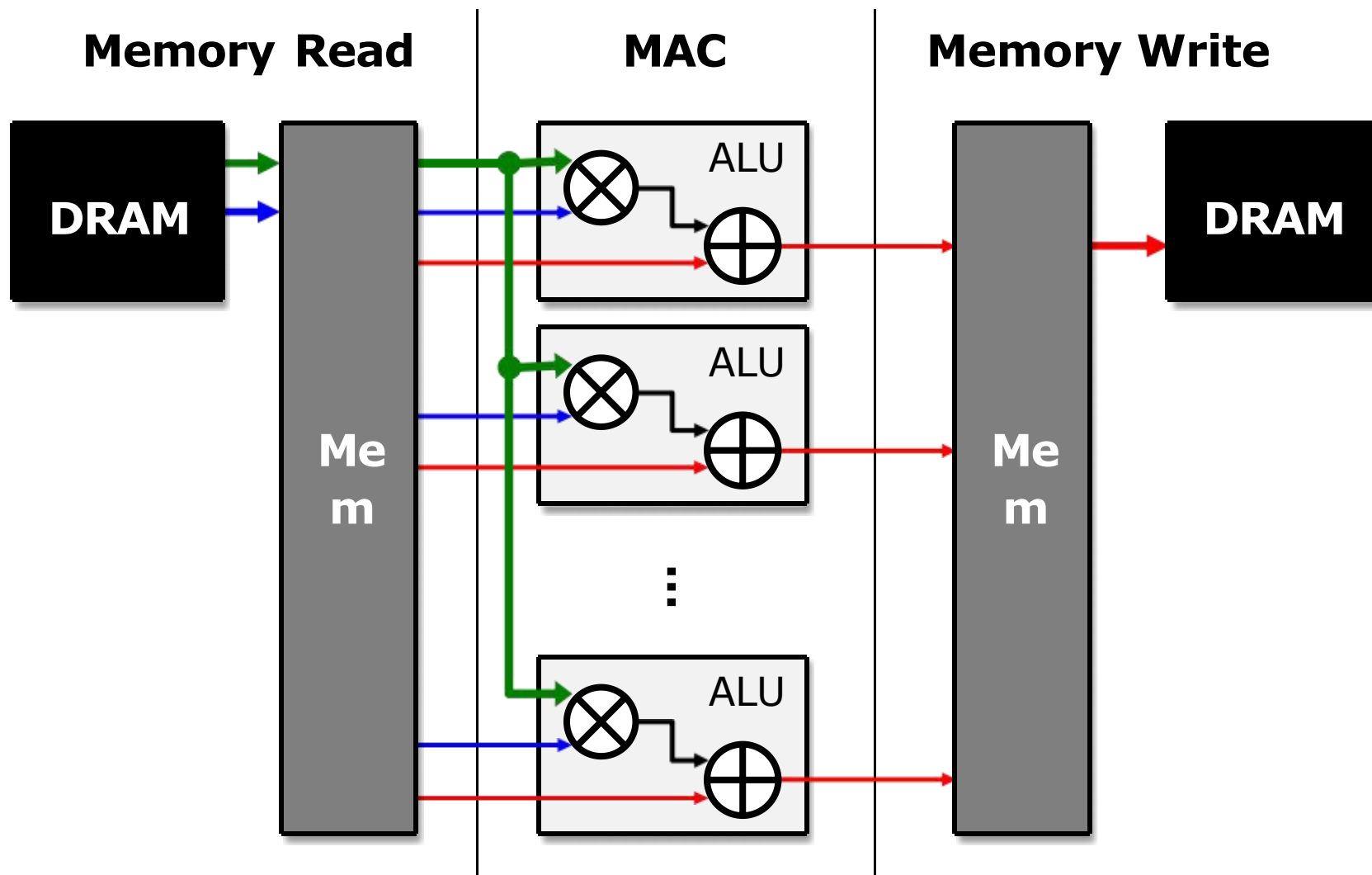
例如：DRAM访问AlexNet可以从2896M减少到61M(最好情况)

设计DNN加速器



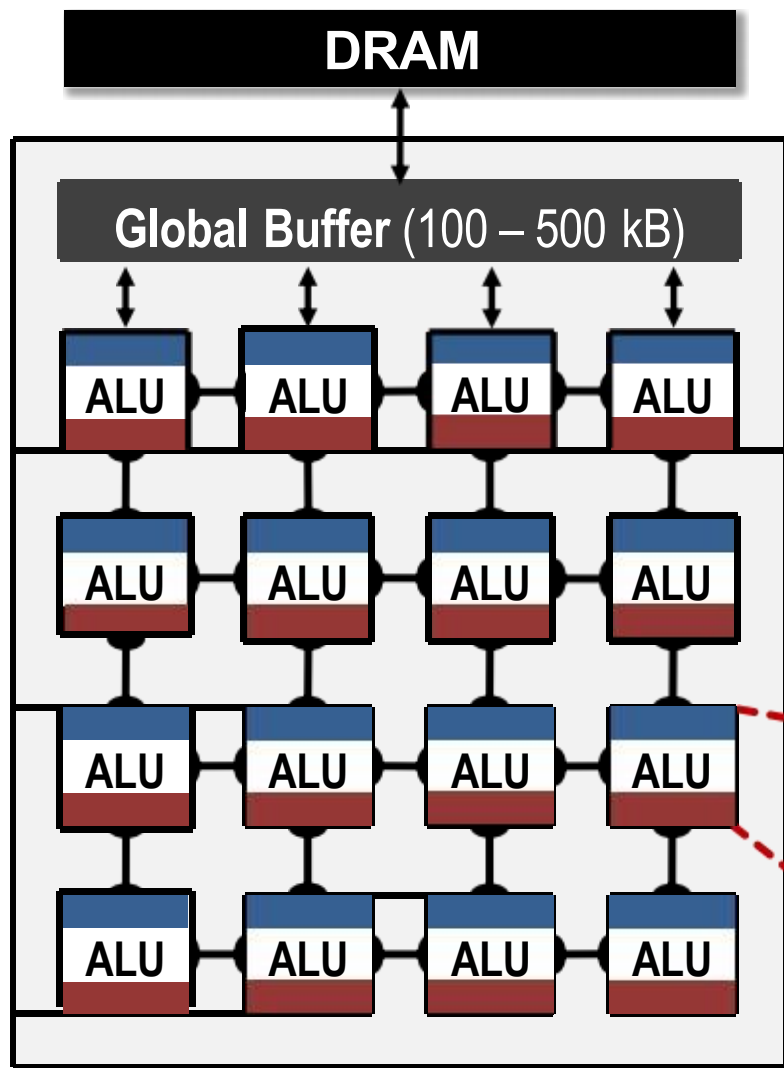
利用并行性提高吞吐量!

卷积操作



利用并行性提高空间数据重用!

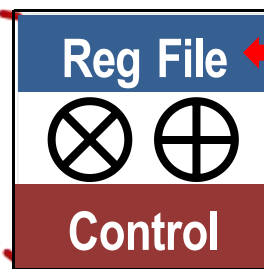
空间型 DNN 加速器



Local Memory Hierarchy

- Global Buffer (全局缓存)
- PE之间直连网络
- PE本地内存 (寄存器)

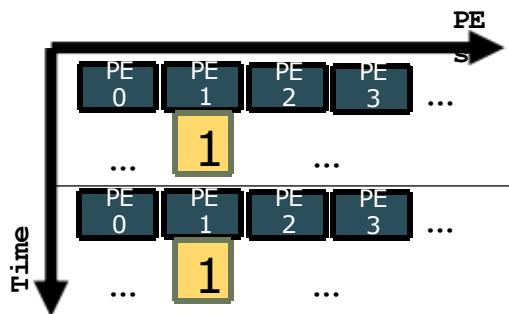
运算单元
(PE)



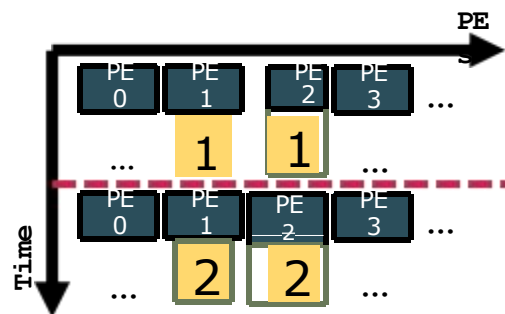
0.5 – 1.0 kB

利用数据重用的硬件结构

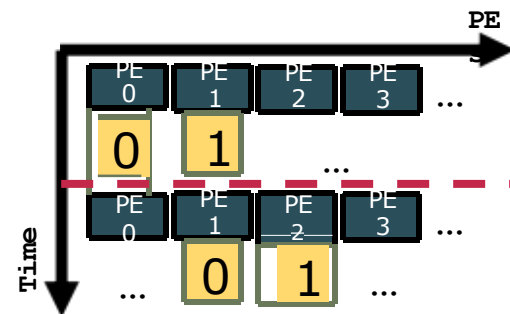
Temporal Reuse



Spatial Reuse

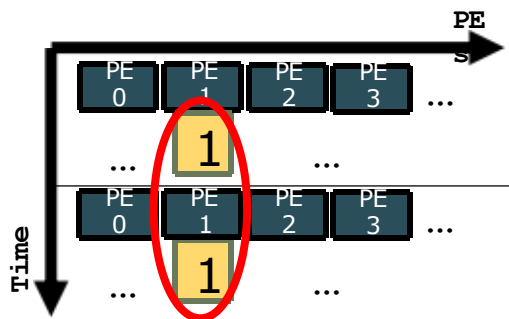


Spatio-Temporal Reuse

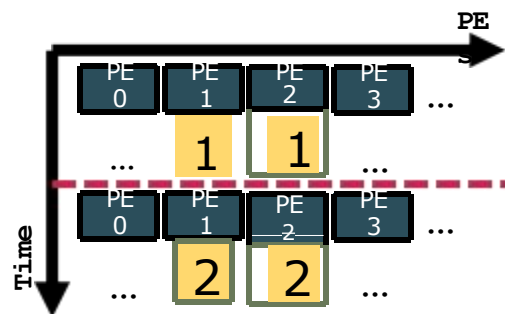


利用数据重用的硬件结构

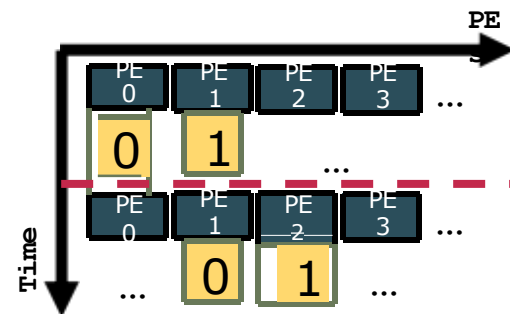
Temporal Reuse



Spatial Reuse

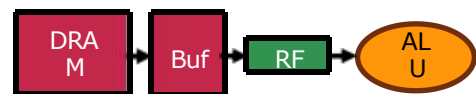
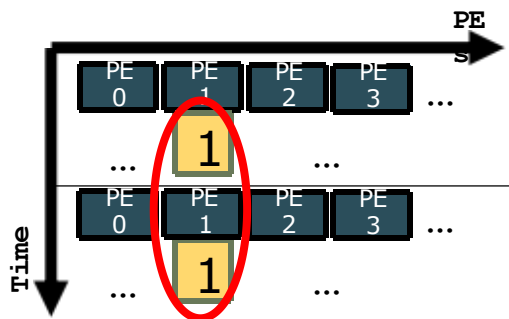


Spatio-Temporal Reuse



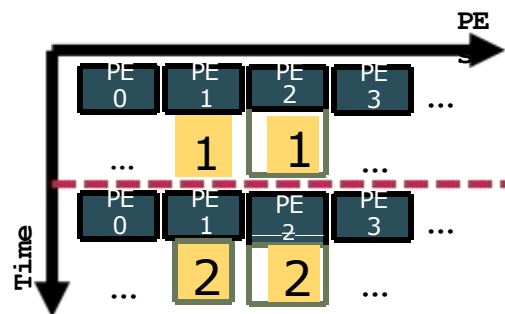
利用数据重用的硬件结构

Temporal Reuse

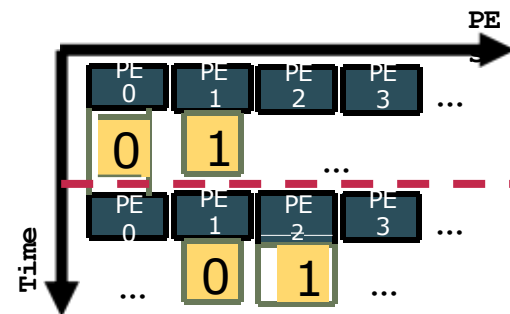


Memory Hierarchy /
Staging Buffers

Spatial Reuse



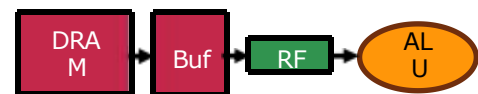
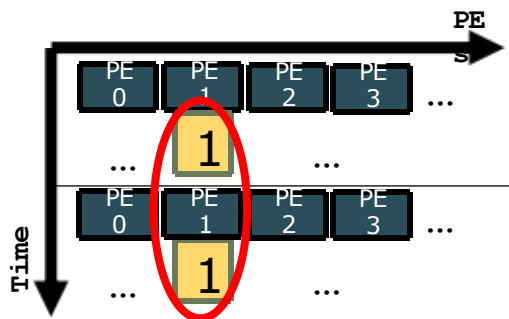
Spatio-Temporal Reuse



传统的memory
hierarchy就能利用时间
局部性！比如cache,
buffer等

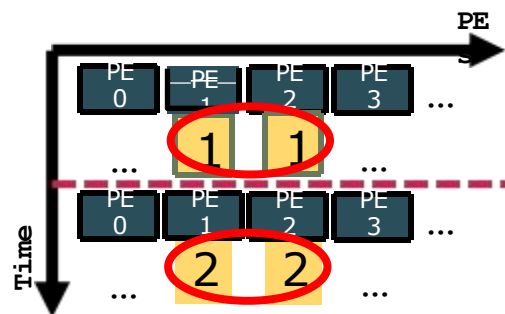
利用数据重用的硬件结构

Temporal Reuse

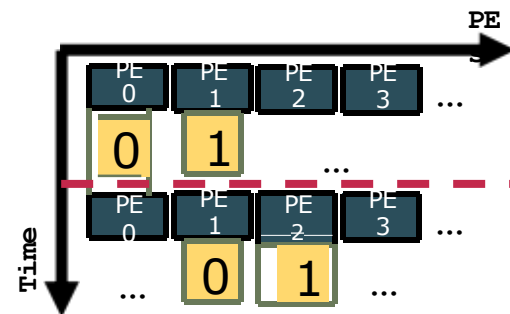


Memory Hierarchy /
Staging Buffers

Spatial Reuse

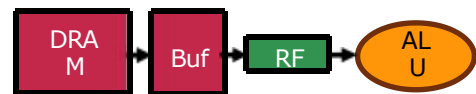
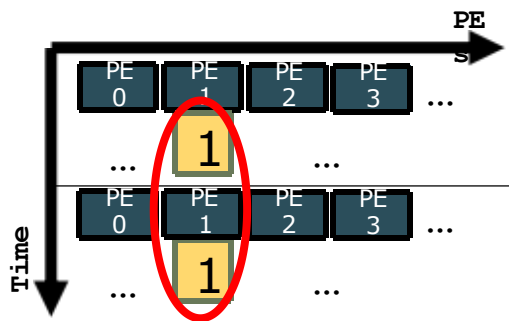


Spatio-Temporal Reuse



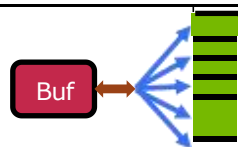
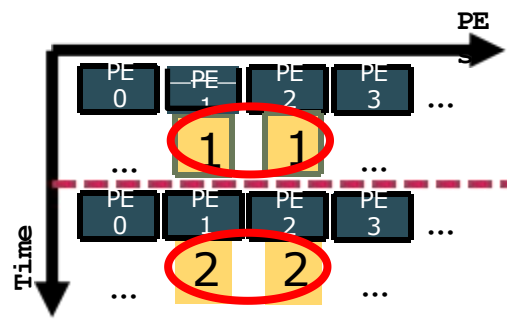
利用数据重用的硬件结构

Temporal Reuse



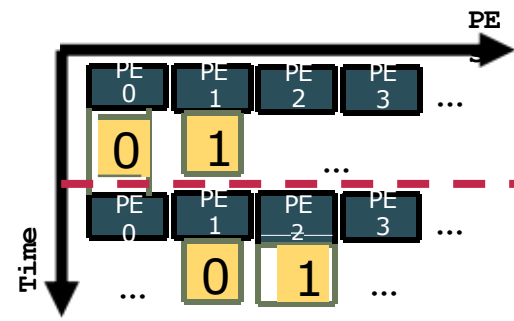
Memory Hierarchy /
Staging Buffers

Spatial Reuse



支持广播的互联网络

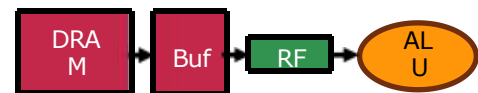
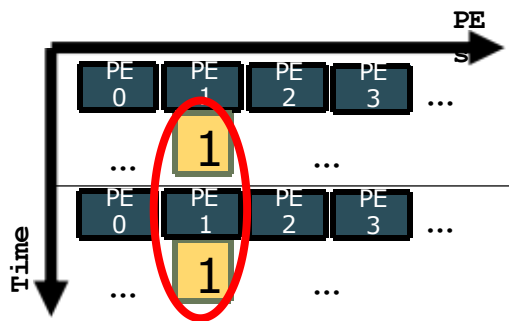
Spatio-Temporal Reuse



通过广播同一个数据可以
被多个PE同时使用

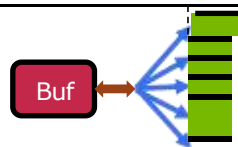
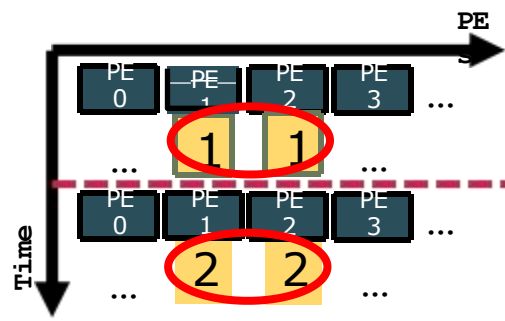
利用数据重用的硬件结构

Temporal Reuse



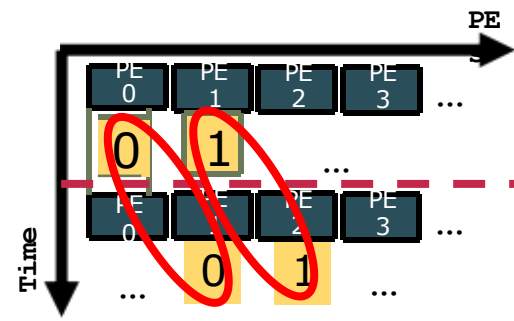
Memory Hierarchy /
Staging Buffers

Spatial Reuse



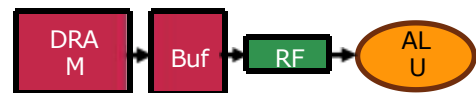
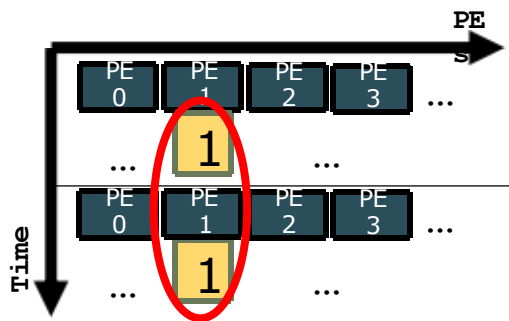
支持广播的互联网络

Spatio-Temporal Reuse



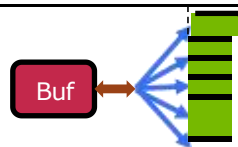
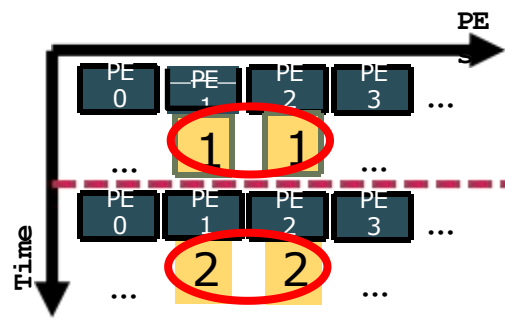
利用数据重用的硬件结构

Temporal Reuse



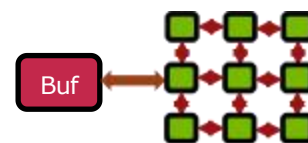
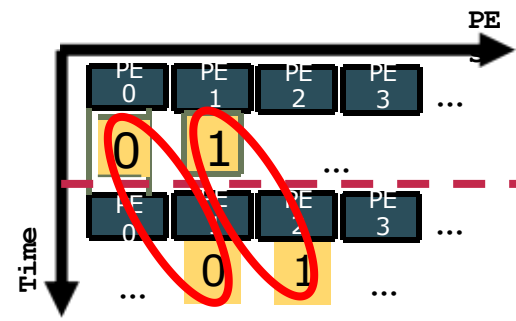
Memory Hierarchy /
Staging Buffers

Spatial Reuse



支持广播的互连网络

Spatio-Temporal Reuse

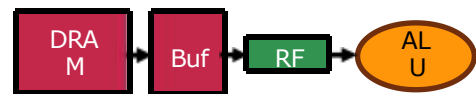
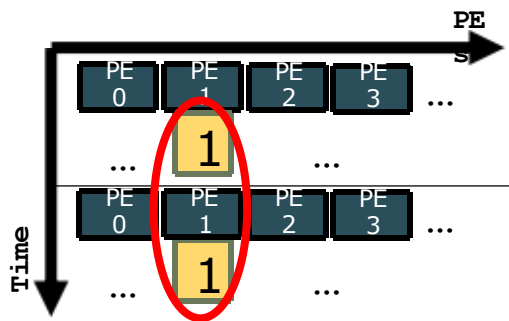


邻居节点之间互联

上一个时间某个PE用的数据可以在下一个时间传递给邻居复用

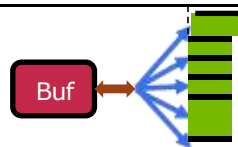
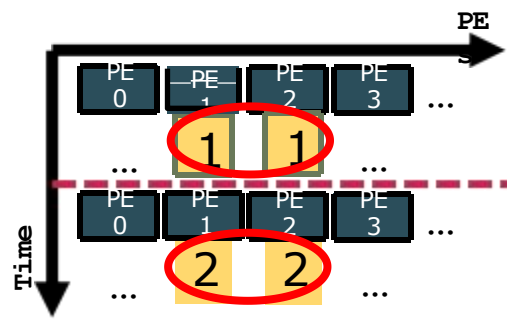
利用数据重用的硬件结构

Temporal Reuse



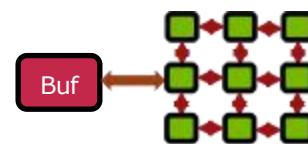
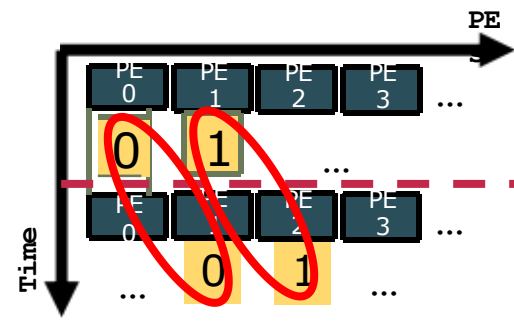
Memory Hierarchy /
Staging Buffers

Spatial Reuse



支持广播的互联网络

Spatio-Temporal Reuse



邻居节点之间互联

硬件结构限制了“映射空间”

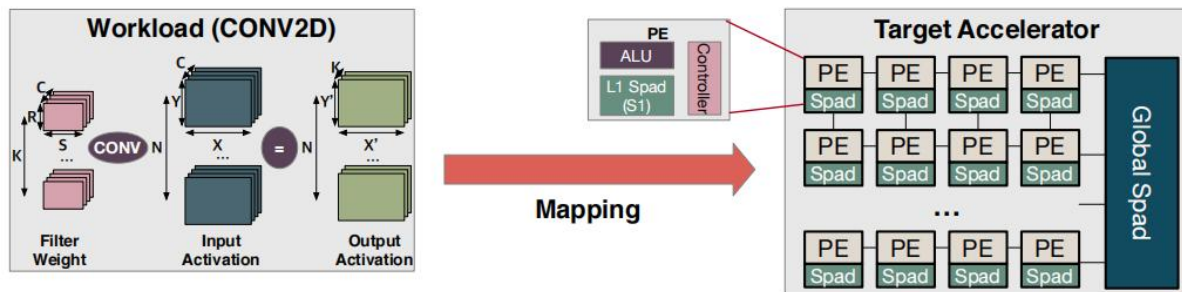
DNN加速器的设计空间

硬件设计空间

映射空间

根据灵活性可由硬件
或软件完成

DNN加速器的设计空间

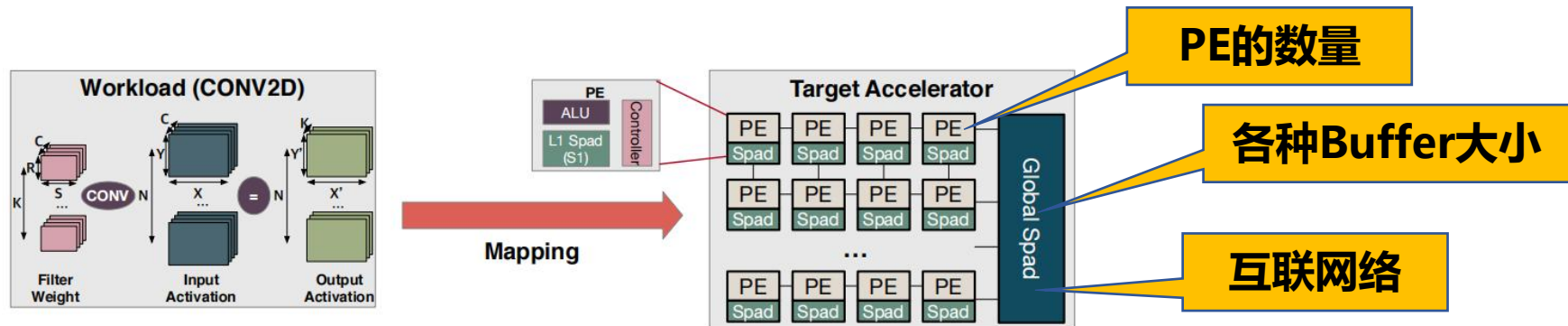


硬件设计空间

映射空间

根据灵活性可由硬件或软件完成

DNN加速器的设计空间

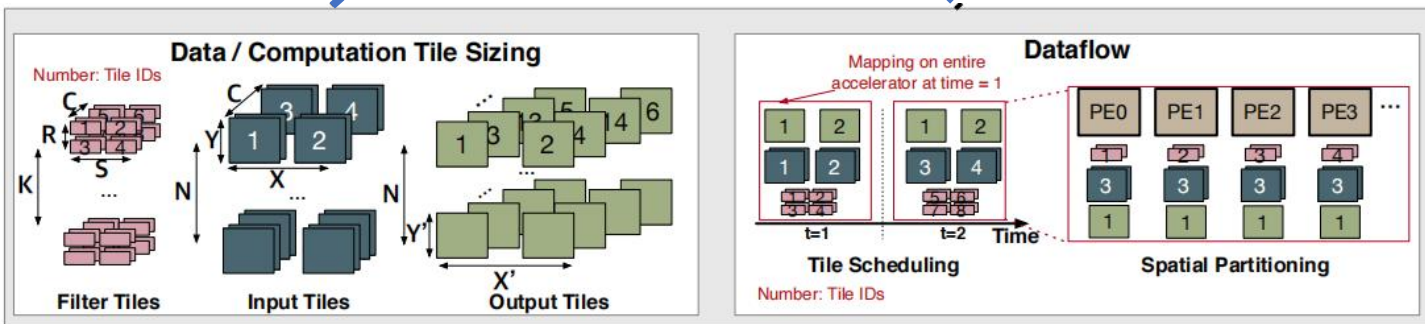
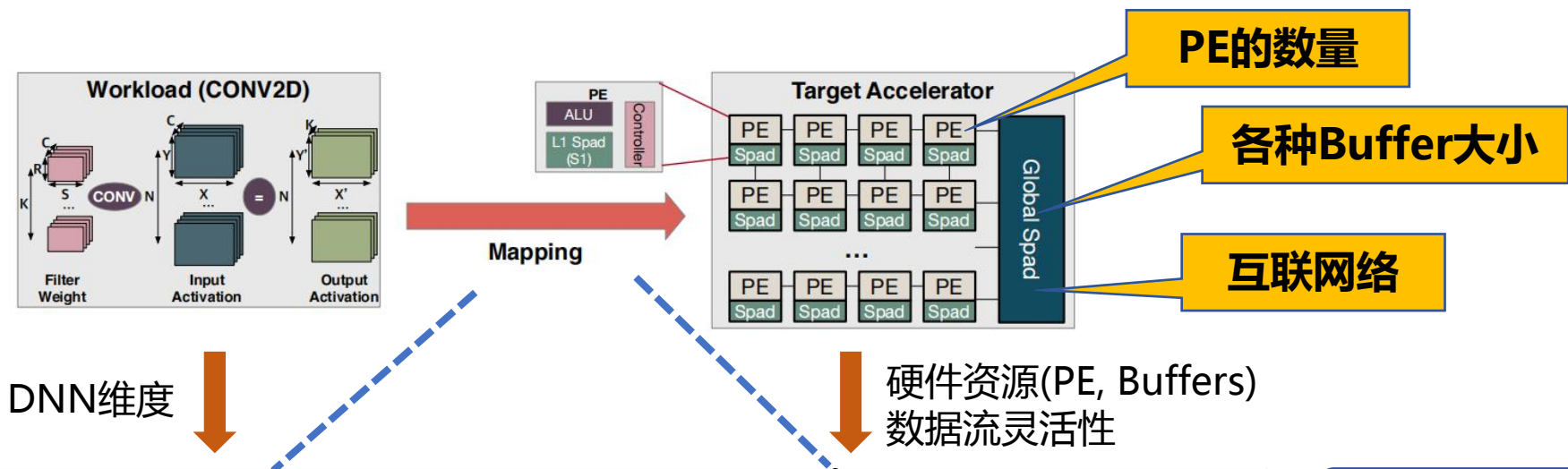


硬件设计空间

映射空间

根据灵活性可由硬件或软件完成

DNN加速器的设计空间

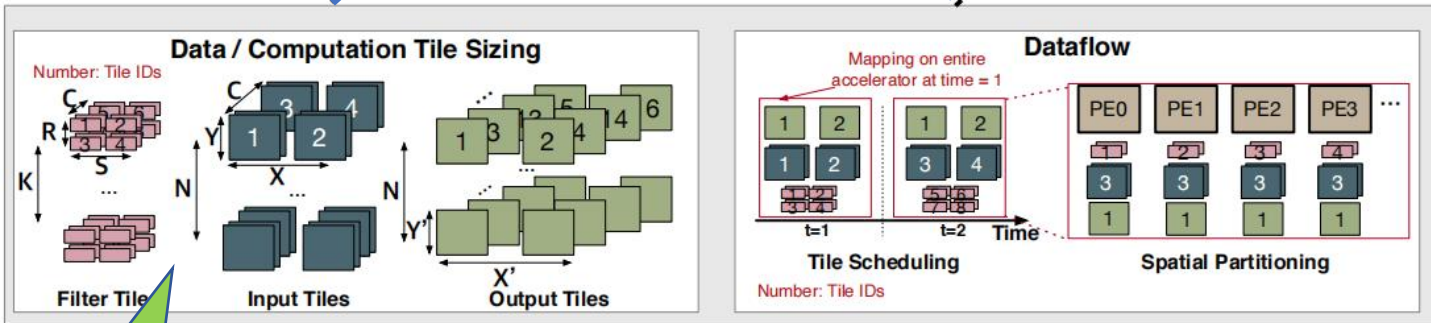
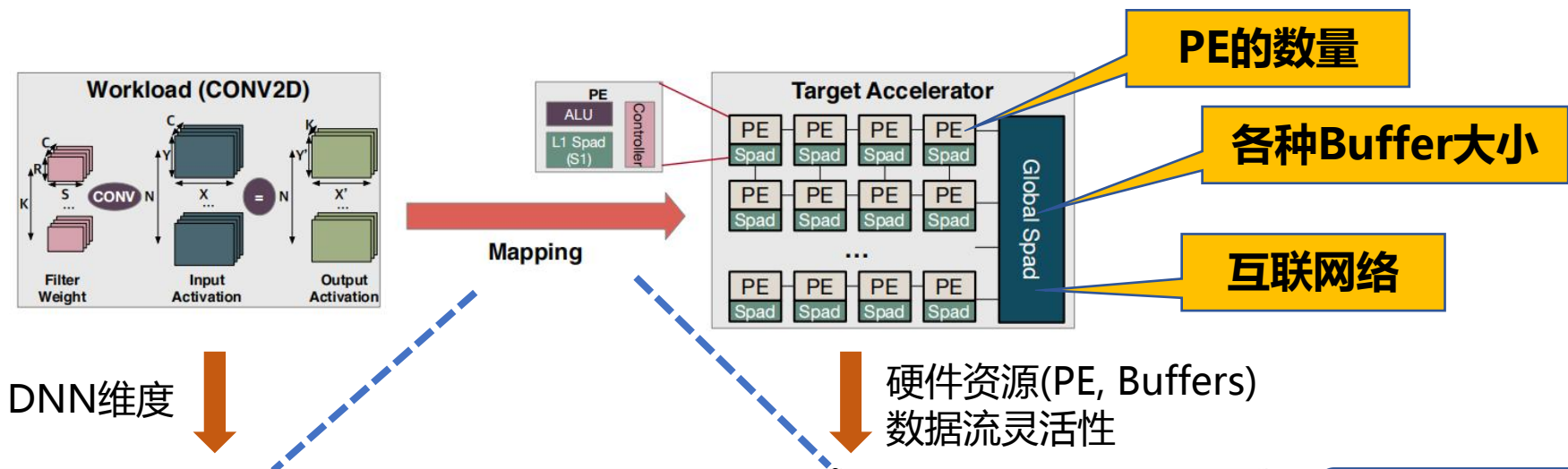


硬件设计空间

映射空间

根据灵活性可由硬件或软件完成

DNN加速器的设计空间

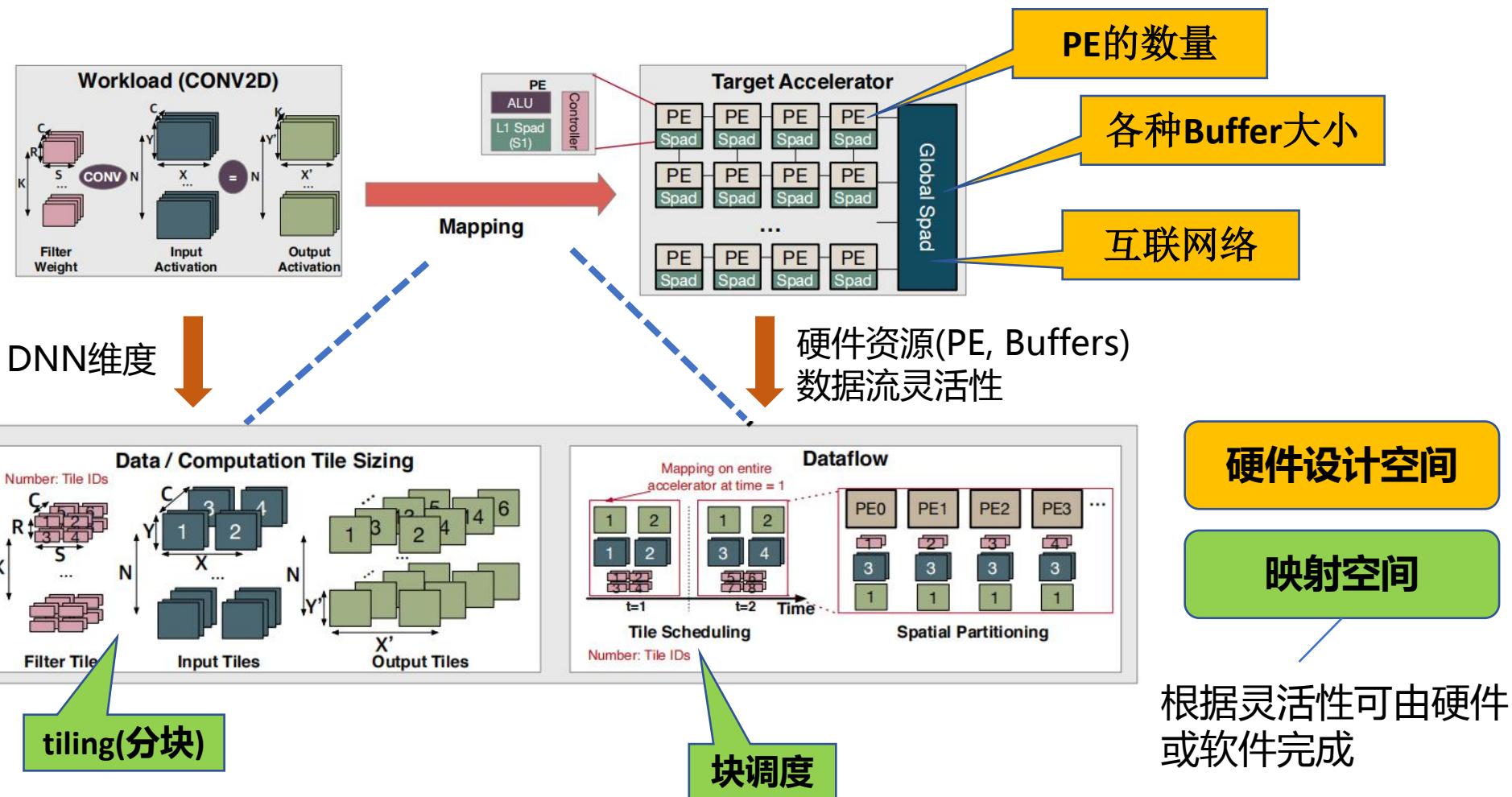


硬件设计空间

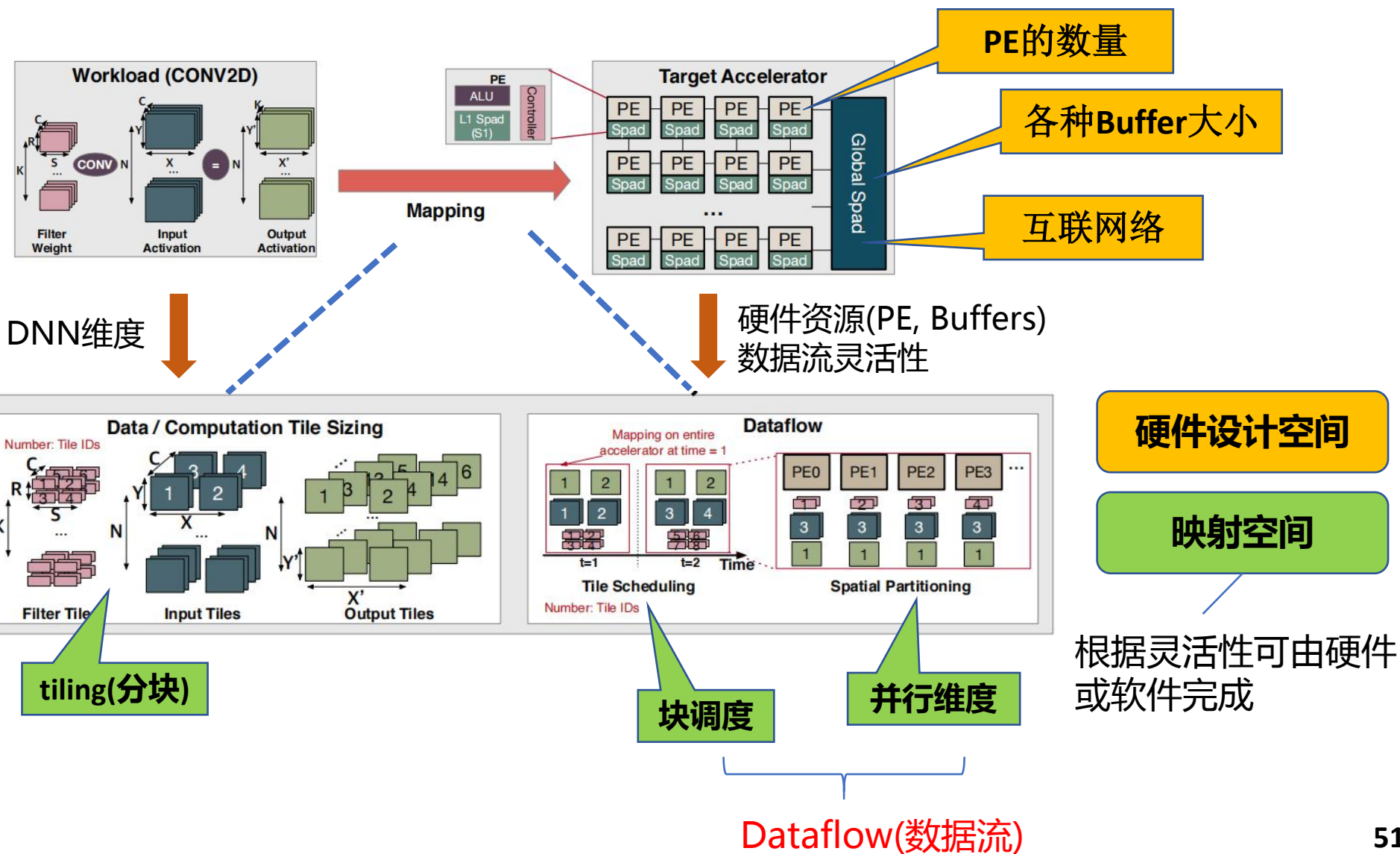
映射空间

根据灵活性可由硬件或软件完成

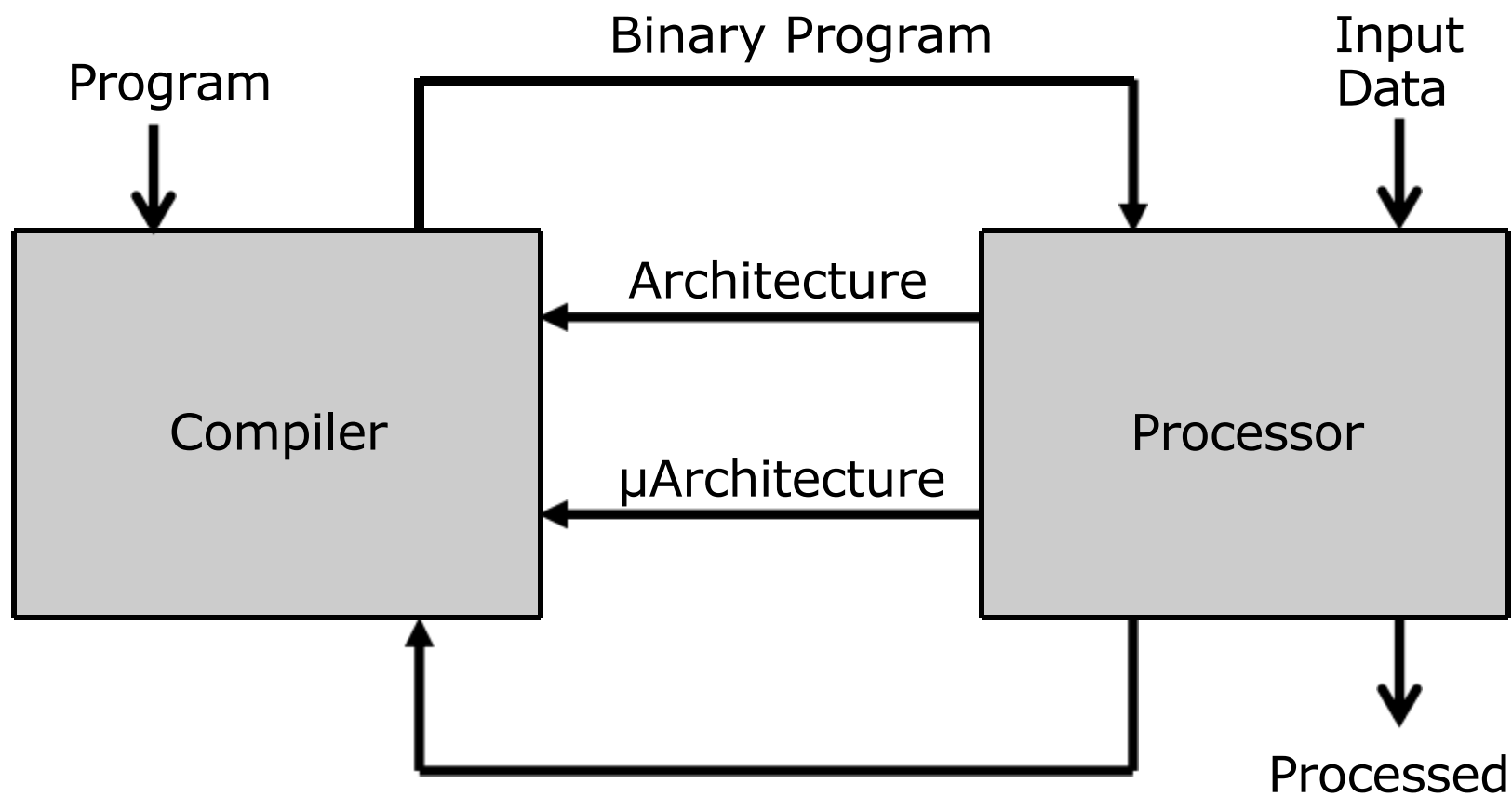
DNN加速器的设计空间



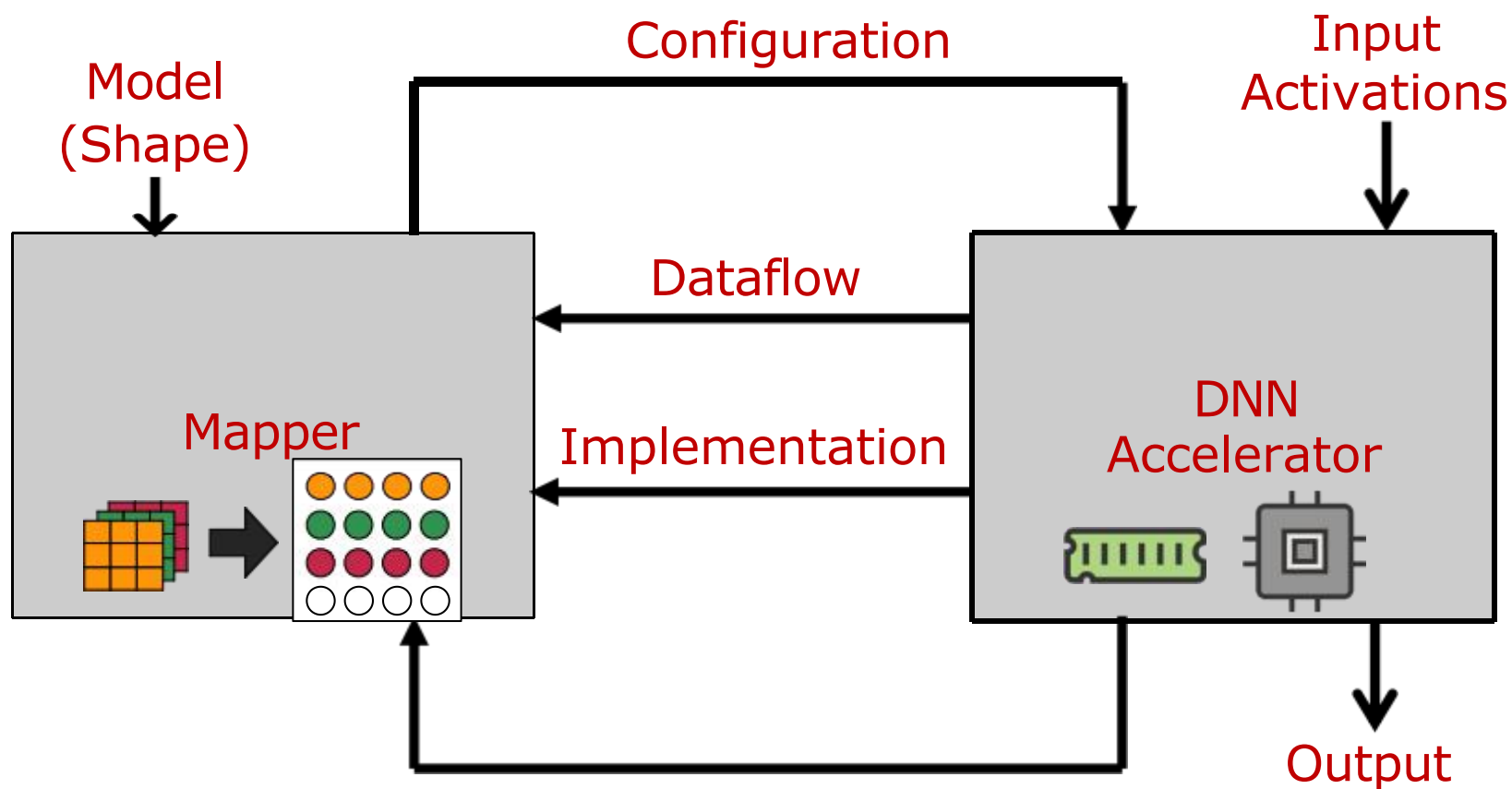
DNN加速器的设计空间



CPU 计算模型

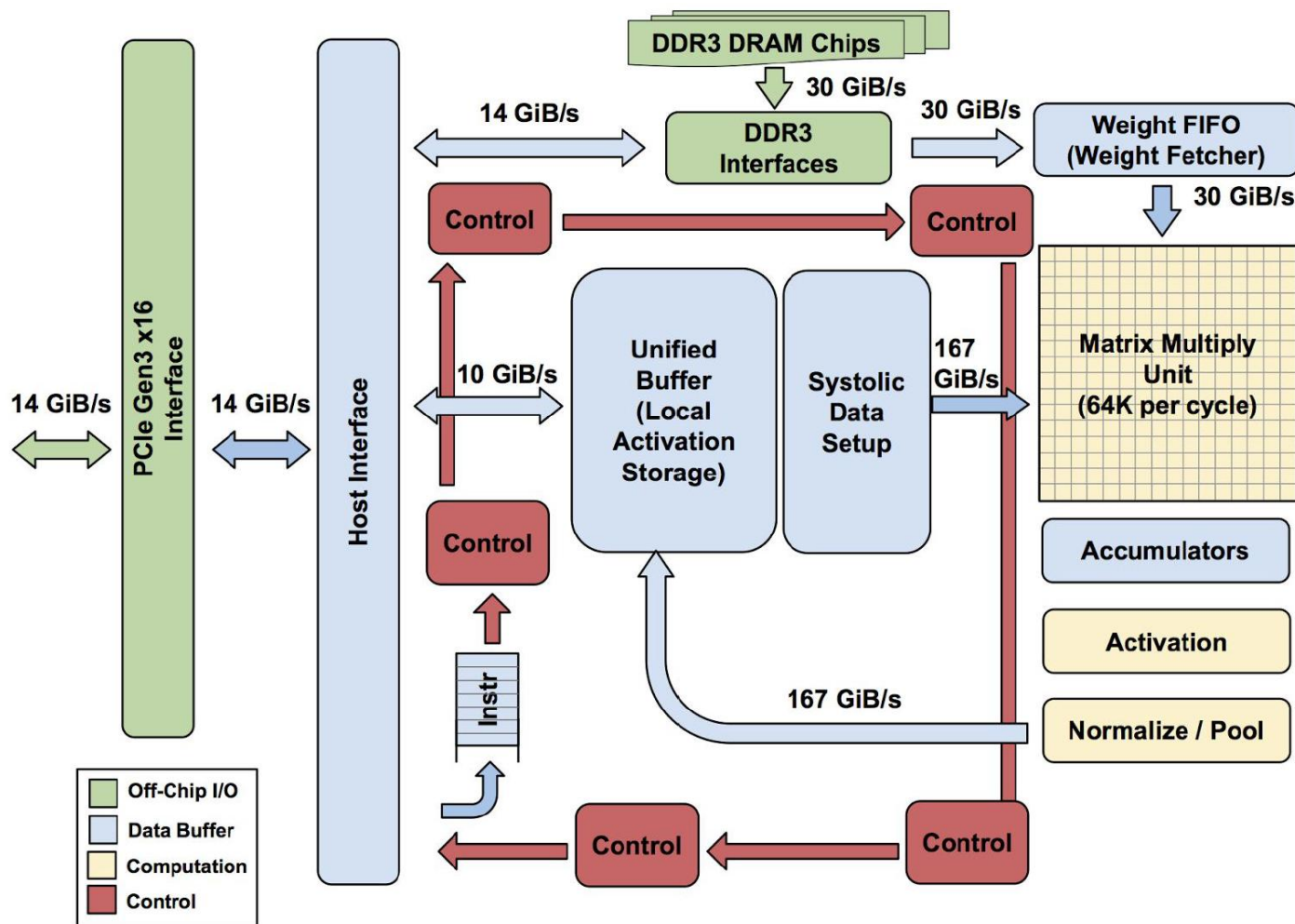


DNN 计算模型



Case Study of Google TPUv1 (2016)

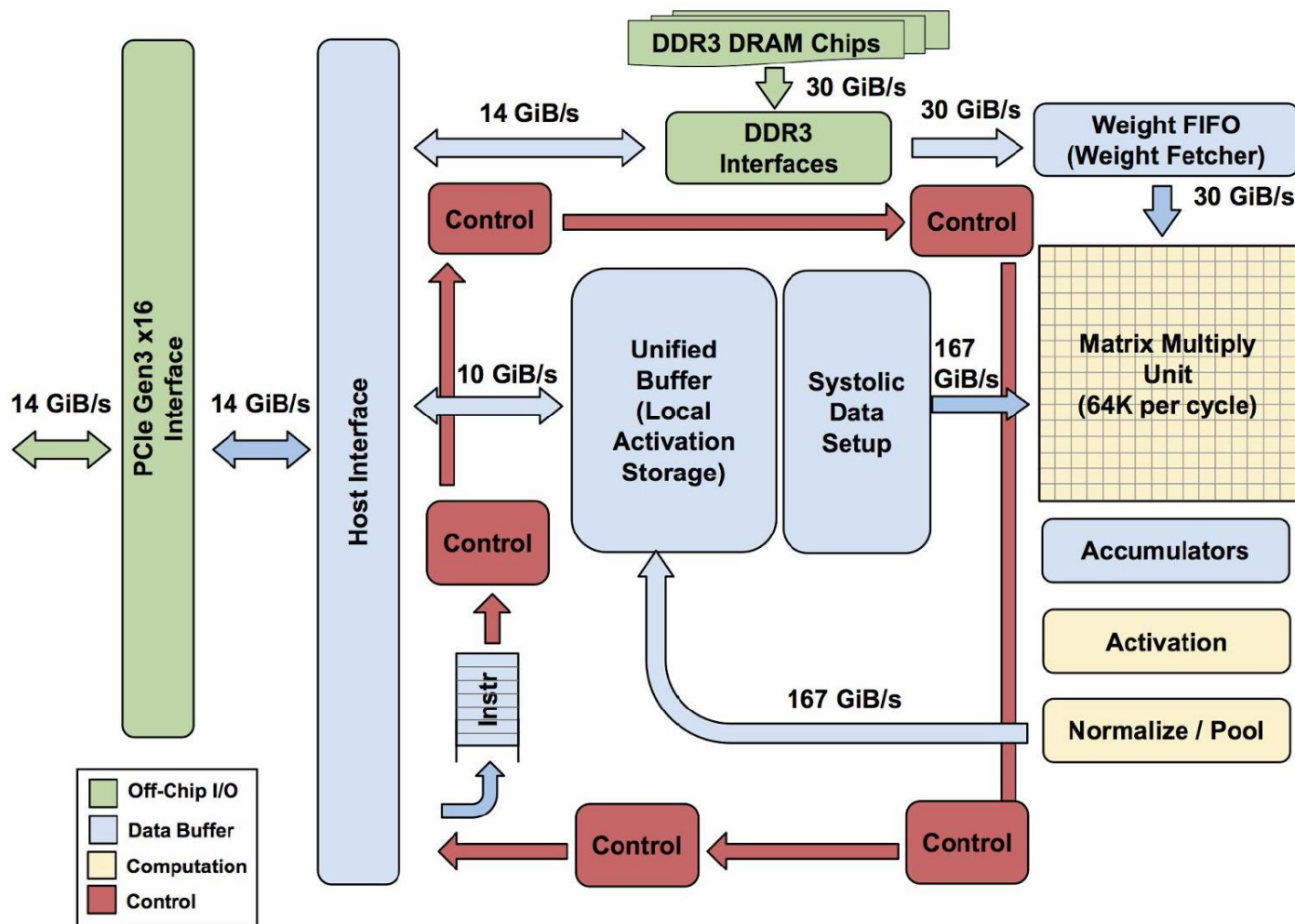
TPU架构非常直观：统一缓冲区（24 MB）存储特征图（初始图像通过PCIe由CPU提供）。这些特征图被送入矩阵乘法单元，这是一个包含64K个乘加器的网格。



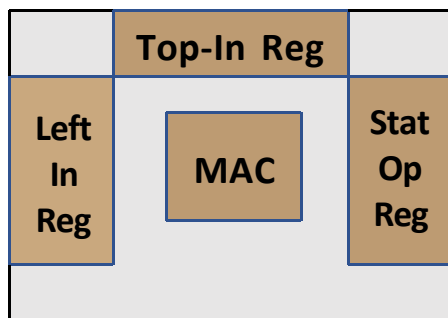
Not to Scale

Case Study of Google TPUv1 (2016)

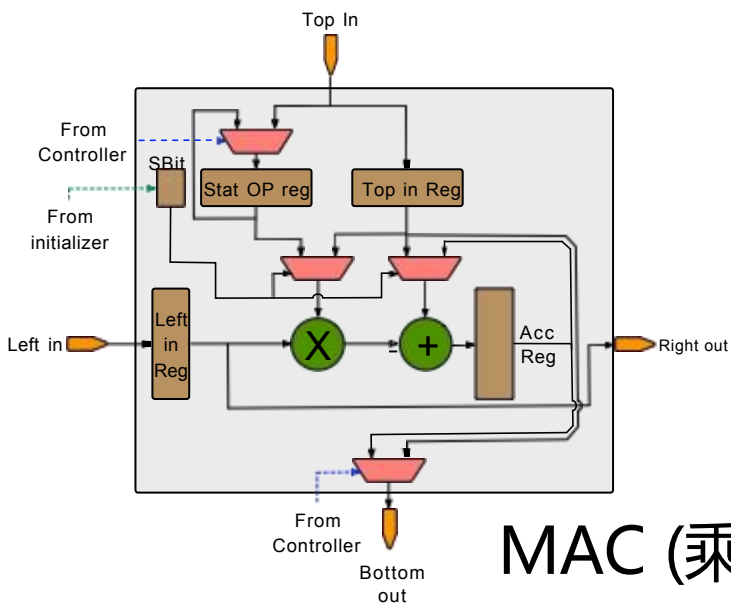
输入特征图然后从左到右流动，并在累加向量处累积。经过激活和池化后，结果被放回统一缓冲区，以便供给下一层



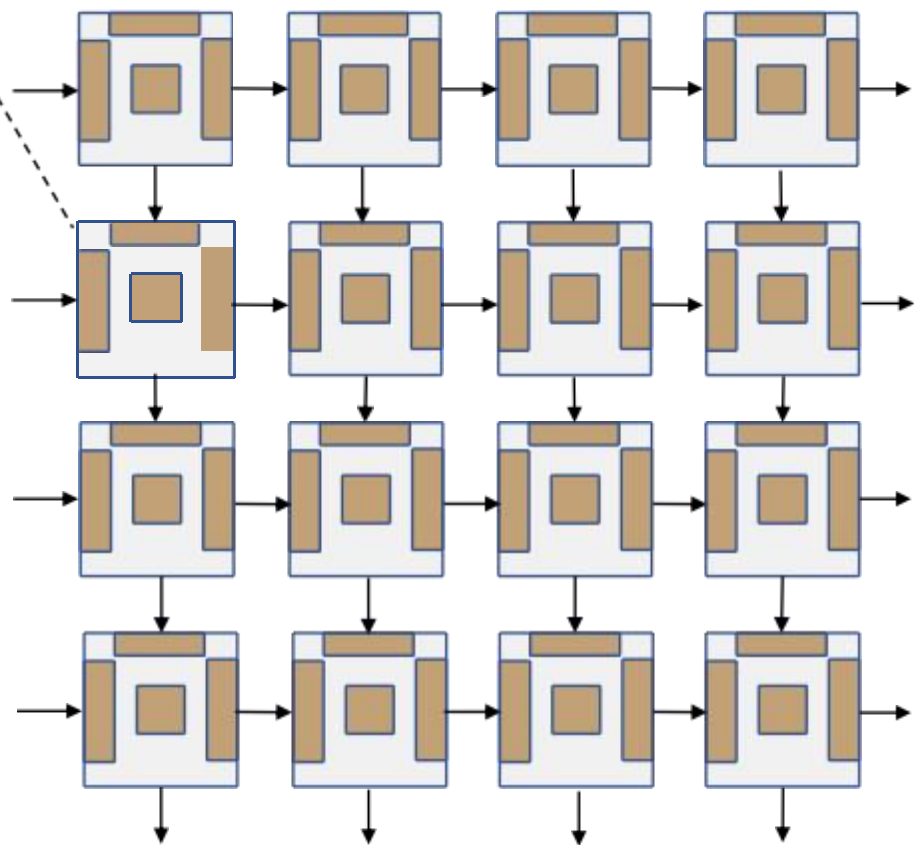
脉冲阵列(Systolic Array)



MAC原理图



MAC (乘加处理单元)



脉冲阵列

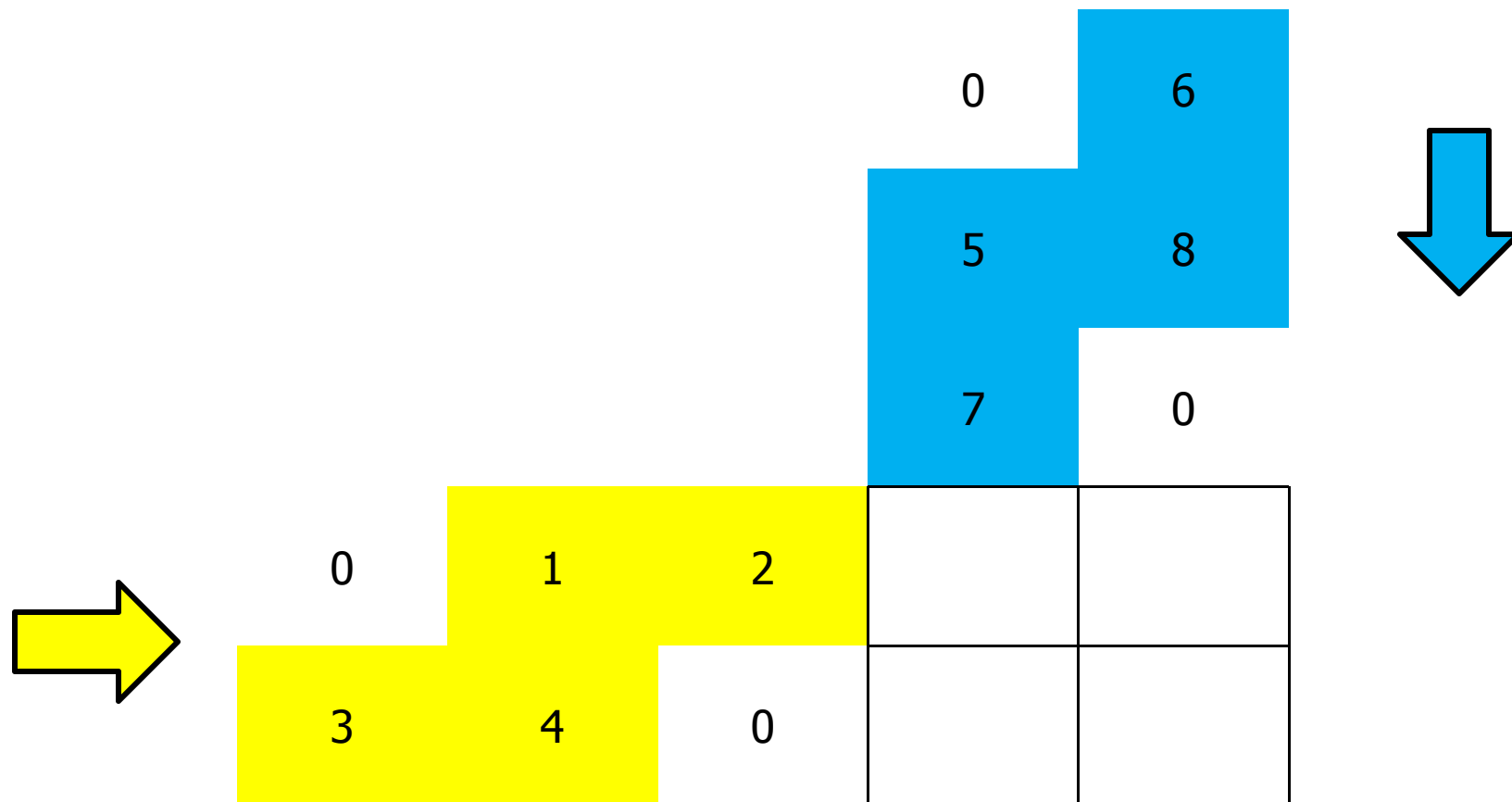
和CPU的区别

- **寄存器分布在PE中**
 - 每个PE的每个操作数1个
- **操作数从一个寄存器传到到另一个**
 - 比读大的寄存器文件能耗更低
- **以确定的方式通过阵列传递数据**
 - 不需要检测数据危害

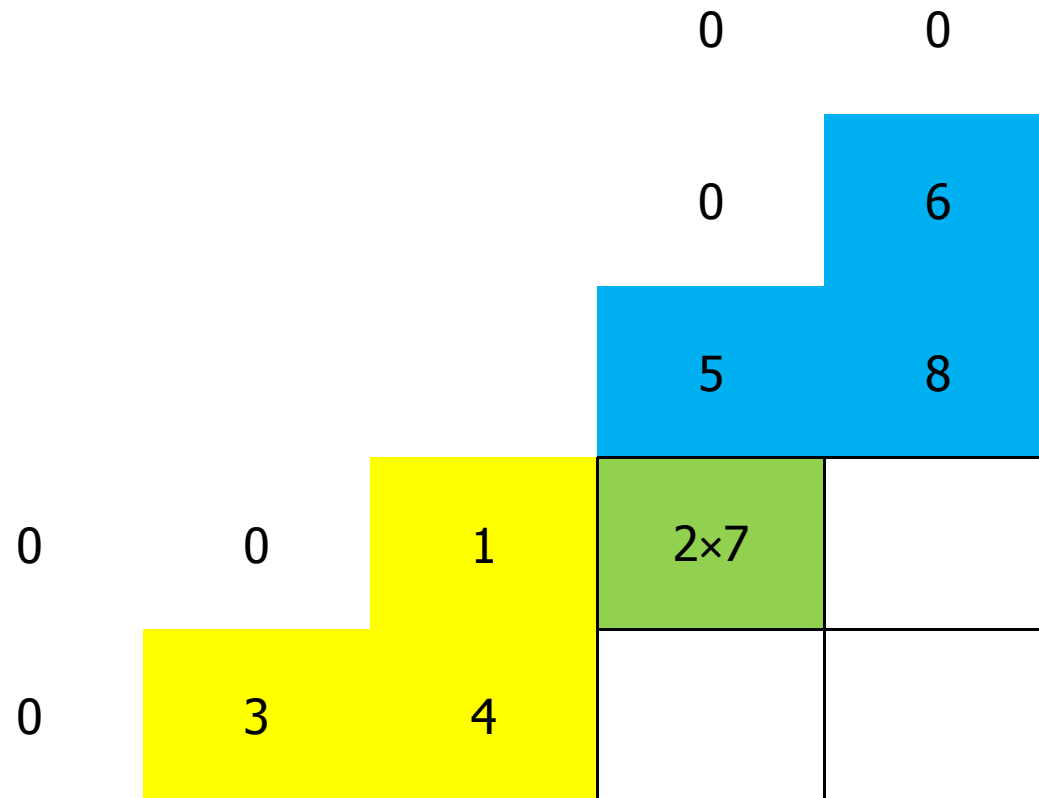
示例 - 计算矩阵相乘

$$\begin{array}{|c|c|} \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} \times \begin{array}{|c|c|} \hline 5 & 6 \\ \hline 7 & 8 \\ \hline \end{array} = \begin{array}{|c|c|} \hline 19 & 22 \\ \hline 43 & 50 \\ \hline \end{array}$$

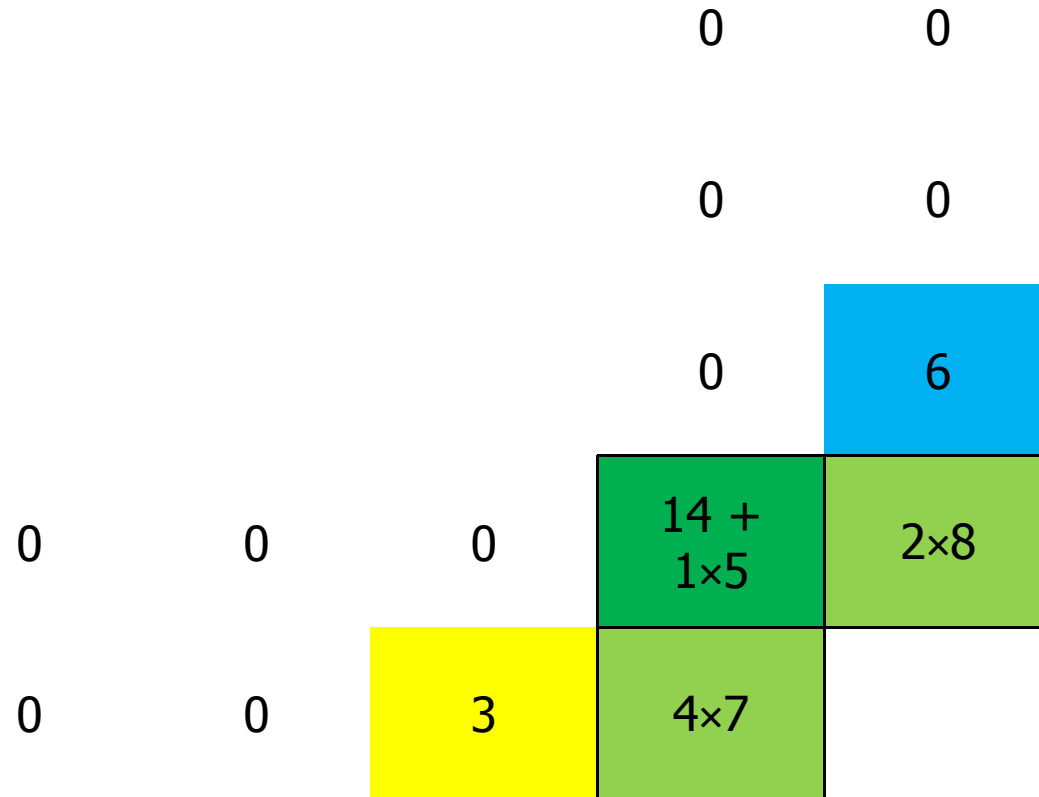
示例 - 计算矩阵相乘



示例 - 计算矩阵相乘



示例 - 计算矩阵相乘



示例 - 计算矩阵相乘

			0	0
			0	0
			0	0
0	0	0	19	16 + 1×6
0	0	0	28 + 3×5	4×8

示例 - 计算矩阵相乘

			0	0
			0	0
			0	0
0	0	0	19	22
0	0	0	43	32 + 3×6

数据流的影响

(Impact of Dataflow)

如何降低带宽需求?

VGG16 conv 3_2	
Multiply Add Ops	1.85 Billion
Weights	590 K
Inputs	803 K
Outputs	803 K

如何降低带宽需求?

VGG16 conv 3_2	
Multiply Add Ops	1.85 Billion
Weights	590 K
Inputs	803 K
Outputs	803 K

数据重用

如何利用数据重用?

如何降低带宽需求?

VGG16 conv 3_2	
Multiply Add Ops	1.85 Billion
Weights	590 K
Inputs	803 K
Outputs	803 K

数据重用

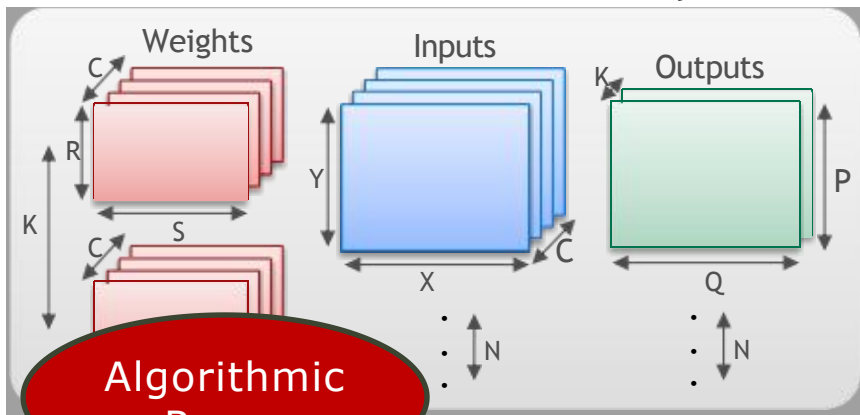
如何利用数据重用? “Dataflow”

细粒度调度计算任务：(1)计算顺序; (2)并行策略

Dataflow的影响:

算法层面的重用 -> 硬件层面的重用

7-dimensional network layer



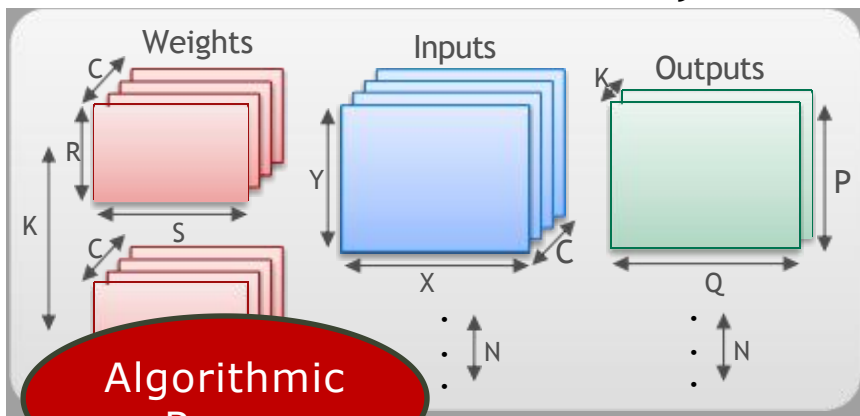
2D hardware array



Dataflow的影响:

算法层面的重用 -> 硬件层面的重用

7-dimensional network layer



2D hardware array

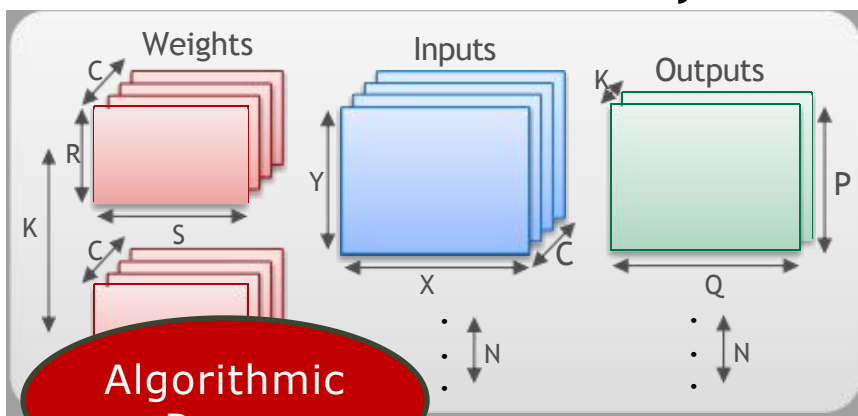


- **7D 计算空间**
 - $R * S * X * Y * C * K * N$
- **4D 操作数/结果数据空间**
 - Weights - $R * S * C * K$
 - Inputs - $X * Y * C * N$
 - Outputs - $P * Q * K * N$

Dataflow的影响:

算法层面的重用 -> 硬件层面的重用

7-dimensional network layer



- **7D 计算空间**
 - $R * S * X * Y * C * K * N$
- **4D 操作数/结果数据空间**
 - Weights - $R * S * C * K$
 - Inputs - $X * Y * C * N$
 - Outputs - $P * Q * K * N$



2D hardware array

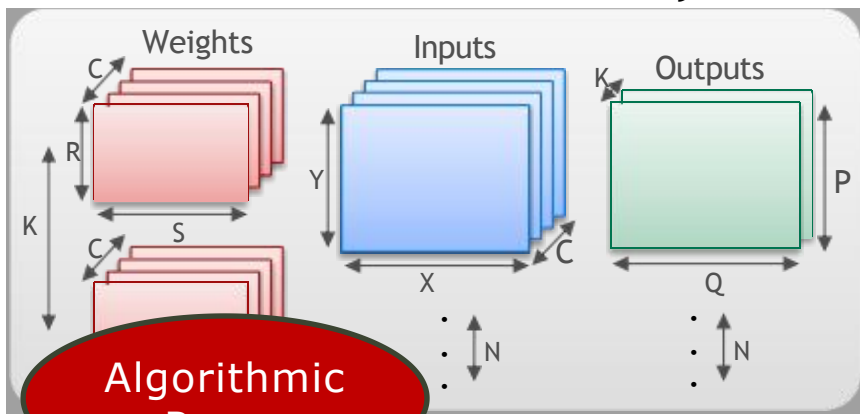


- **硬件设计空间**
 - PE的数量
 - 内存架构(空间和带宽)
 - 互连网络

Dataflow的影响:

算法层面的重用 -> 硬件层面的重用

7-dimensional network layer



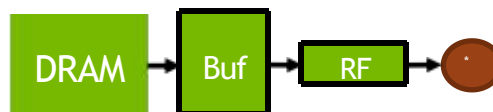
- **7D 计算空间**
 - $R * S * X * Y * C * K * N$
- **4D 操作数/结果数据空间**
 - Weights - $R * S * C * K$
 - Inputs - $X * Y * C * N$
 - Outputs - $P * Q * K * N$

2D hardware array



- **硬件设计空间**
 - PE的数量
 - 内存架构(空间和带宽)
 - 互连网络

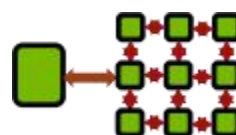
- **硬件重用结构**
时间重用



空间重用
(Multicast)



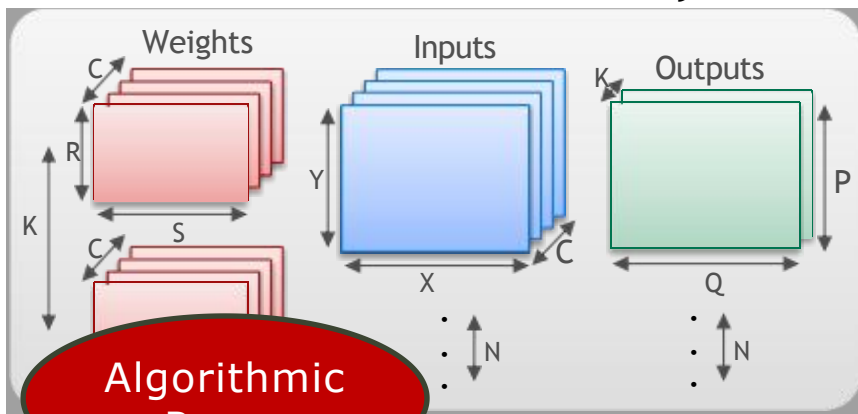
空间重用
(Forwarding)



Dataflow的影响:

算法层面的重用 -> 硬件层面的重用

7-dimensional network layer



Algorithmic Reuse

- 7D 计算空间
 - $R * S * X * Y * C * K * N$
- 4D 操作数/结果数据空间 -
 - Weights - $R * S * C * K$
 - Inputs - $X * Y * C * N$
 - Outputs - $P * Q * K * N$

如何描述并指导硬件设计?

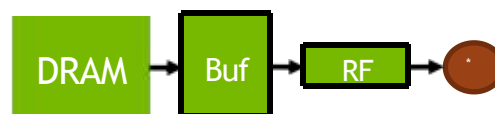
2D hardware array



Hardware Reuse

- 硬件设计空间
 - PE的数量
 - 内存架构(空间和带宽)
 - 互连网络

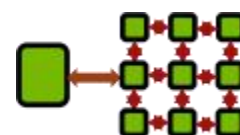
- 硬件重用结构
 - 时间重用



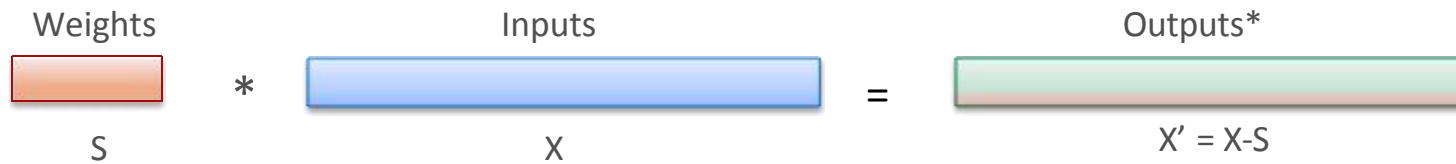
空间重用
(Multicast)



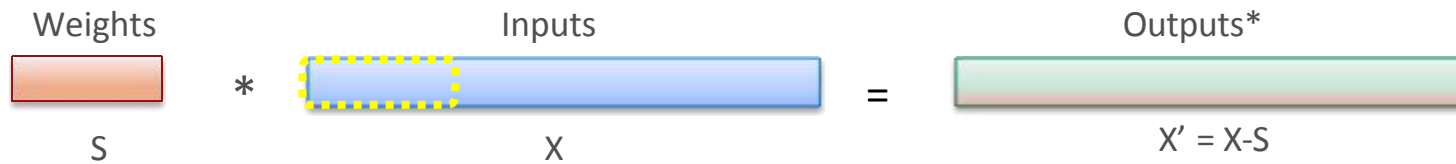
空间重用
(Forwarding)



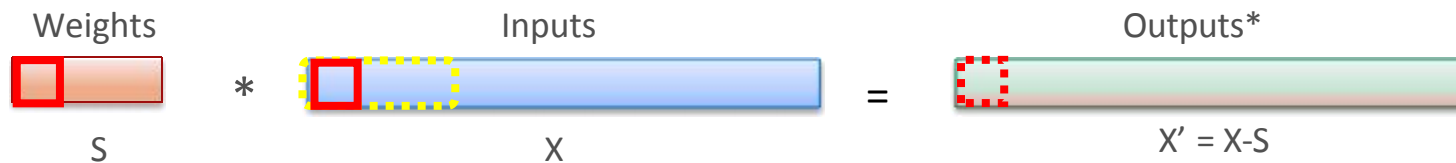
输出固定的数据流(Output Stationary, OS)



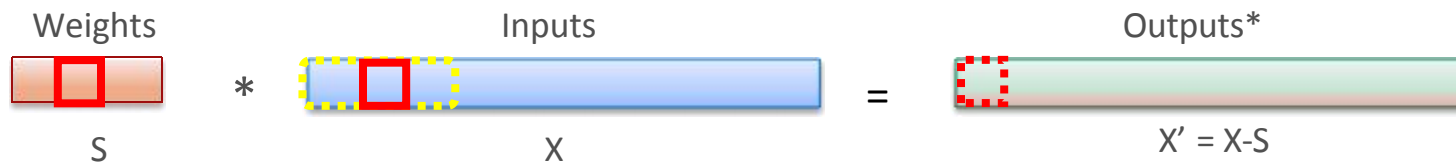
输出固定的数据流(Output Stationary, OS)



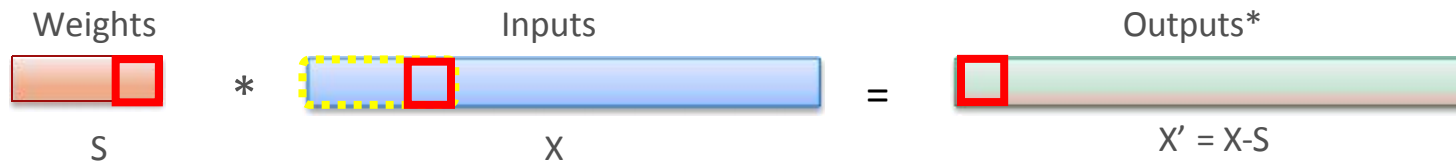
输出固定的数据流(Output Stationary, OS)



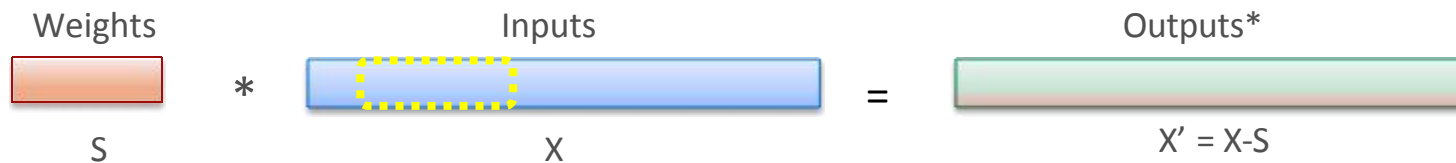
输出固定的数据流(Output Stationary, OS)



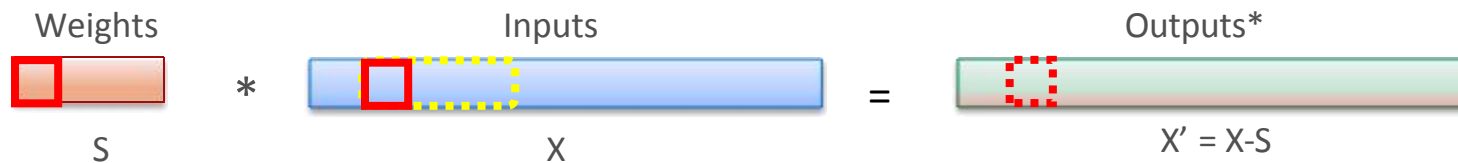
输出固定的数据流(Output Stationary, OS)



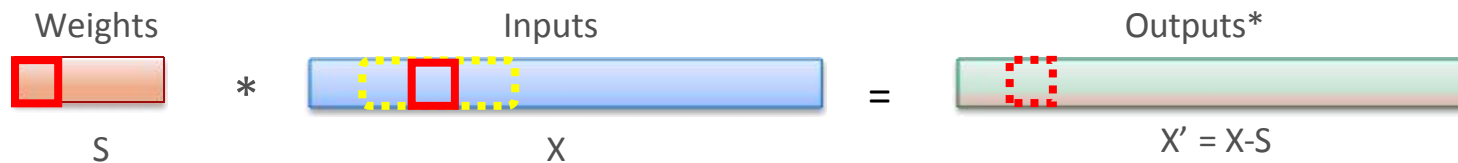
输出固定的数据流(Output Stationary, OS)



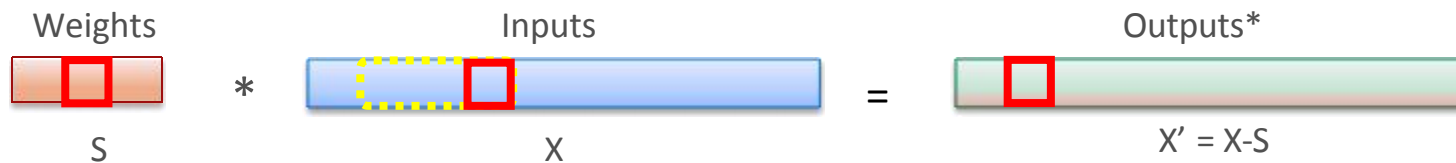
输出固定的数据流(Output Stationary, OS)



输出固定的数据流(Output Stationary, OS)



输出固定的数据流(Output Stationary, OS)



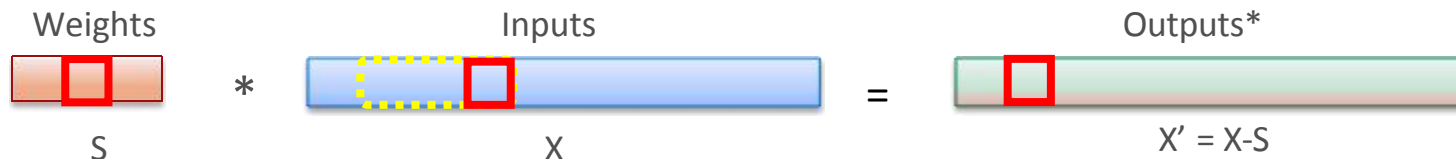
输出固定的数据流(Output Stationary, OS)



Computation

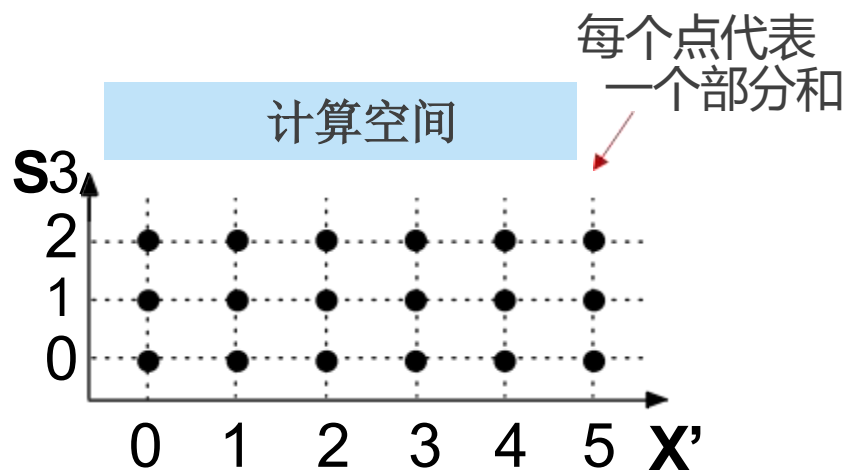
```
for(int x = 0; x < X'; x++)  
  for(int s = 0; s < S; s++)  
    Output[x] += Weight[s] * Input[x+s]
```

输出固定的数据流(Output Stationary, OS)

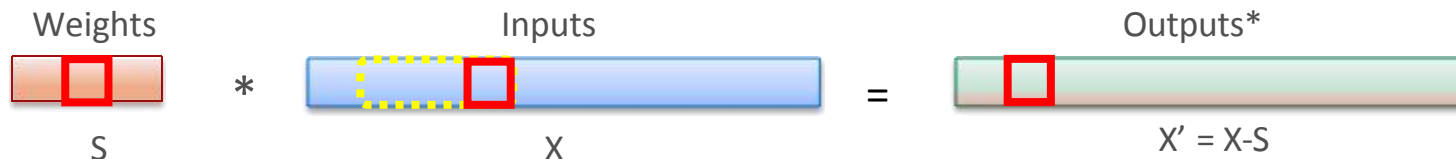


Computation

```
for(int x = 0; x < X'; x++)  
  for(int s = 0; s < S; s++)  
    Output[x] += Weight[s] * Input[x+s]
```



输出固定的数据流(Output Stationary, OS)



Computation

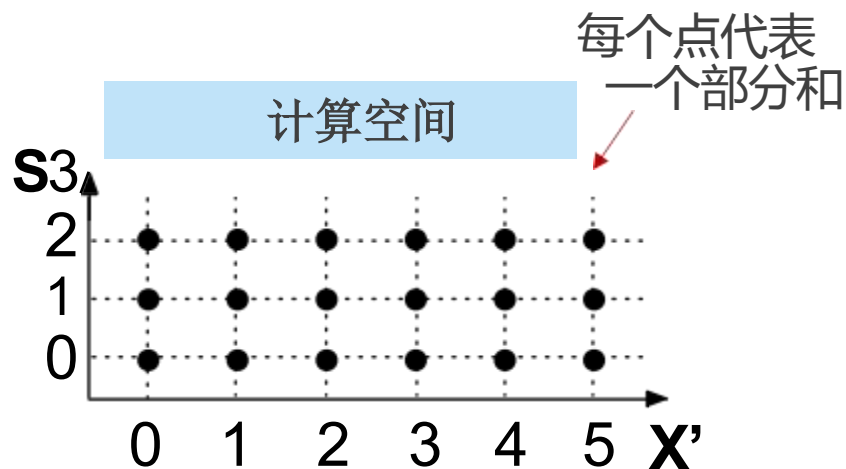
```
for(int x = 0; x < X'; x++)  
  for(int s = 0; s < S; s++)  
    Output[x] += Weight[s] * Input[x+s]
```



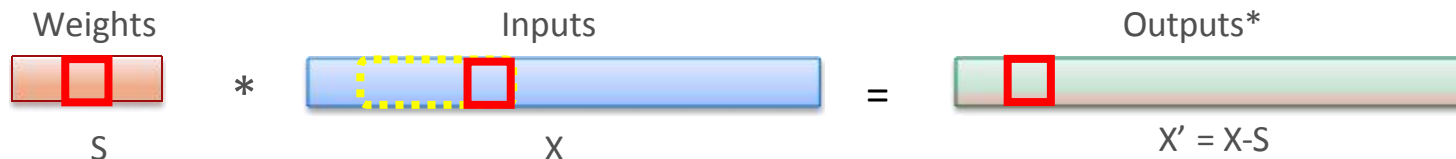
Data

PartialSum[X'][S] needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]



输出固定的数据流(Output Stationary, OS)



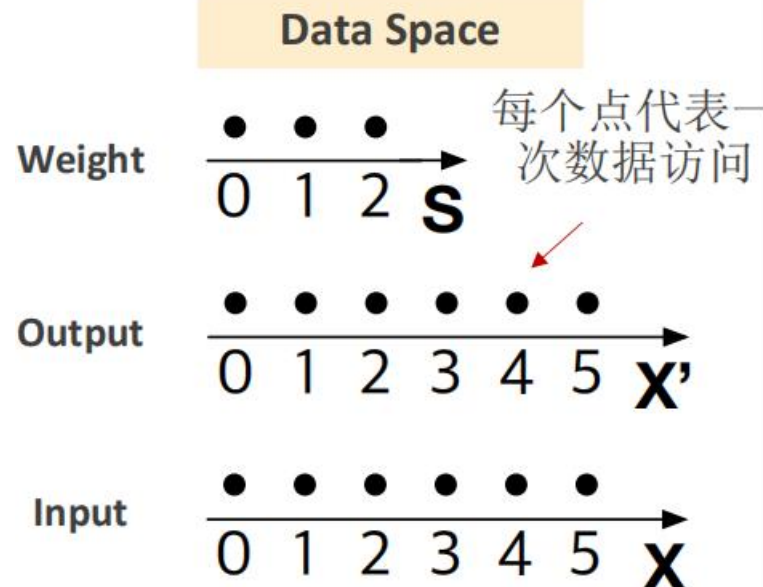
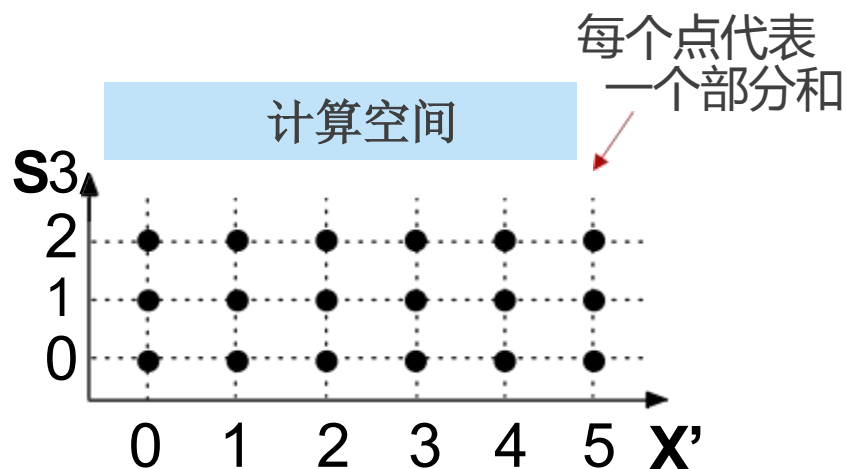
Computation

```
for(int x = 0; x < X'; x++)  
  for(int s = 0; s < S; s++)  
    Output[x] += Weight[s] * Input[x+s]
```

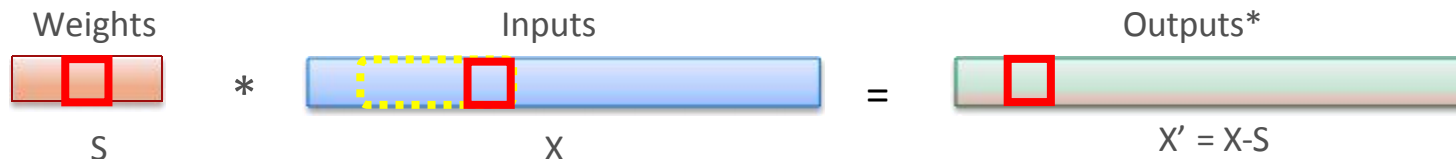
Data

PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]



输出固定的数据流(Output Stationary, OS)



Computation

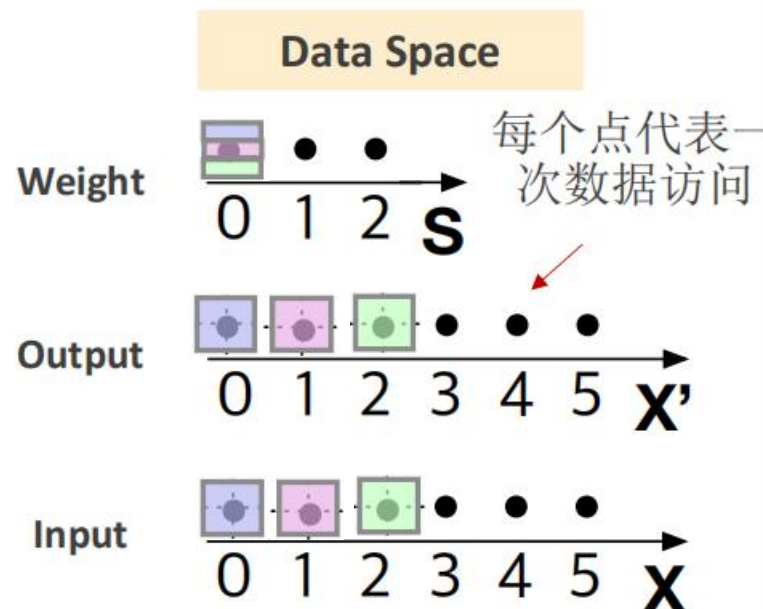
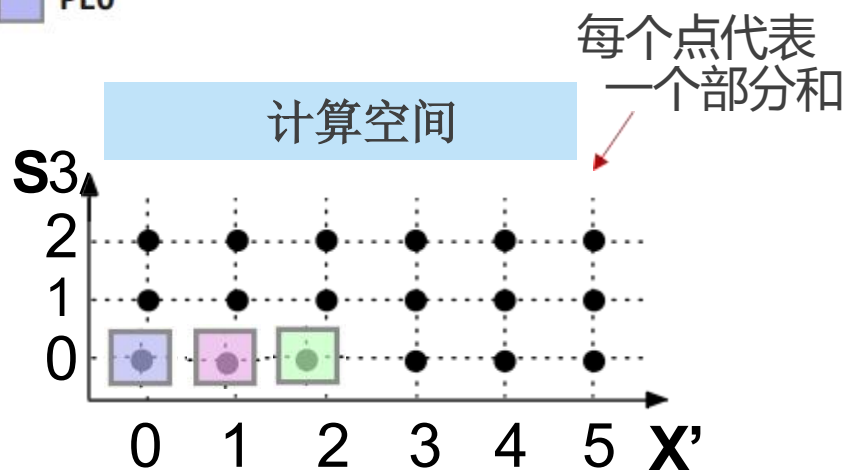
```
for(int x = 0; x < X'; x++)
    for(int s = 0; s < S; s++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

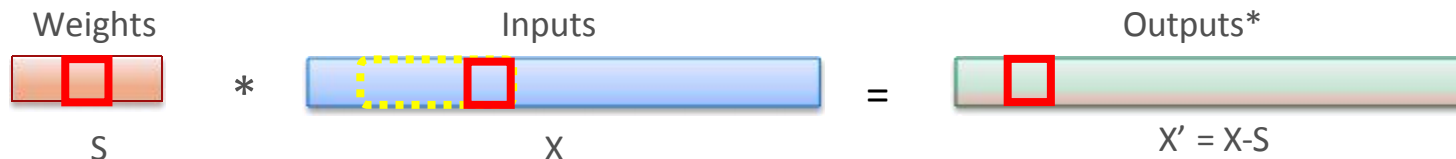
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

PE2
PE1
PE0



输出固定的数据流(Output Stationary, OS)



Computation

```
for(int x = 0; x < X'; x++)
    for(int s = 0; s < S; s++)
        Output[x] += Weight[s] * Input[x+s]
```

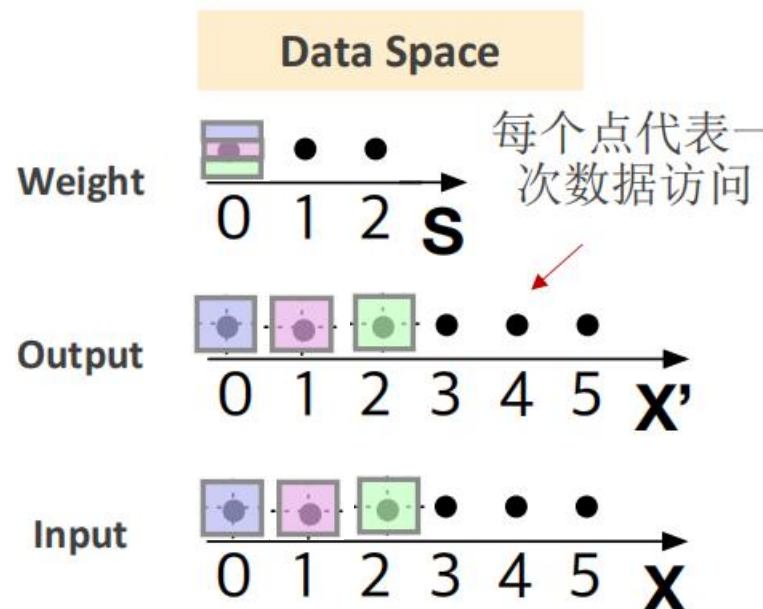
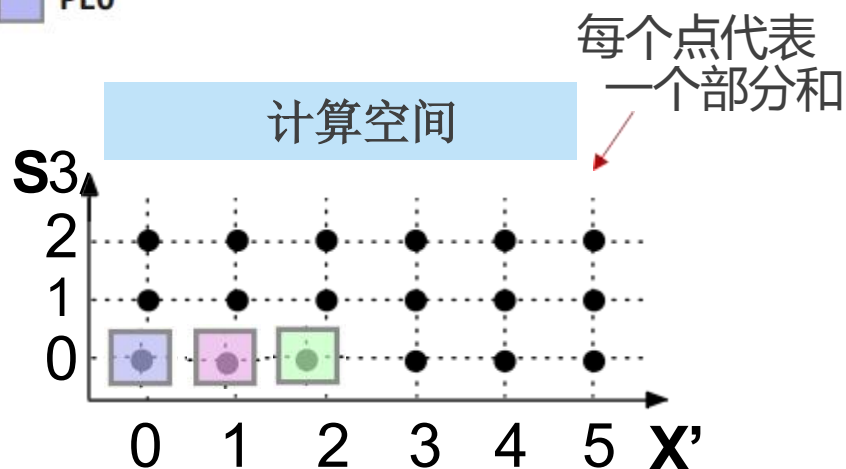
Data

PartialSum[X'] $[S]$ needs to access:

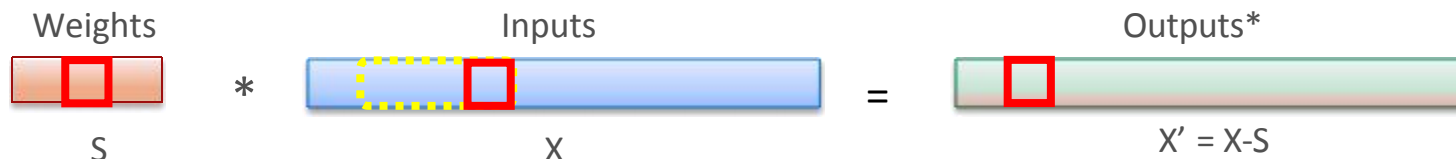
- Weight[s]
- Output[x']
- Input[x'+s]

PE2
PE1
PE0

Time = 0



输出固定的数据流(Output Stationary, OS)



Computation

```
for(int x = 0; x < X'; x++)
  for(int s = 0; s < S; s++)
    Output[x] += Weight[s] * Input[x+s]
```

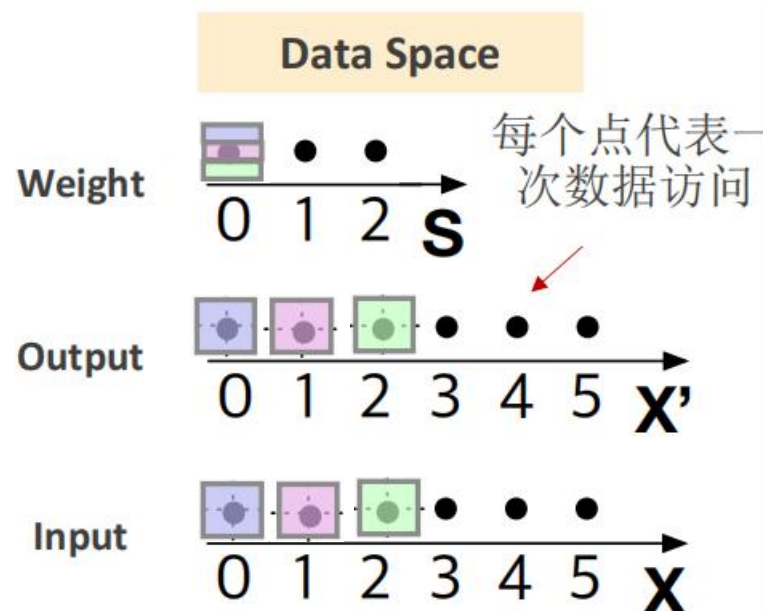
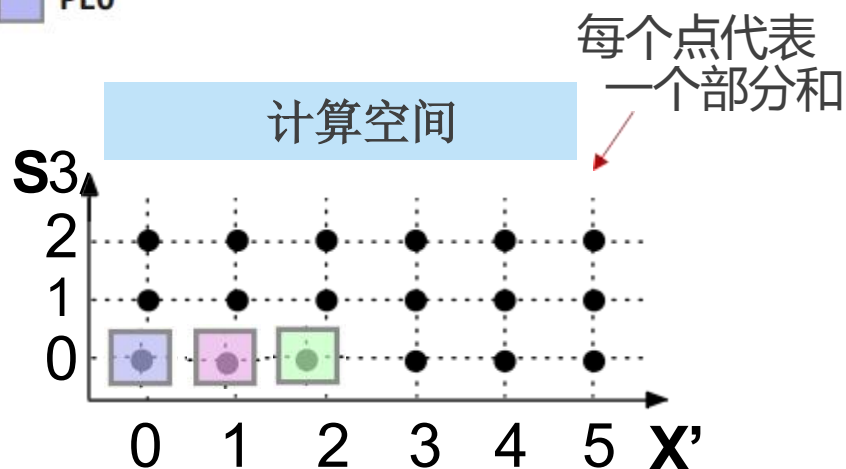
Data

PartialSum[X'][S] needs to access:

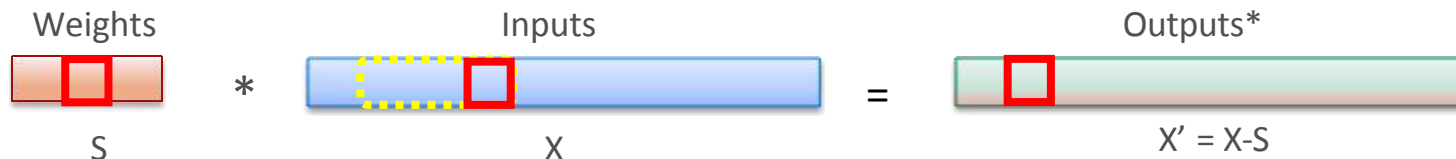
- Weight[s]
- Output[x']
- Input[x'+s]

PE2
PE1
PE0

Time = 1



输出固定的数据流(Output Stationary, OS)



Computation

```
for(int x = 0; x < X'; x++)
  for(int s = 0; s < S; s++)
    Output[x] += Weight[s] * Input[x+s]
```

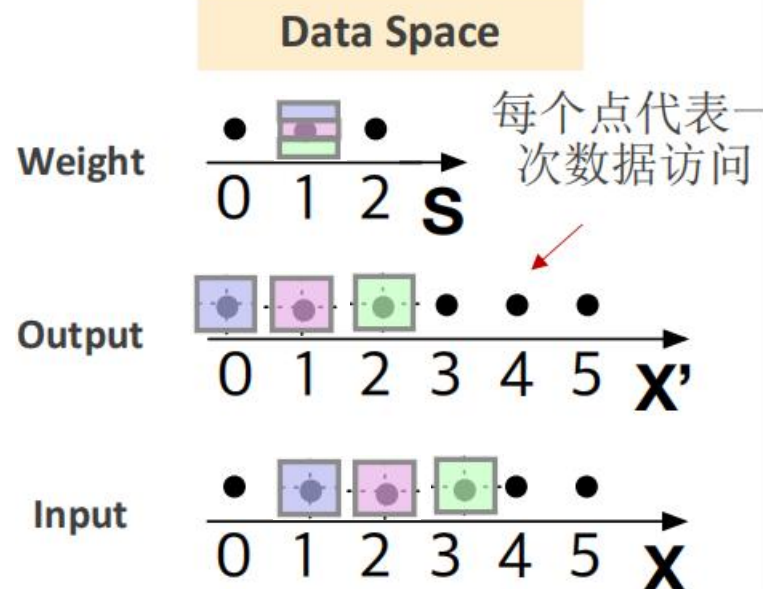
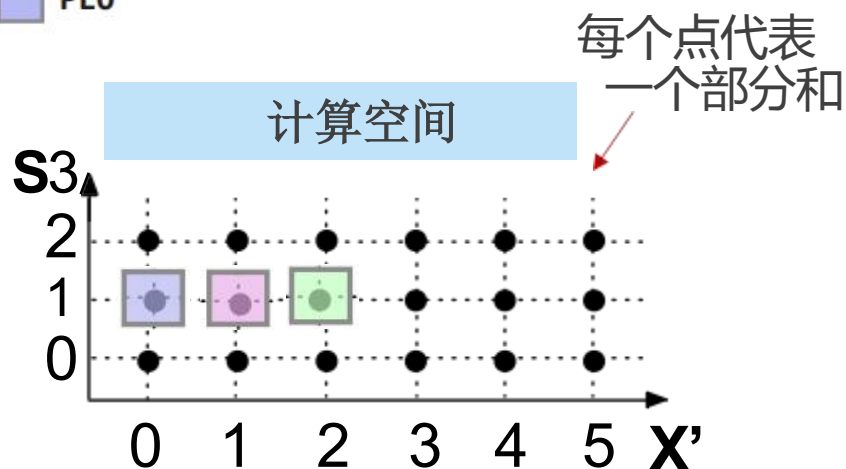
Data

PartialSum[X'] $[S]$ needs to access:

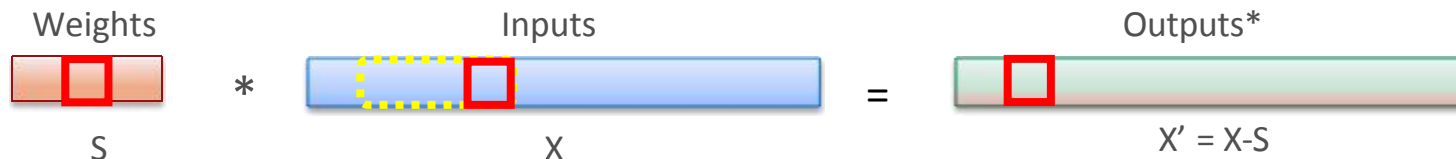
- Weight[s]
- Output[x']
- Input[x'+s]

PE2
PE1
PE0

Time = 1



输出固定的数据流(Output Stationary, OS)



Computation

```
for(int x = 0; x < X'; x++)
  for(int s = 0; s < S; s++)
    Output[x] += Weight[s] * Input[x+s]
```

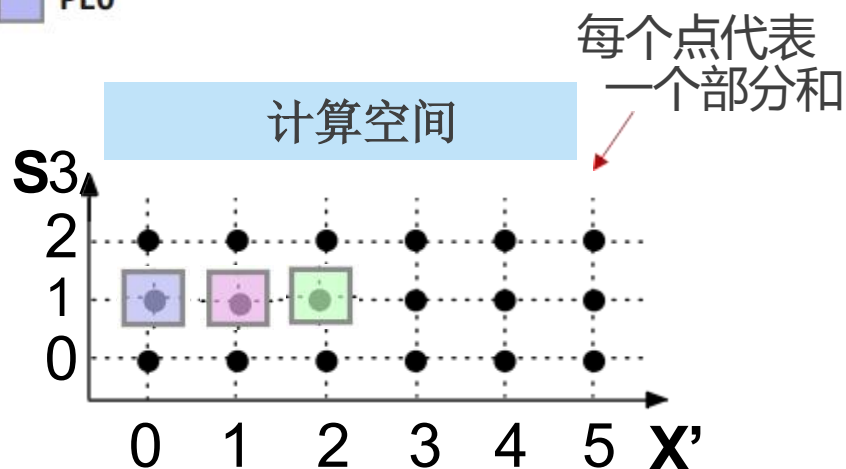
Data

PartialSum[X'][S] needs to access:

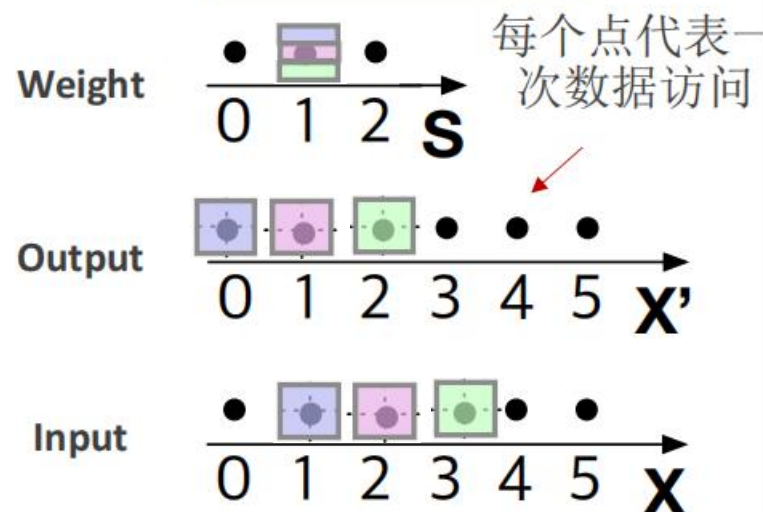
- Weight[s]
- Output[x']
- Input[x'+s]

PE2
PE1
PE0

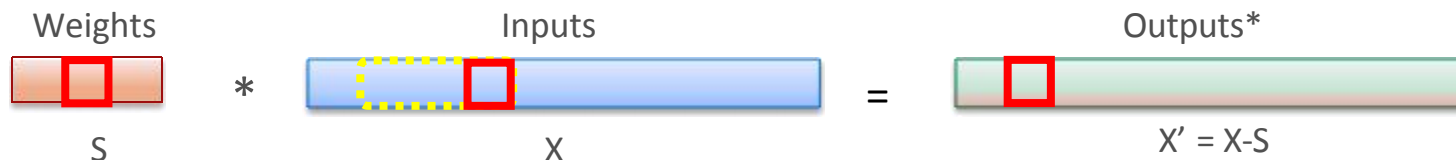
Time = 2



Data Space



输出固定的数据流(Output Stationary, OS)



Computation

```
for(int x = 0; x < X'; x++)
  for(int s = 0; s < S; s++)
    Output[x] += Weight[s] * Input[x+s]
```

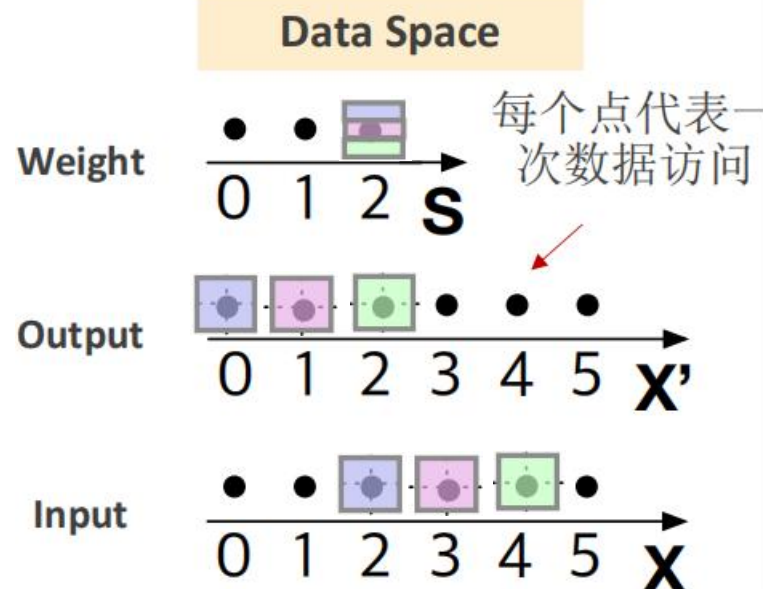
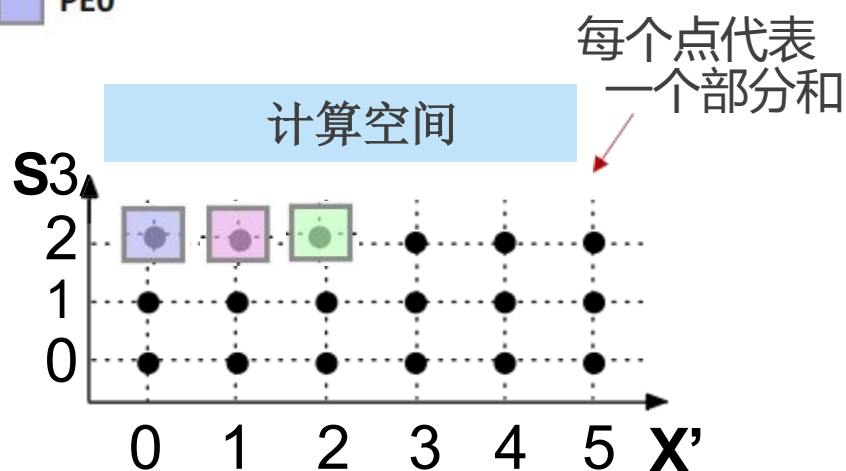
Data

PartialSum[X'][S] needs to access:

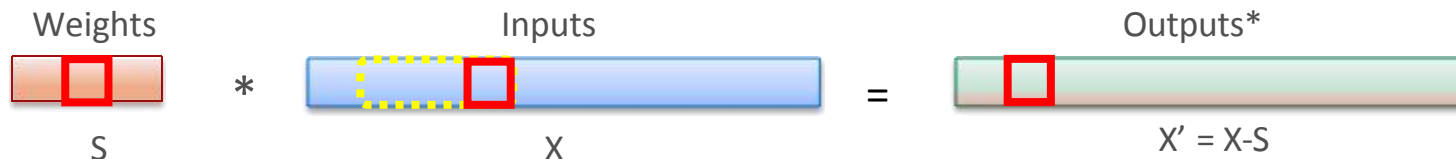
- Weight[s]
- Output[x']
- Input[x'+s]

PE2
PE1
PE0

Time = 2



输出固定的数据流(Output Stationary, OS)



Computation

```
for(int x = 0; x < X'; x++)
    for(int s = 0; s < S; s++)
        Output[x] += Weight[s] * Input[x+s]
```

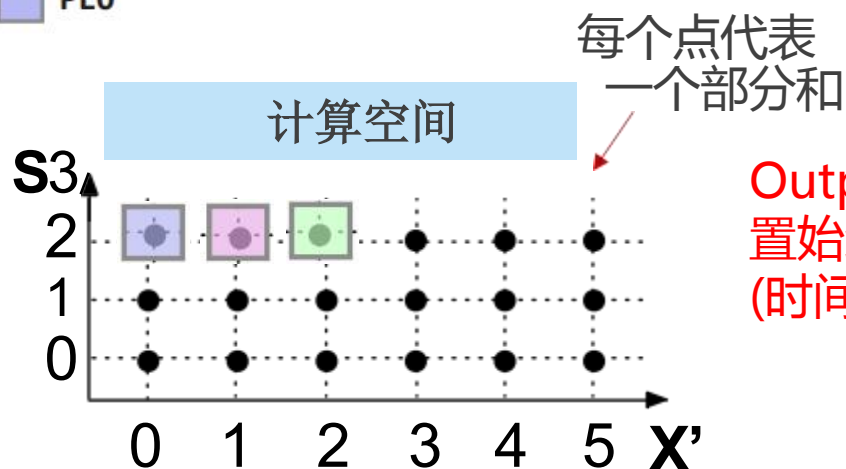
Data

PartialSum[X'] $[S]$ needs to access:

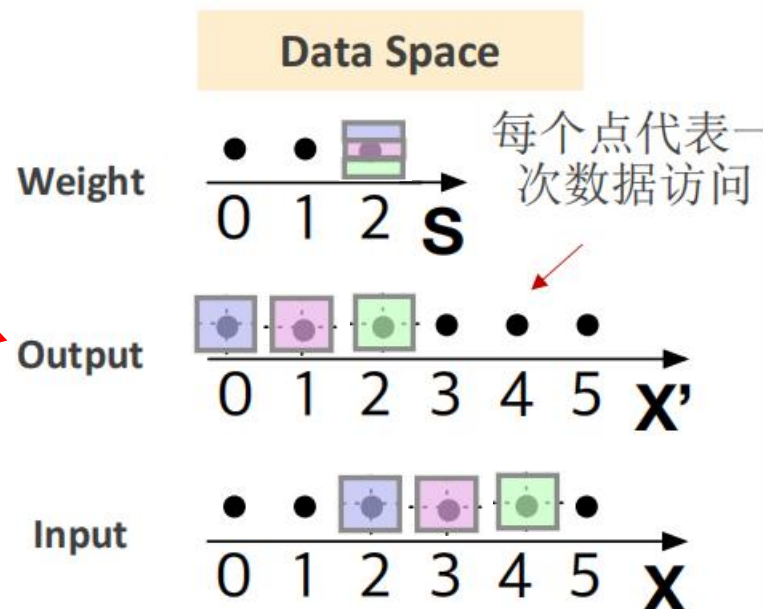
- Weight[s]
- Output[x']
- Input[x'+s]



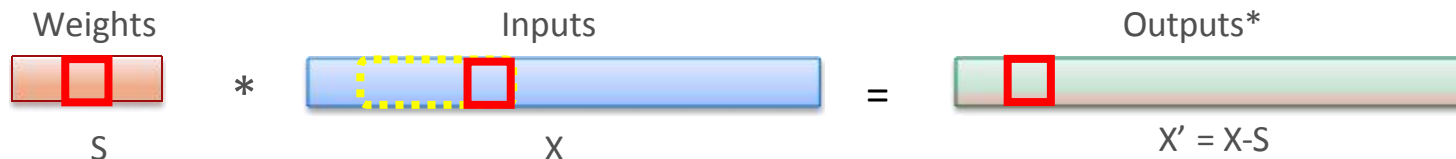
Time = 2



Output位置始终不动
(时间重用)



输出固定的数据流(Output Stationary, OS)



Computation

```
for(int x = 0; x < X'; x++)
    for(int s = 0; s < S; s++)
        Output[x] += Weight[s] * Input[x+s]
```

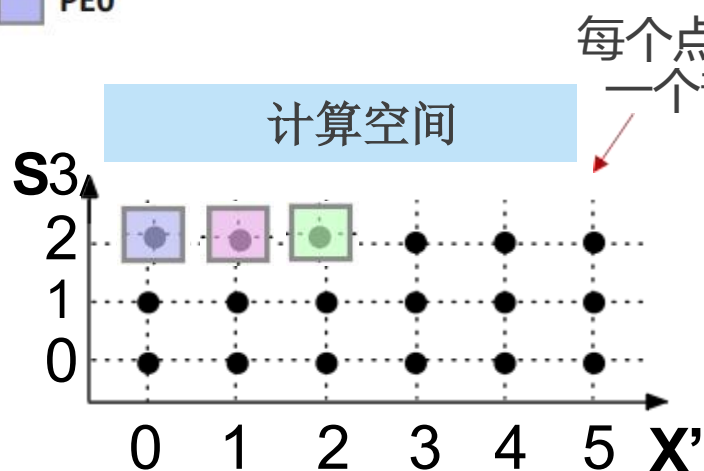
Data

PartialSum[X'][S] needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]



Time = 2



空间重用(广播 weight)

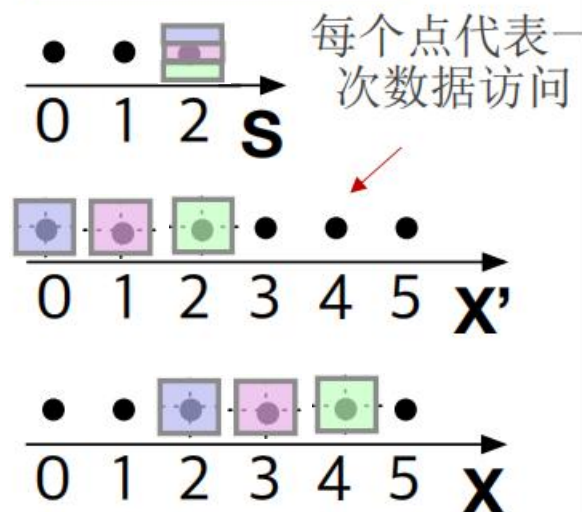
Output位置始终不动(时间重用)

Weight

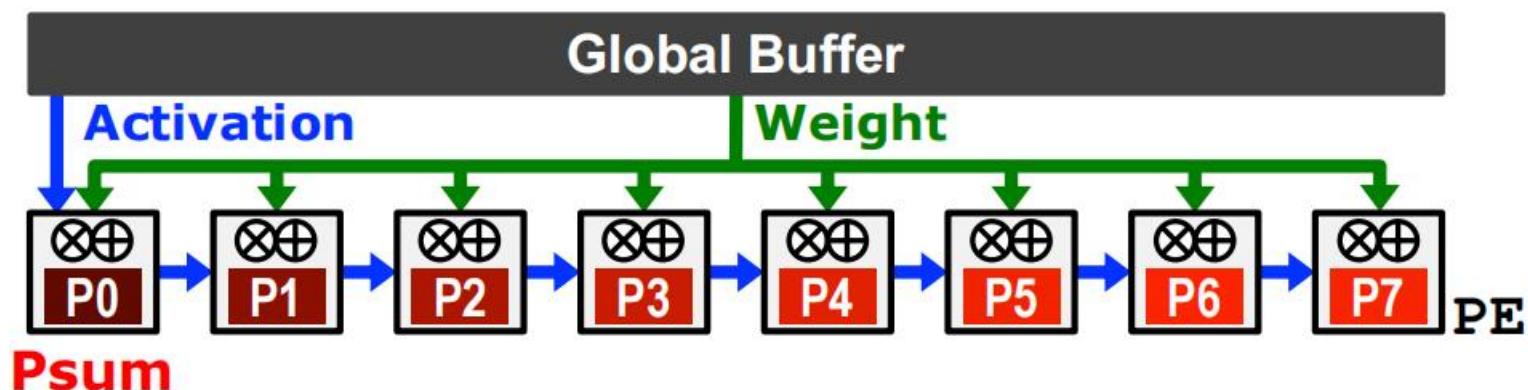
Output

Input

Data Space



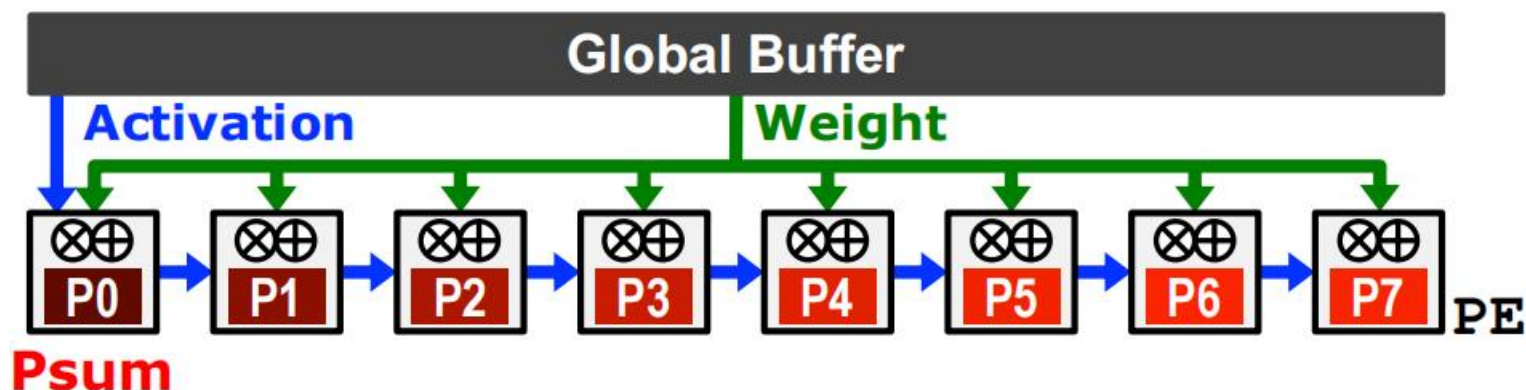
输出固定数据流的实现



第一轮：

1. $w[0]$ 广播给所有 $P0, \dots, P7$
2. $A[0], \dots, A[7]$ 分别存放在 $P7, \dots, P0$ 上
3. **运算:** $P7$ 的结果为 $w[0] * A[0]$, $P6$ 的结果为 $w[0] * A[1]$, $P0$ 的结果为 $w[0] * A[7]$

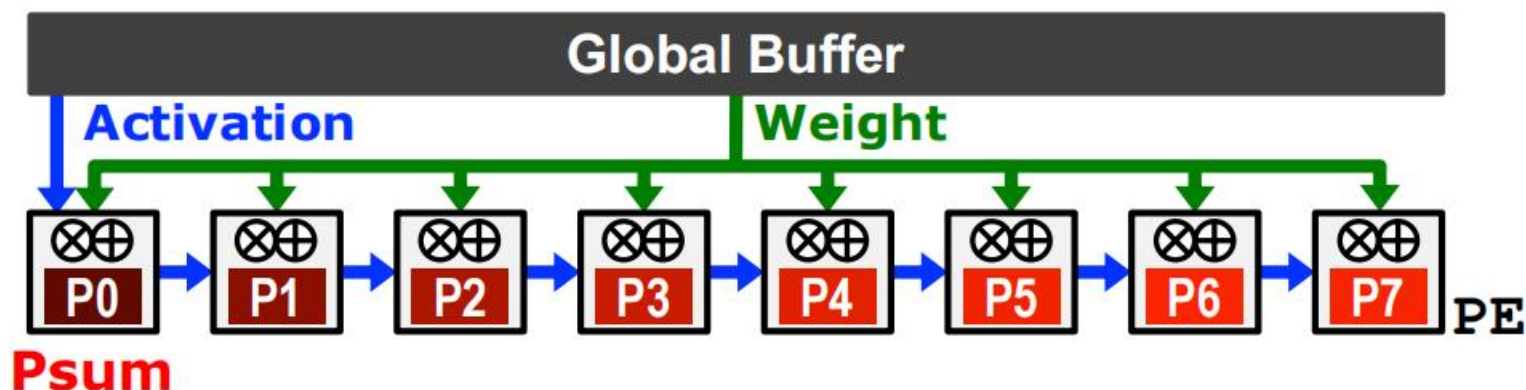
输出固定数据流的实现



第二轮：

1. $w[1]$ 广播给所有 $P0, \dots, P7$
2. $A[0], \dots, A[7]$ 向右移动, $A[1], \dots, A[6]$ 分别存在 $P7, \dots, P1$ 上
3. **运算:** $P7$ 的结果为 $w[0]*A[0]+w[1]*A[1]$, $P6$ 的结果为 $w[0]*A[1]+w[1]*A[2]$, 以此类推

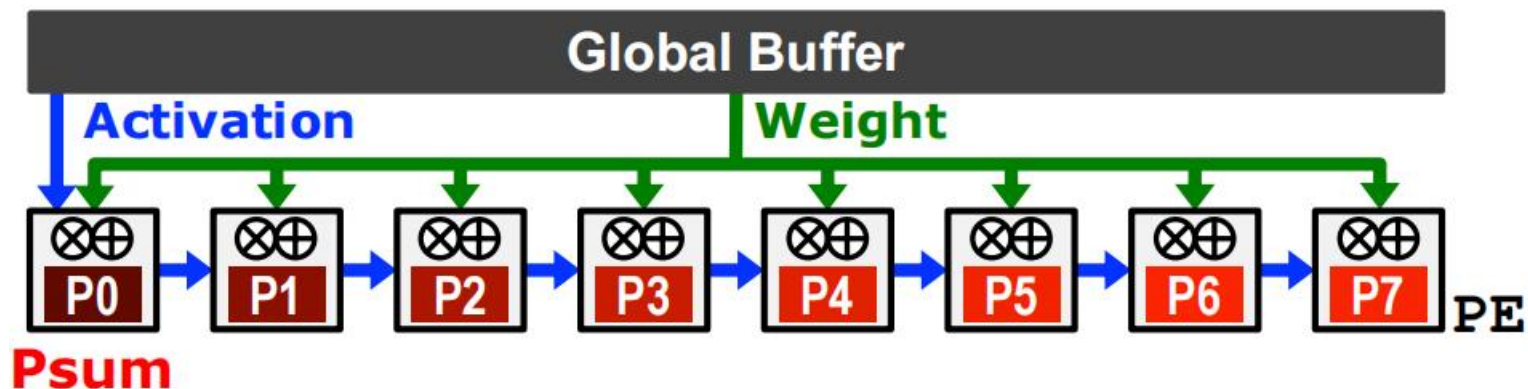
输出固定数据流的实现



第八轮：

1. $w[7]$ 广播给所有 $P0, \dots, P7$
2. $A[0], \dots, A[7]$ 向右移动， $A[7]$ 存在 $P7$
3. **运算：** $P7$ 的结果为 $w[0]*A[0]+w[1]*A[1]+\dots+A[7]*A[7]$ ，正好是卷积结果的第一个元素

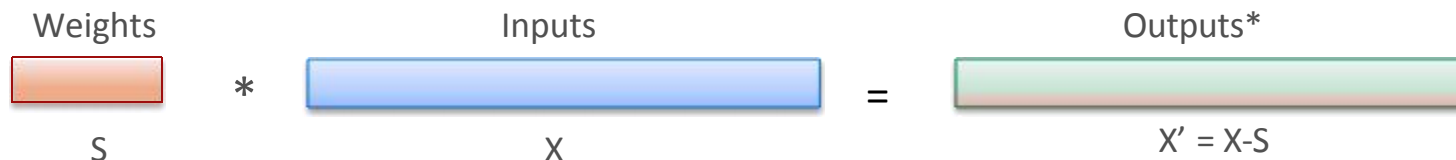
输出固定数据流的实现



在整个过程中：

- 部分和Psum始终保持不动，减少了访存次数
- 每次广播weight，空间复用
- Input Activation每次移动，在PE之间空间复用

权重固定的数据流



Computation

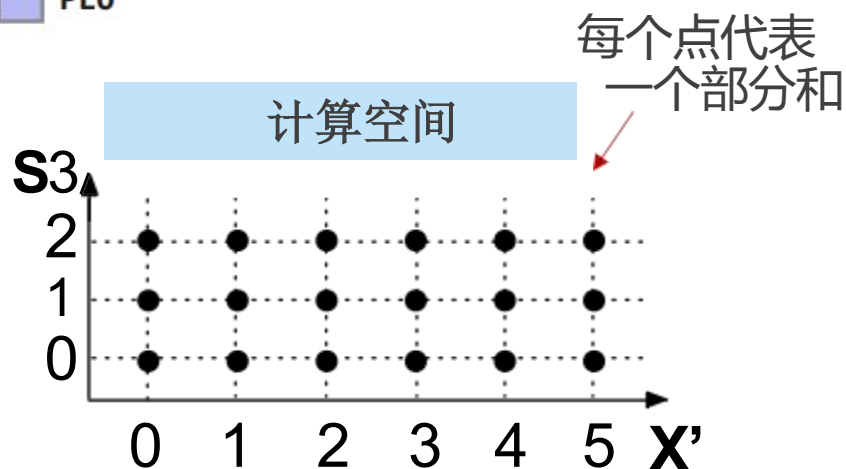
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

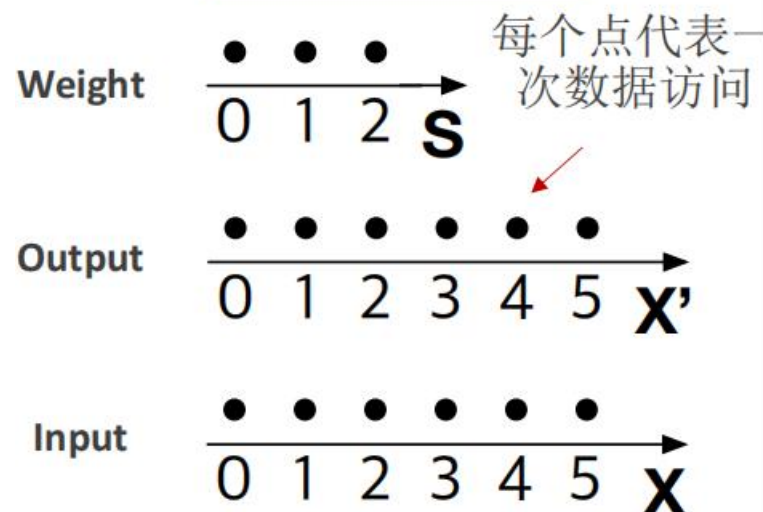
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

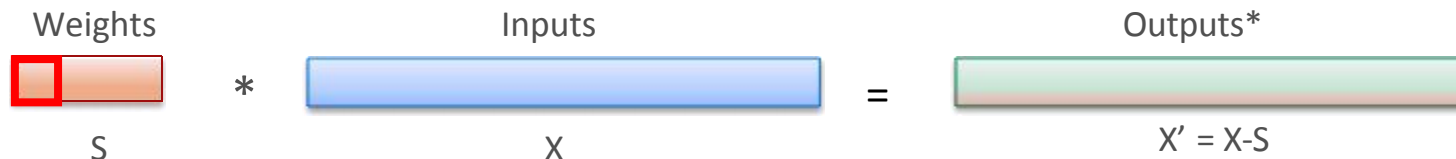
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

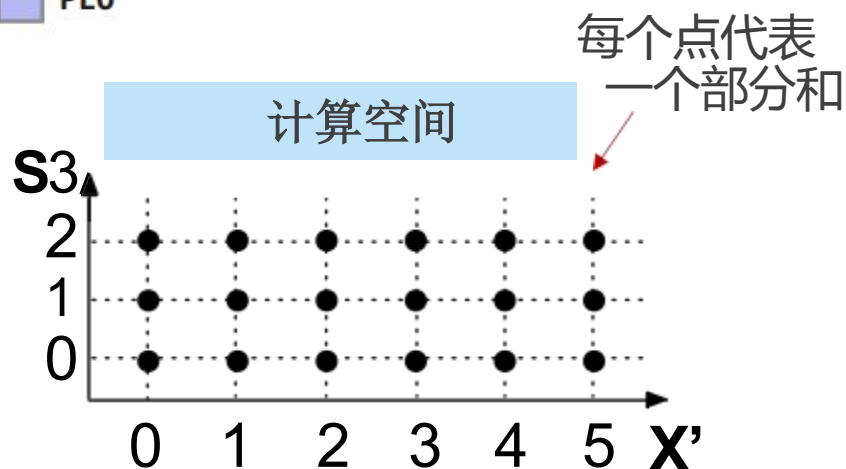
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

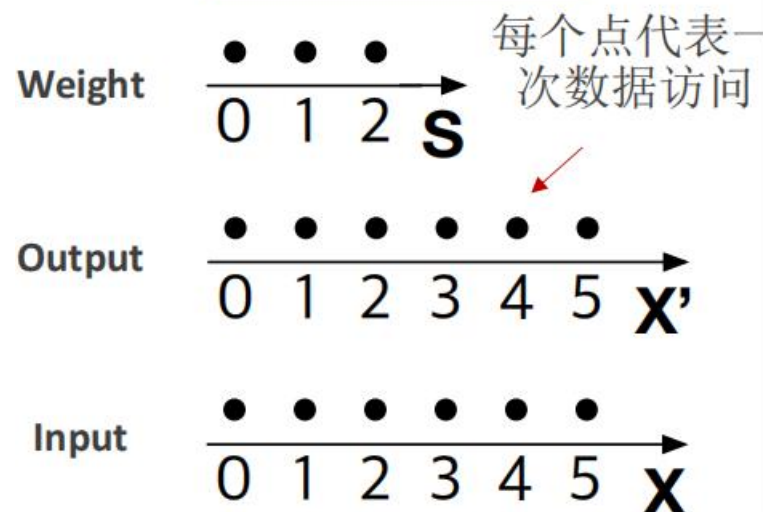
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

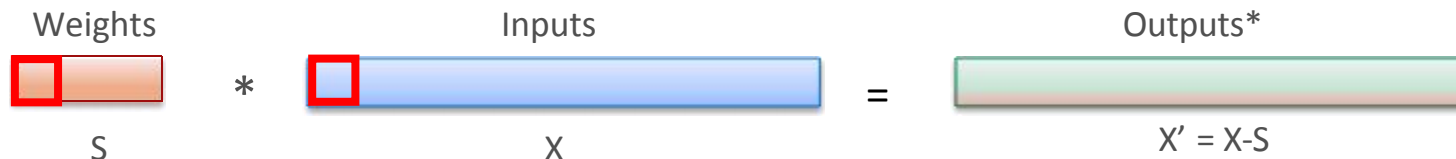
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

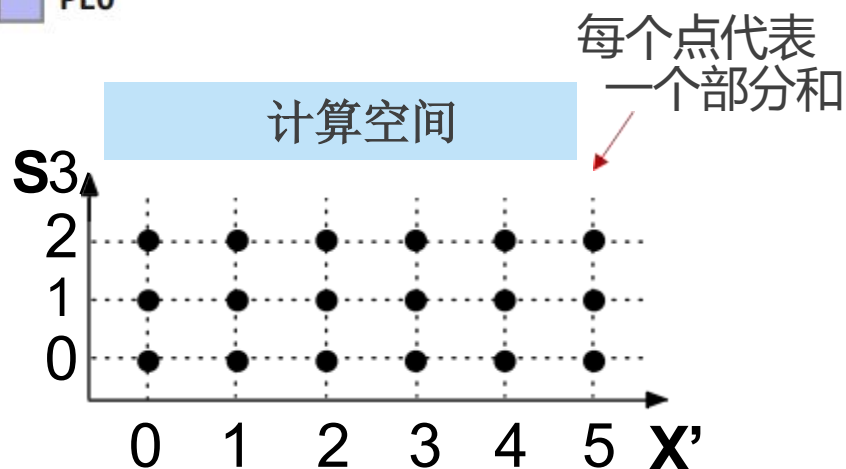
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

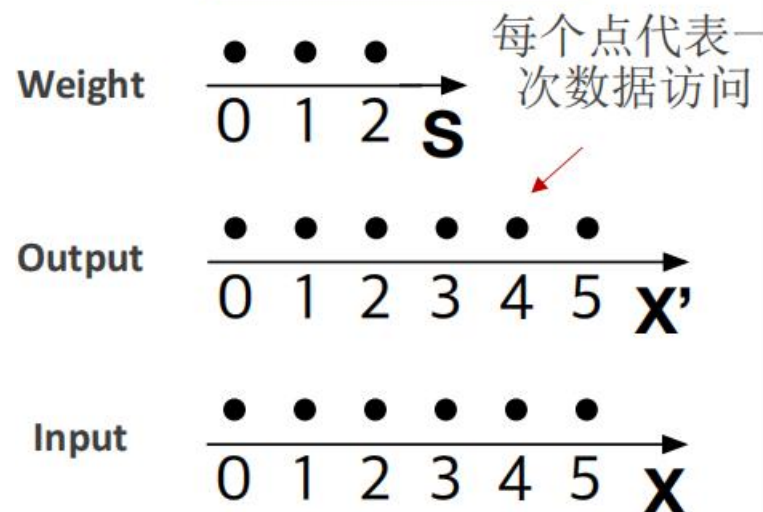
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

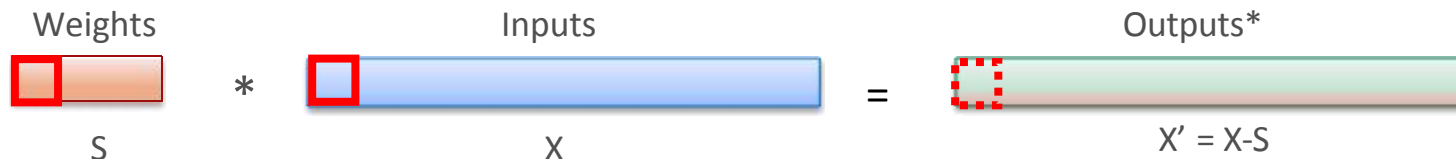
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

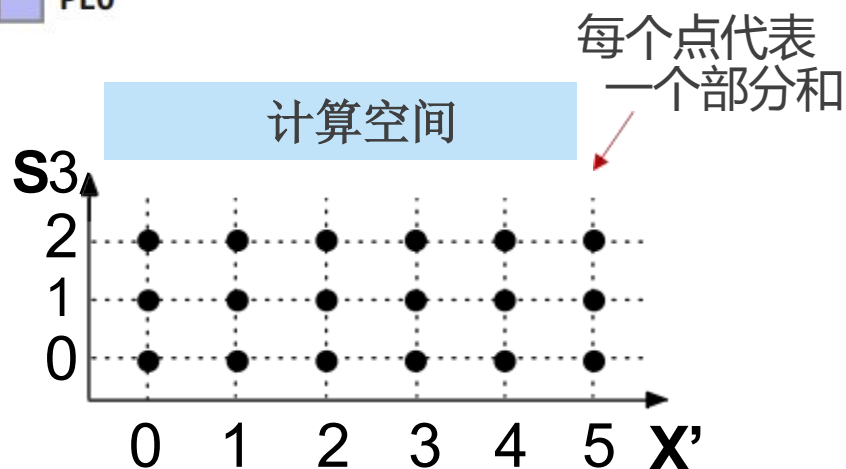
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

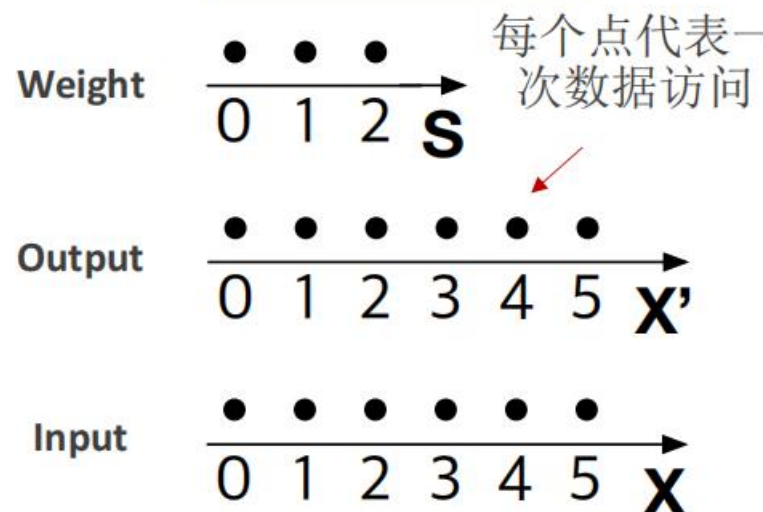
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

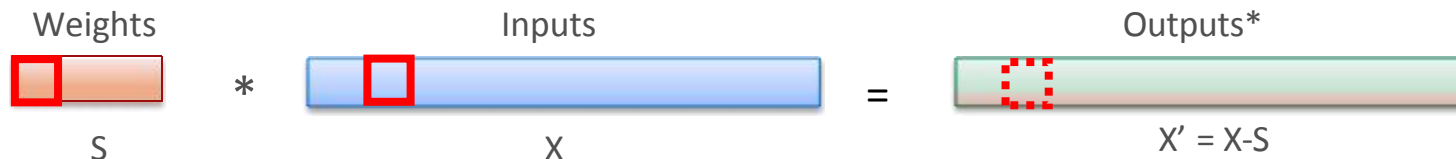
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

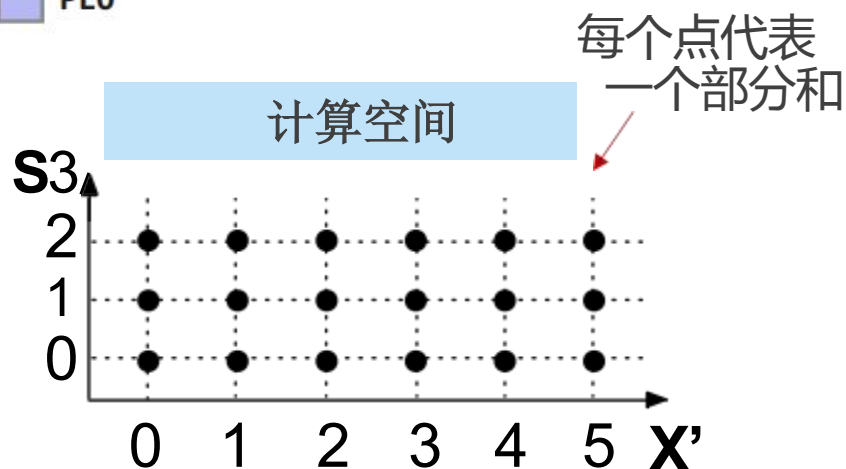
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

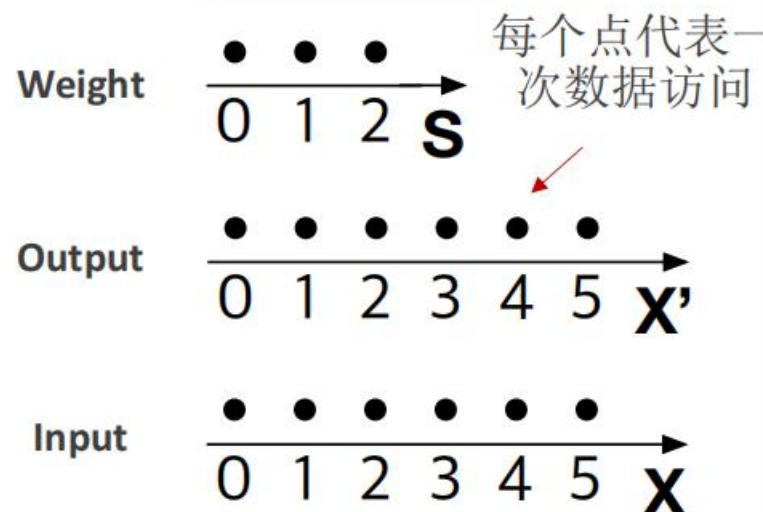
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

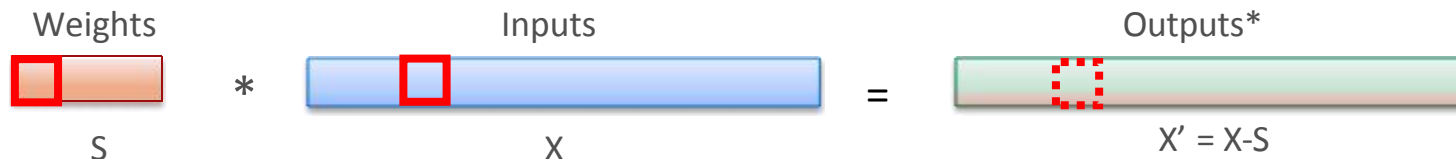
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

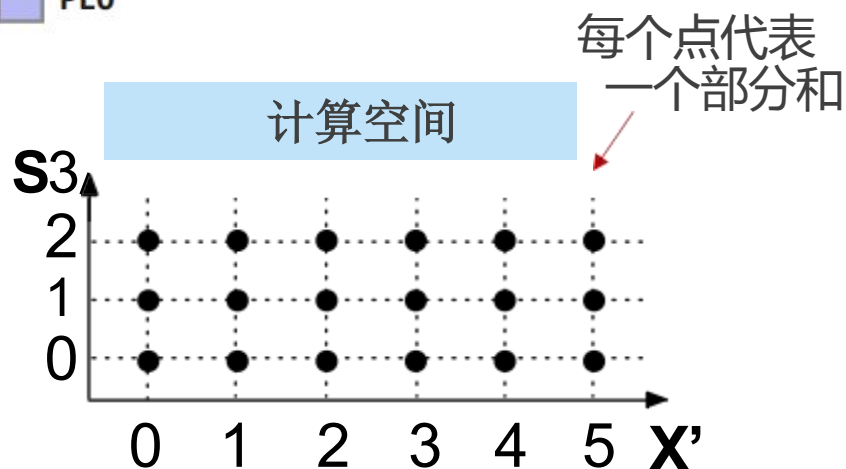
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

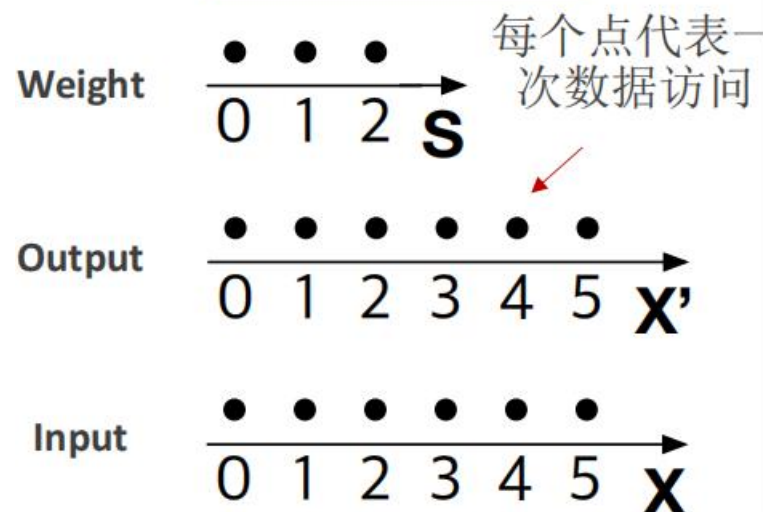
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

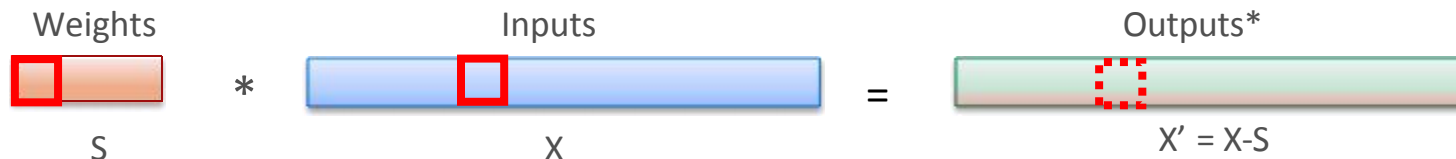
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

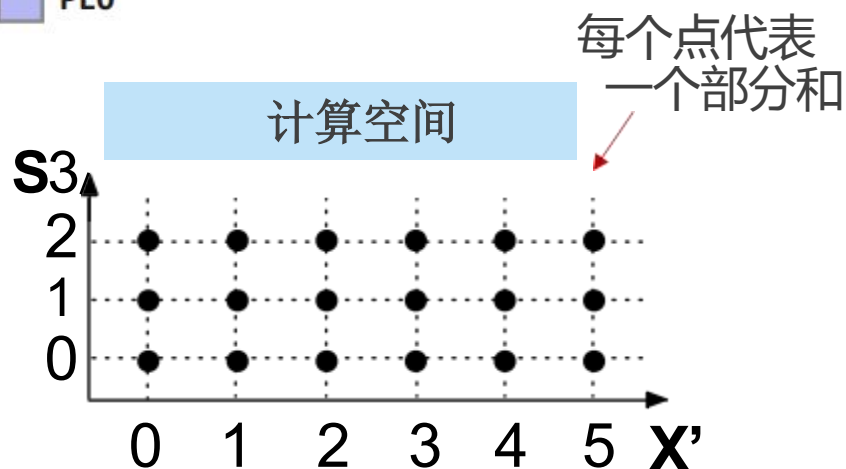
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

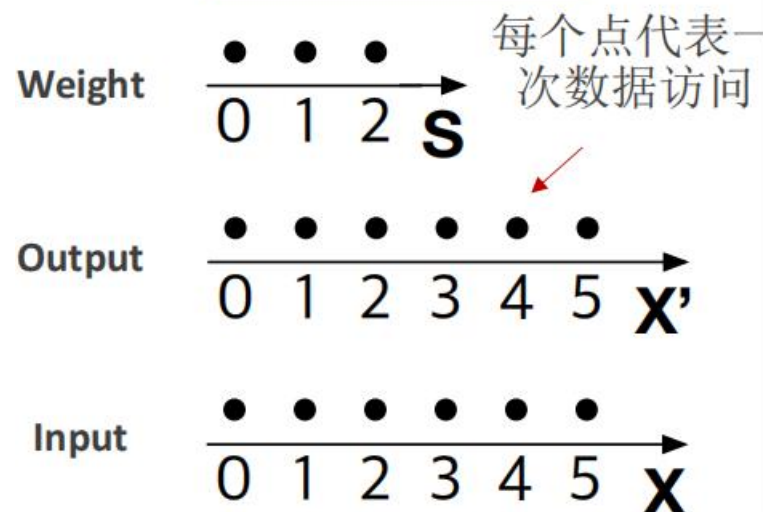
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

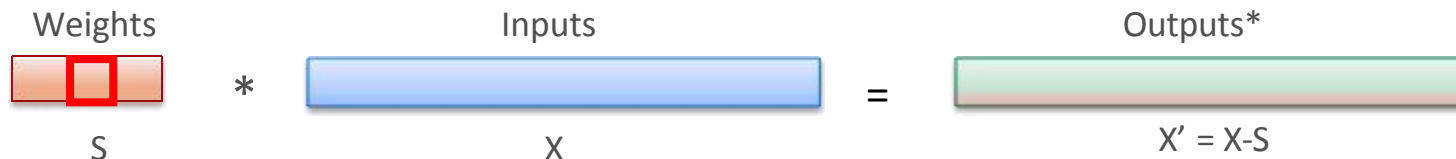
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

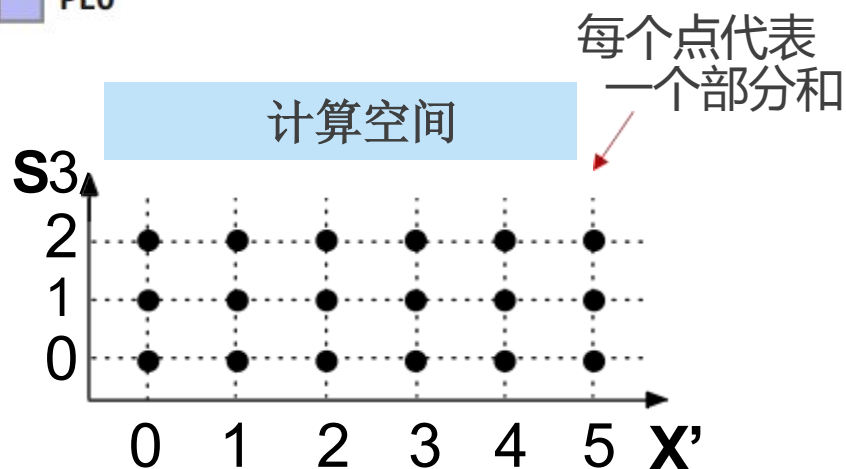
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

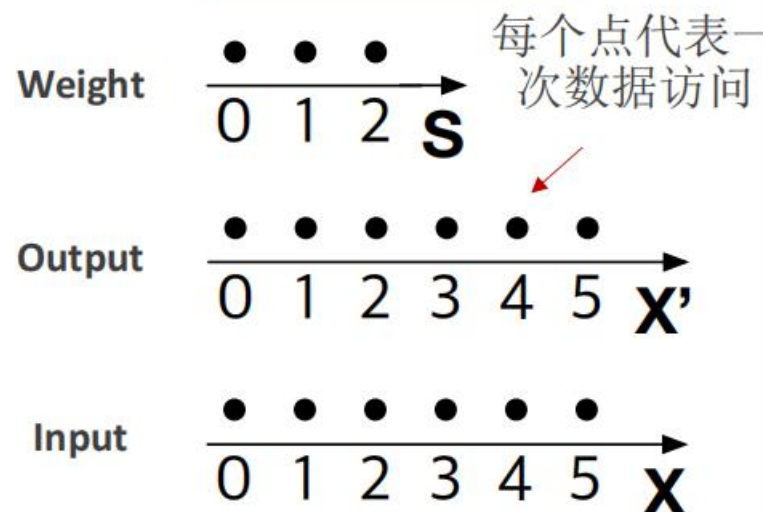
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

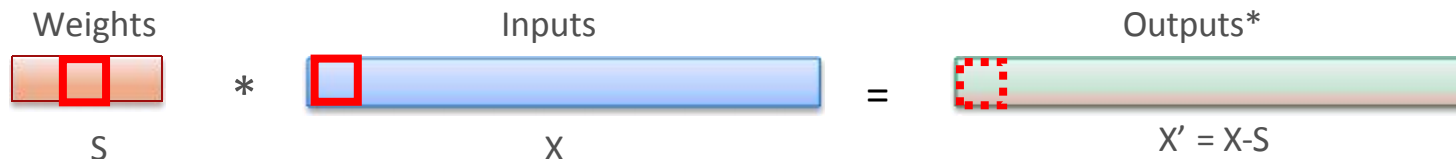
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

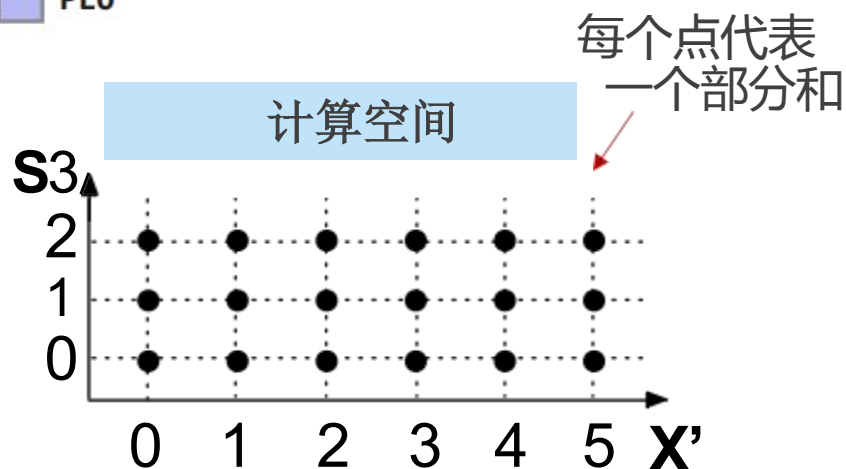
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

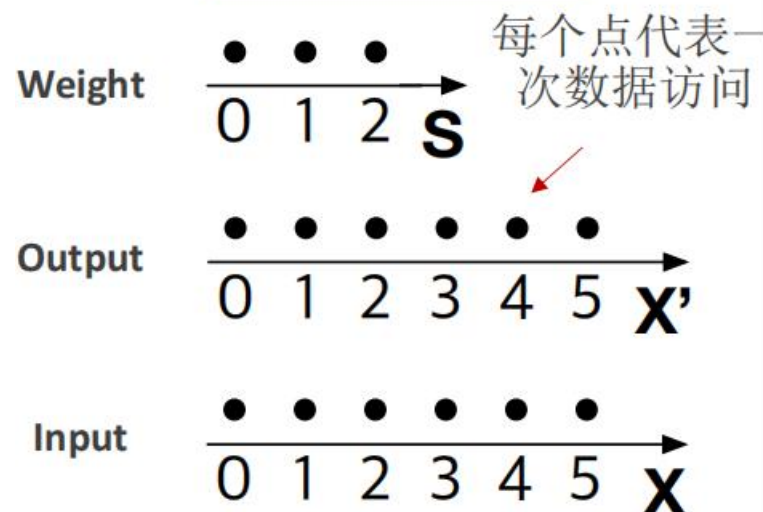
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

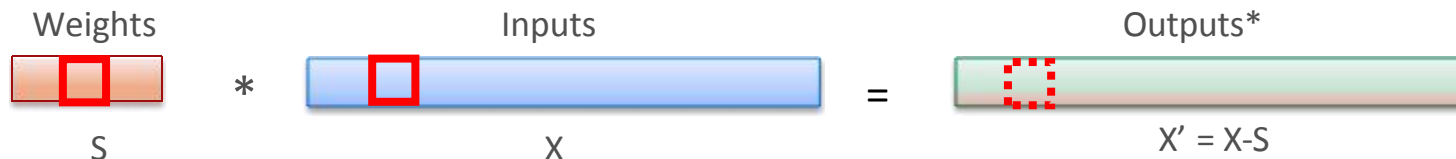
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

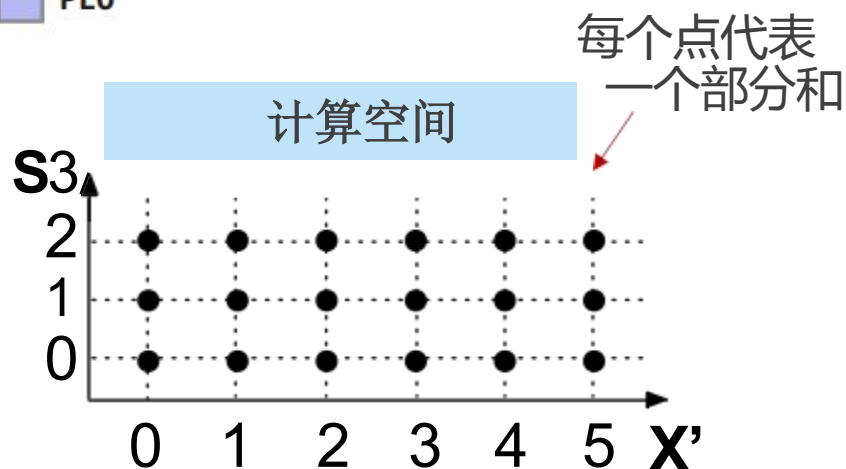
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

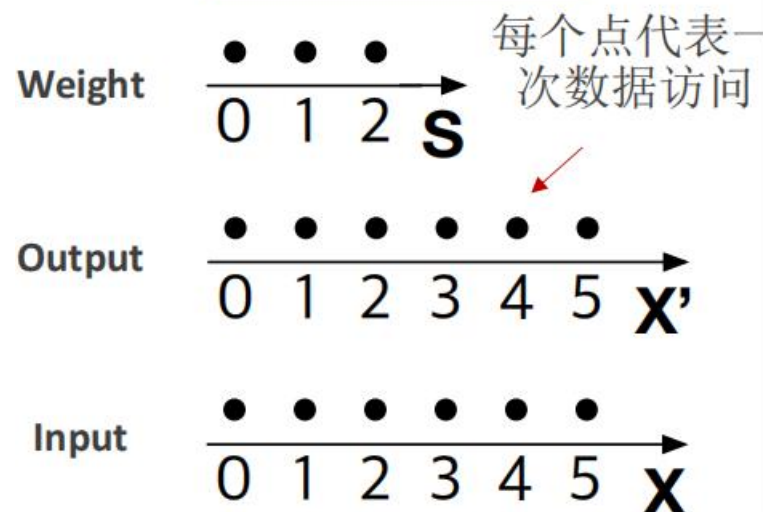
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

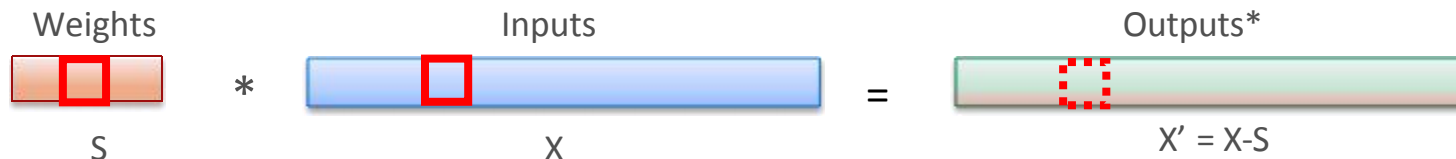
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

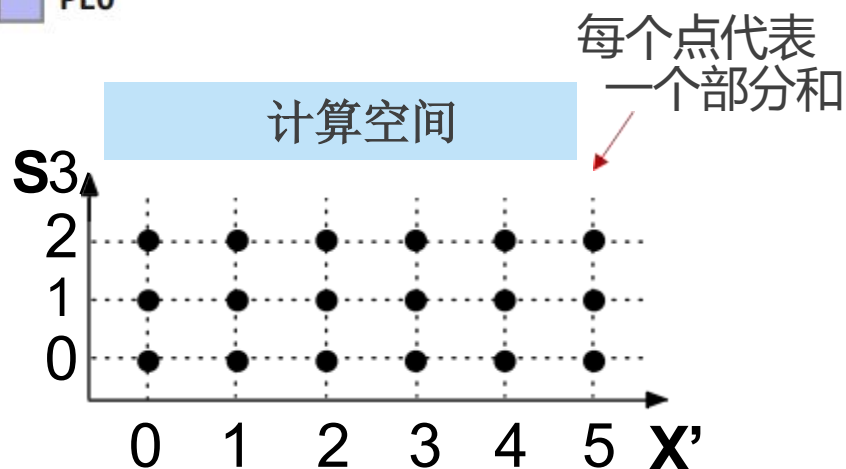
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

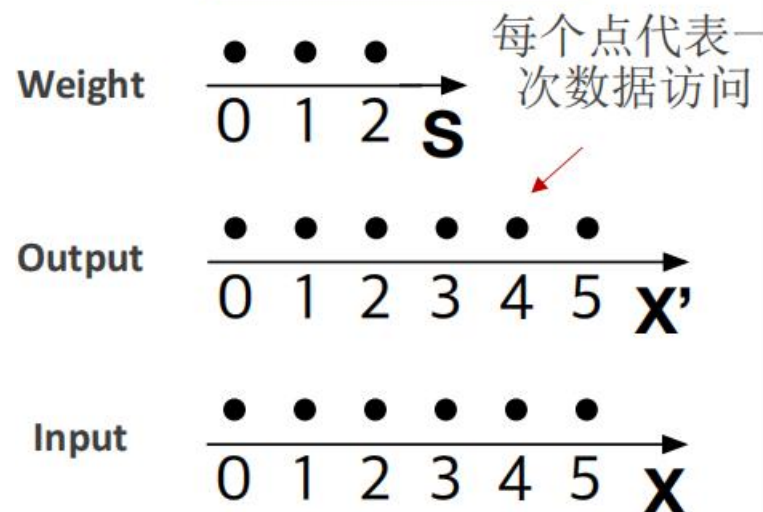
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

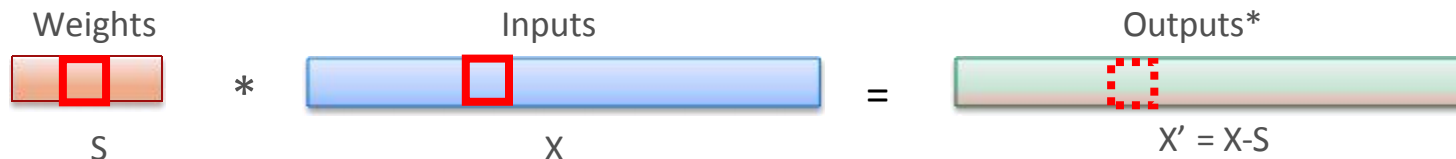
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

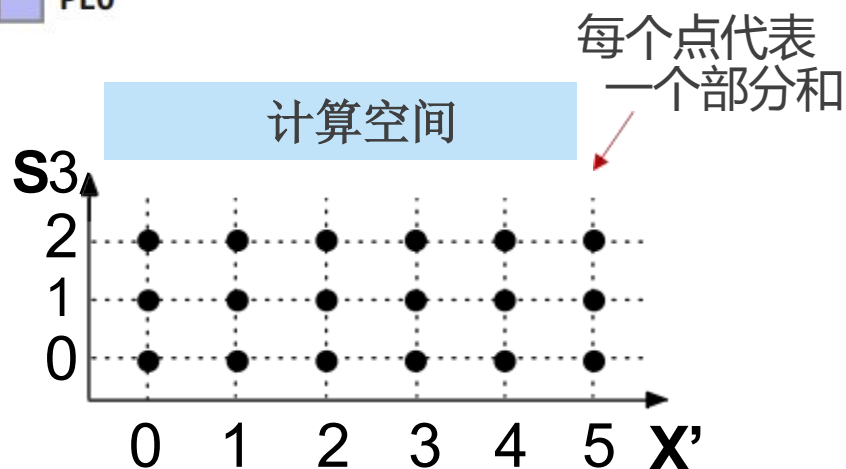
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

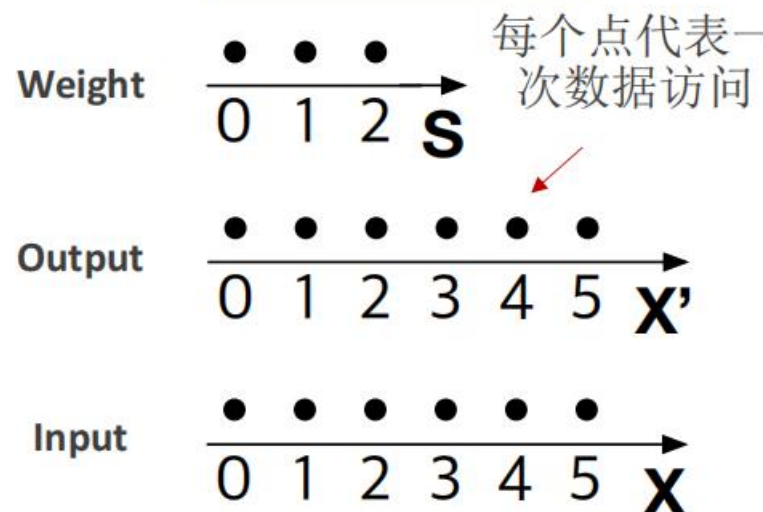
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

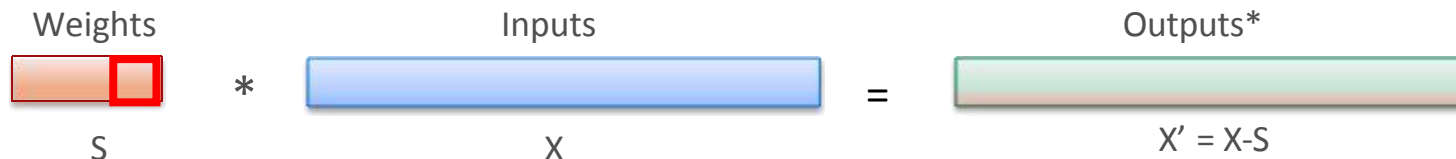
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

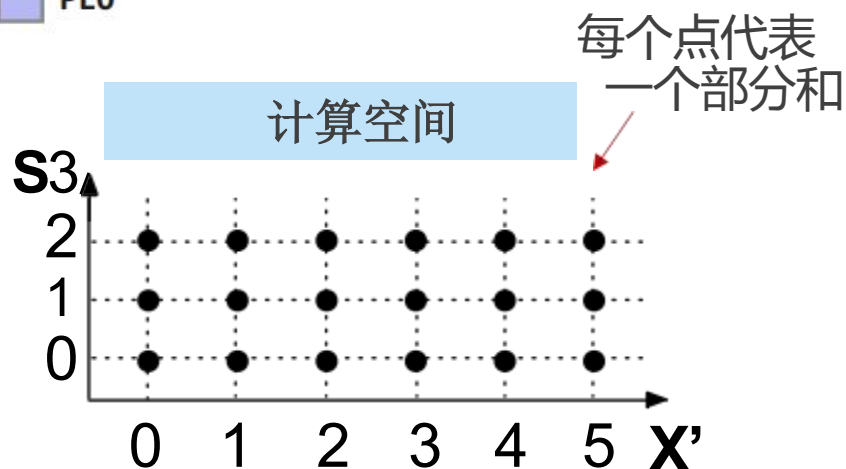
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

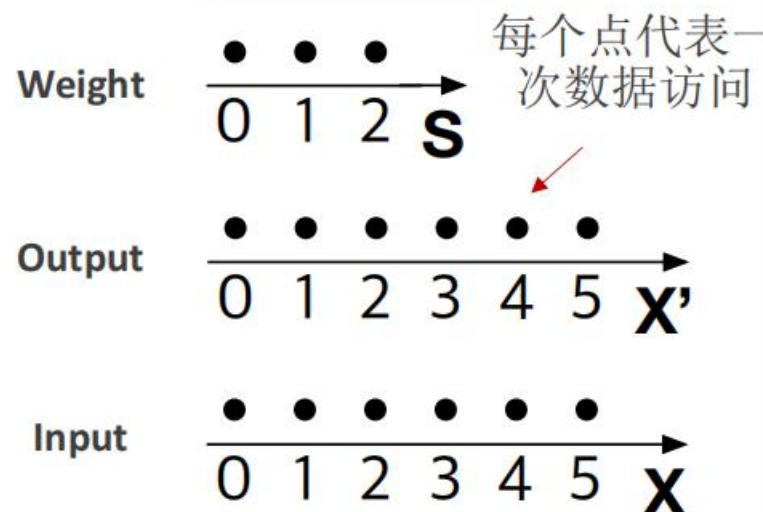
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

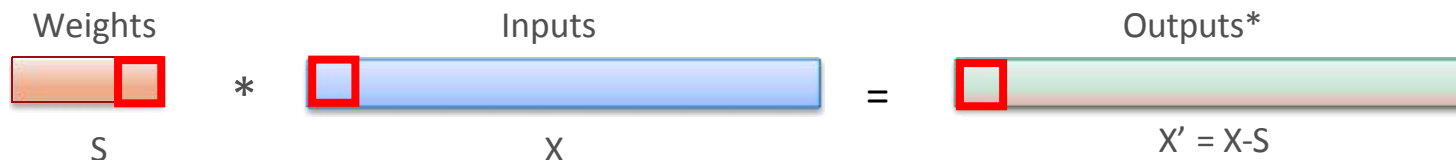
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

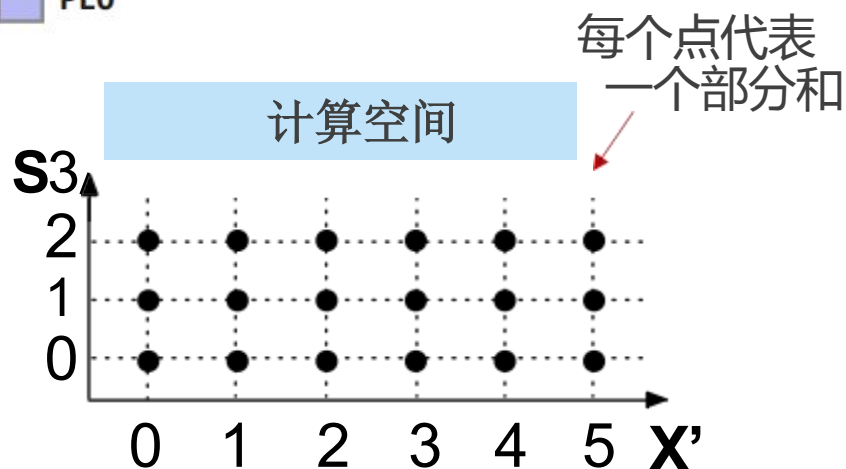
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

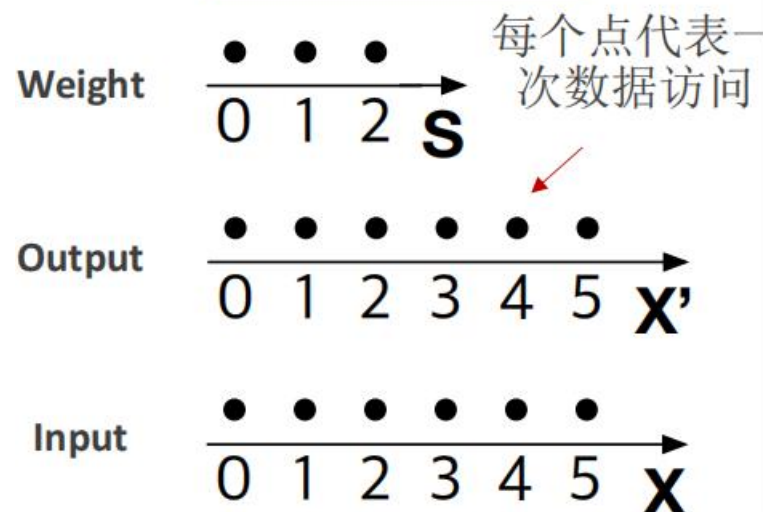
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

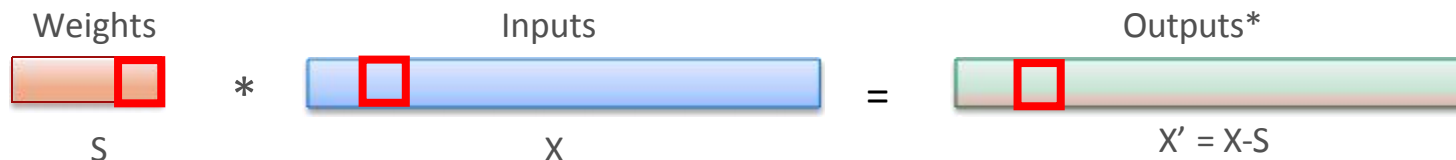
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

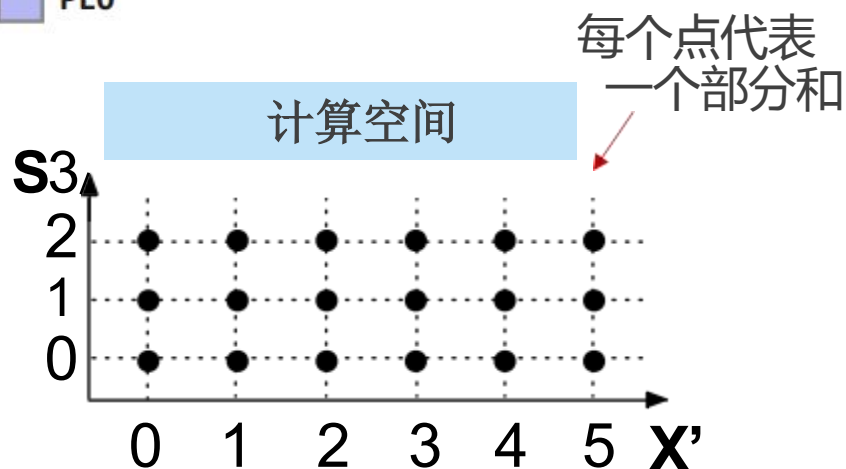
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

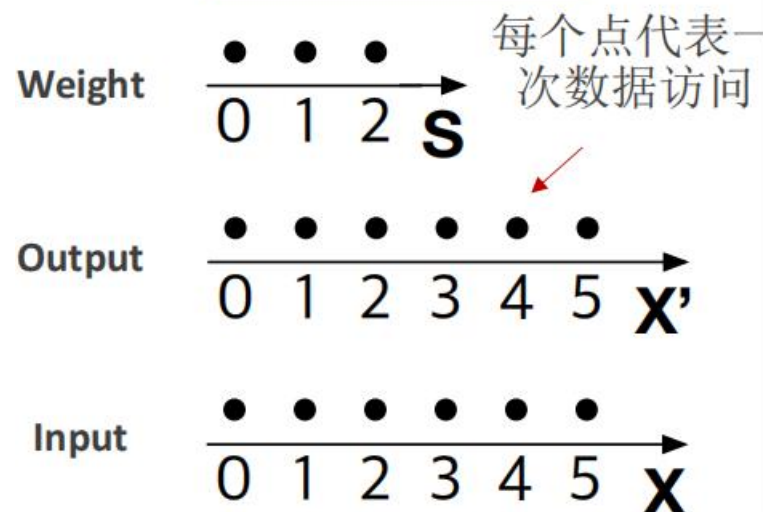
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

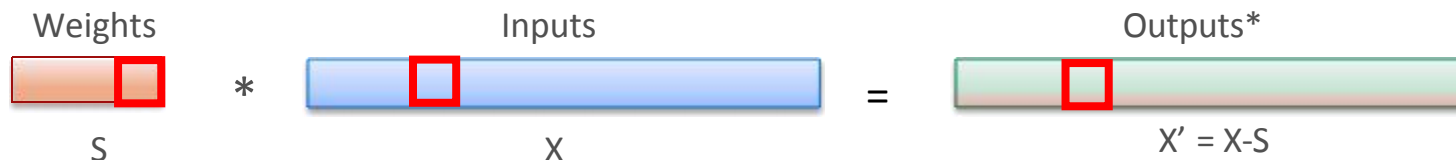
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

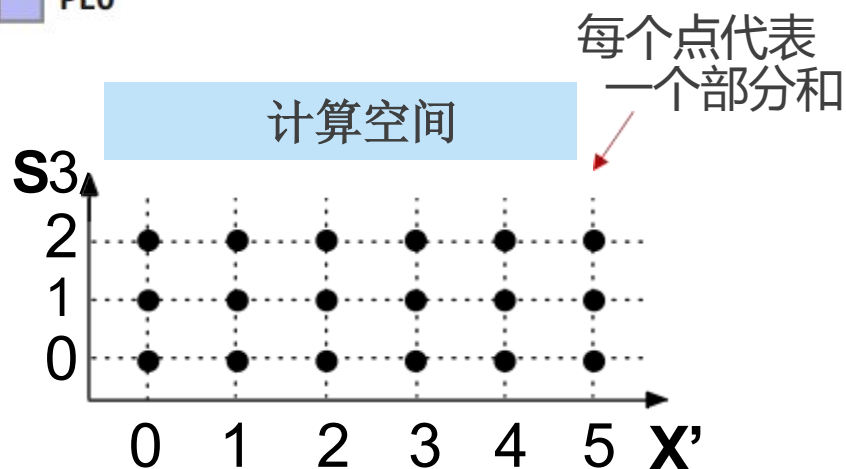
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

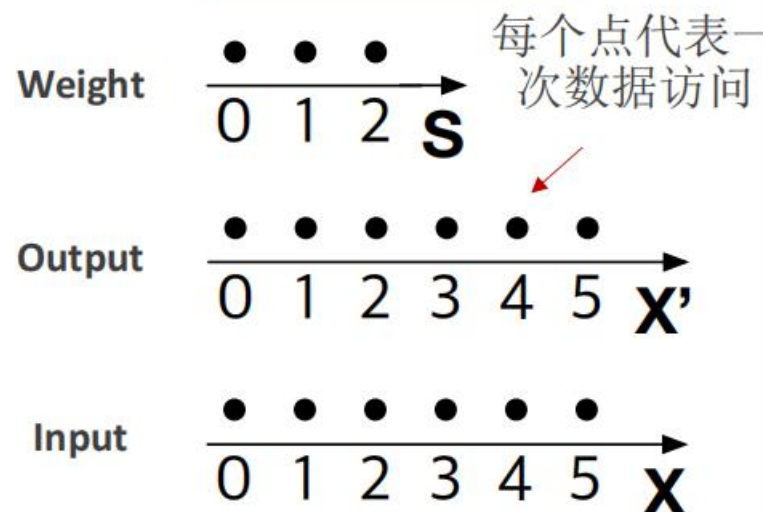
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

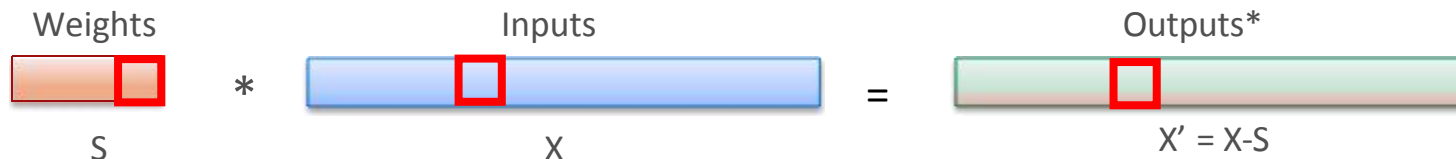
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

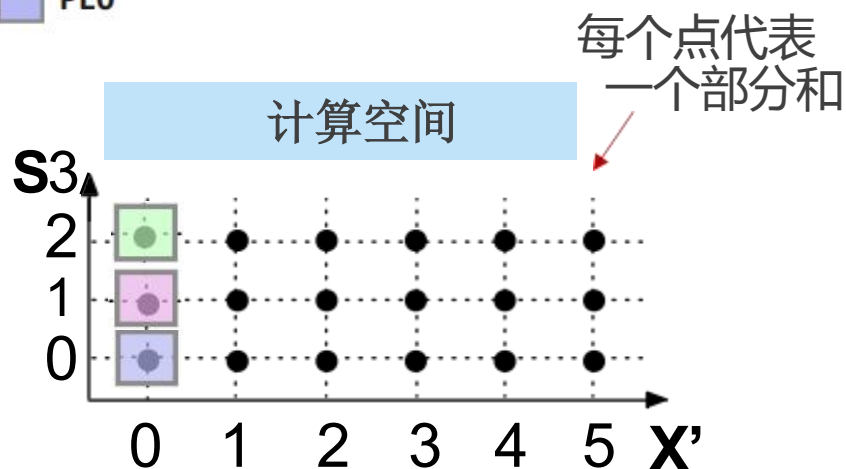
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

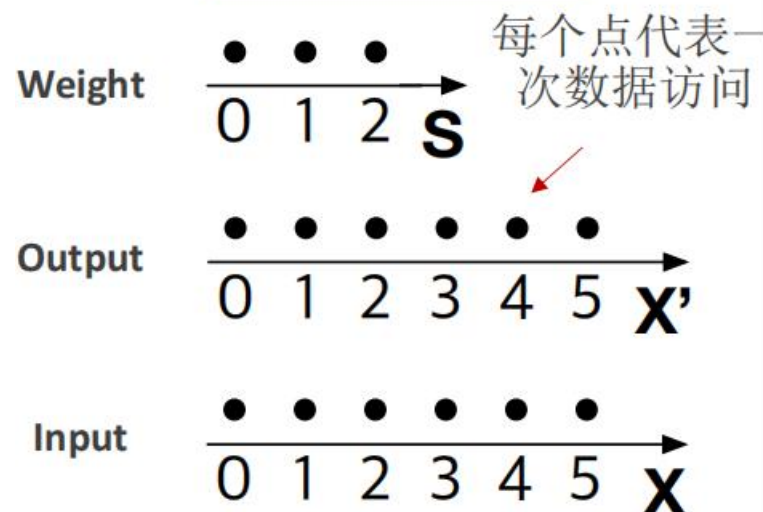
PartialSum[X'] [S] needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

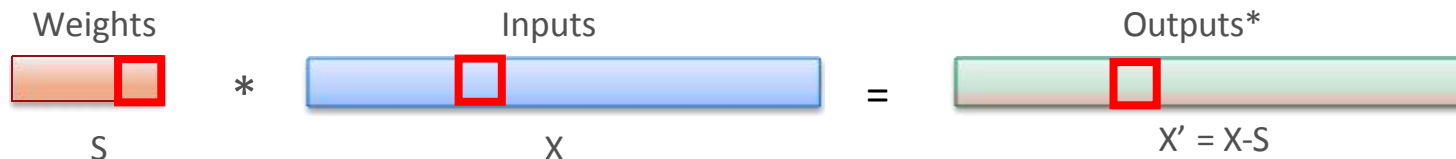
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

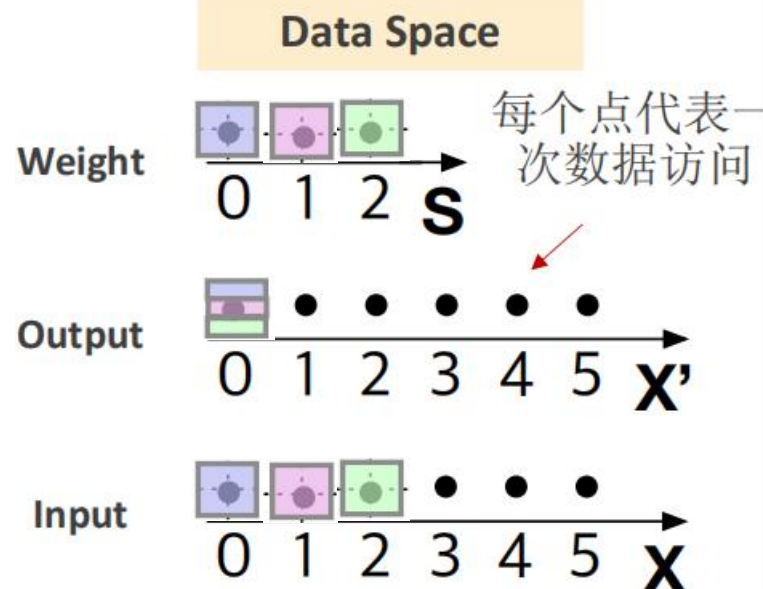
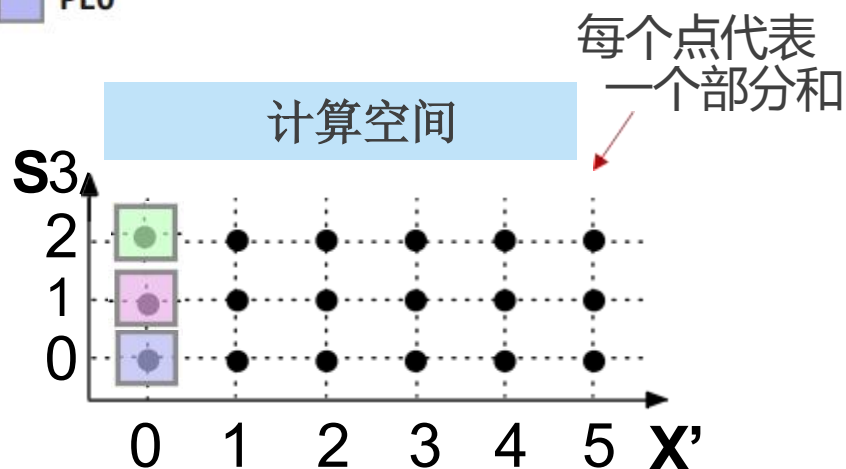
```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

Data

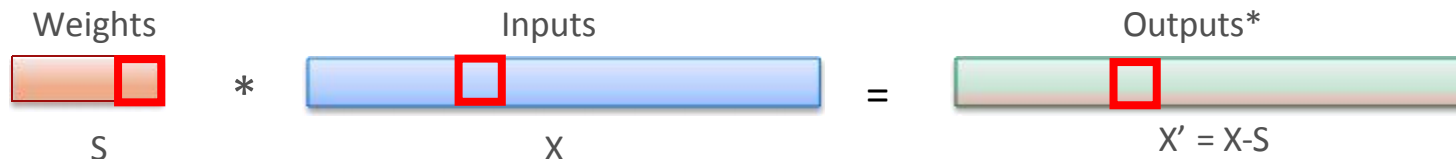
PartialSum[X'] $[S]$ needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

PE2
PE1
PE0



权重固定的数据流



Computation

```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

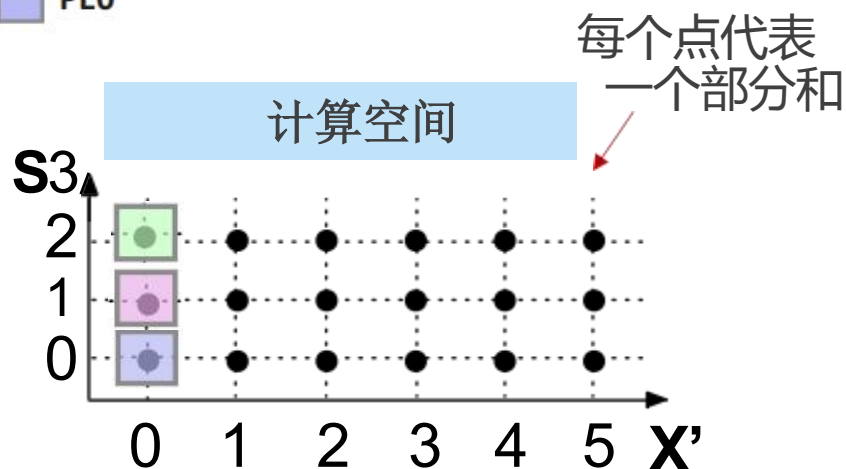
Data

PartialSum[X'] [S] needs to access:

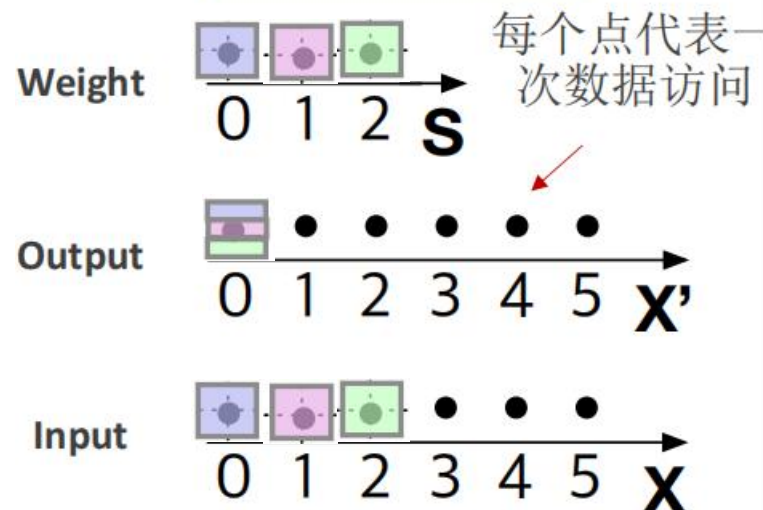
- Weight[s]
- Output[x']
- Input[x'+s]

Time = 0

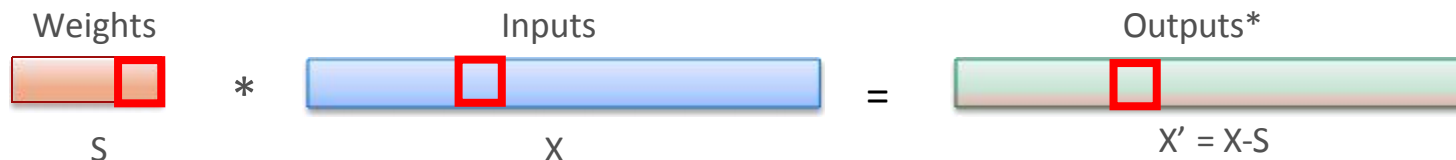
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

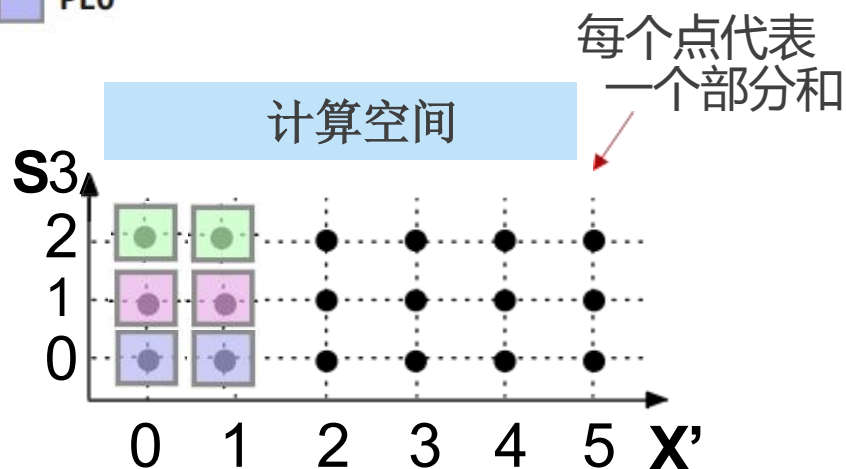
Data

PartialSum[X'] [S] needs to access:

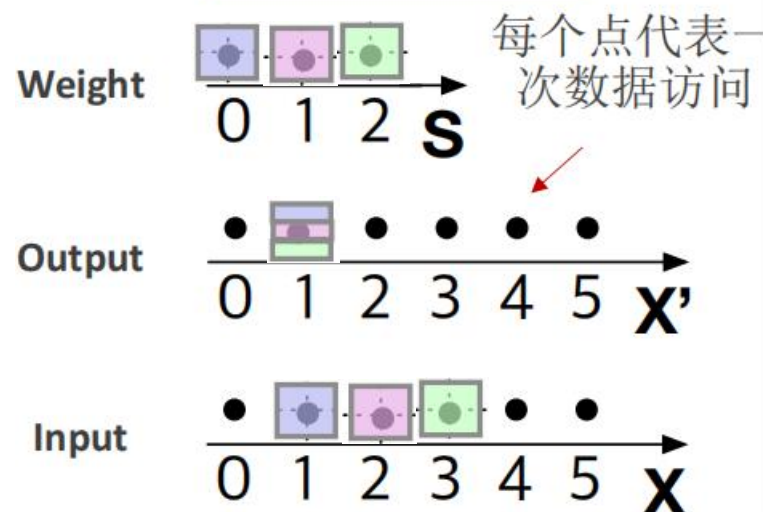
- Weight[s]
- Output[x']
- Input[x'+s]

Time = 1

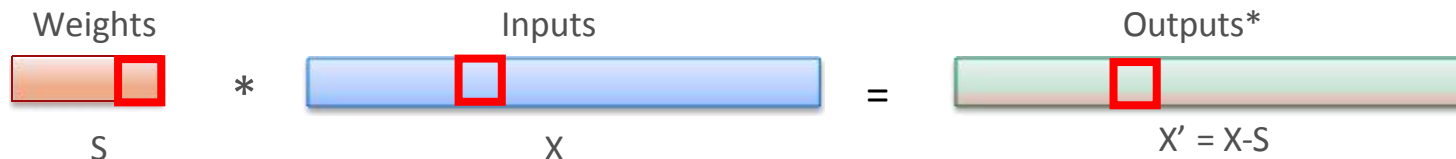
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

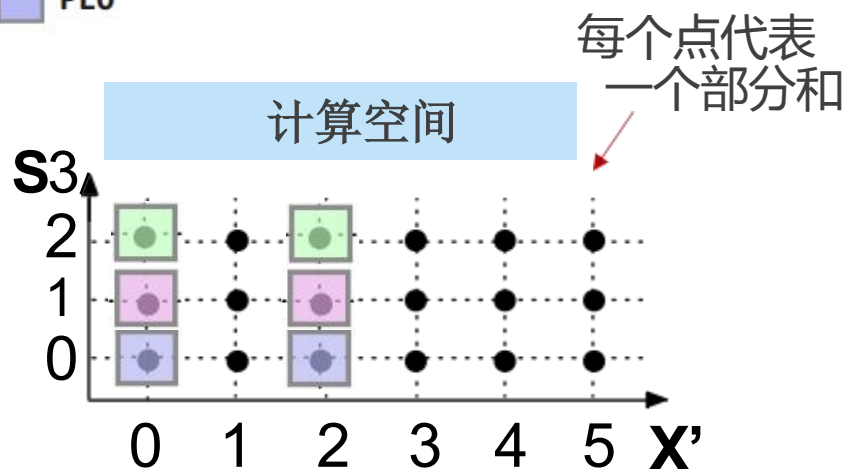
Data

PartialSum[X'] $[S]$ needs to access:

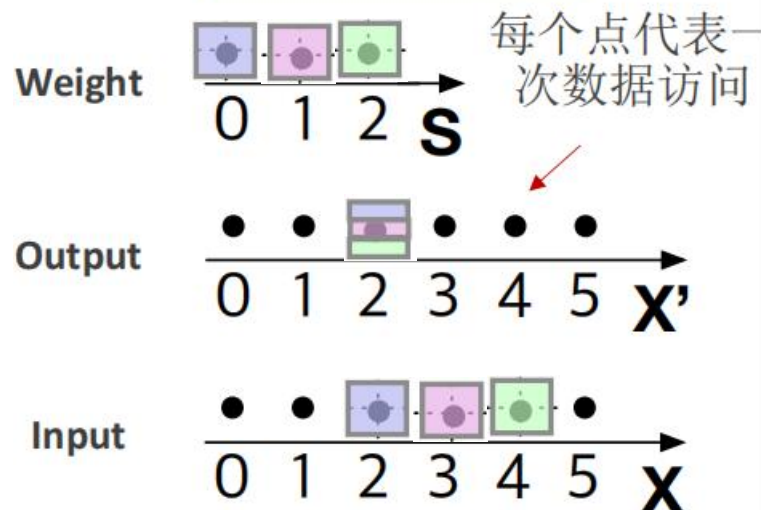
- Weight[s]
- Output[x']
- Input[x'+s]

Time = 2

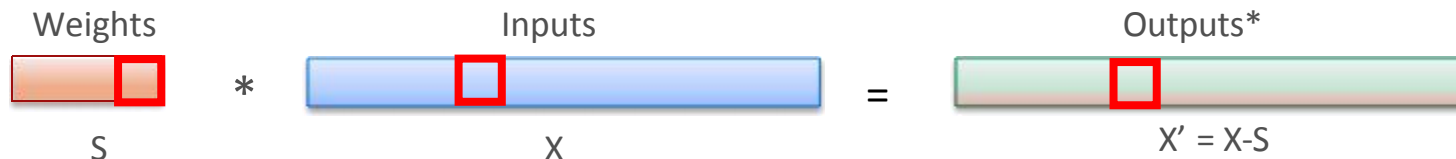
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

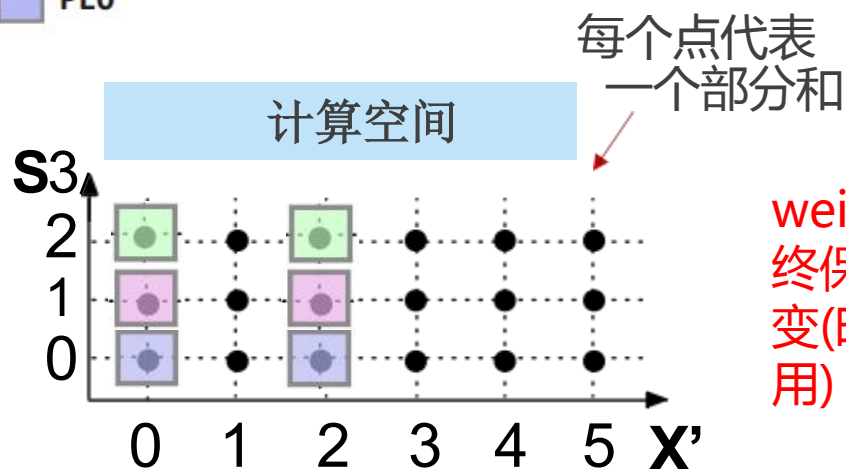
Data

PartialSum[X'] $[S]$ needs to access:

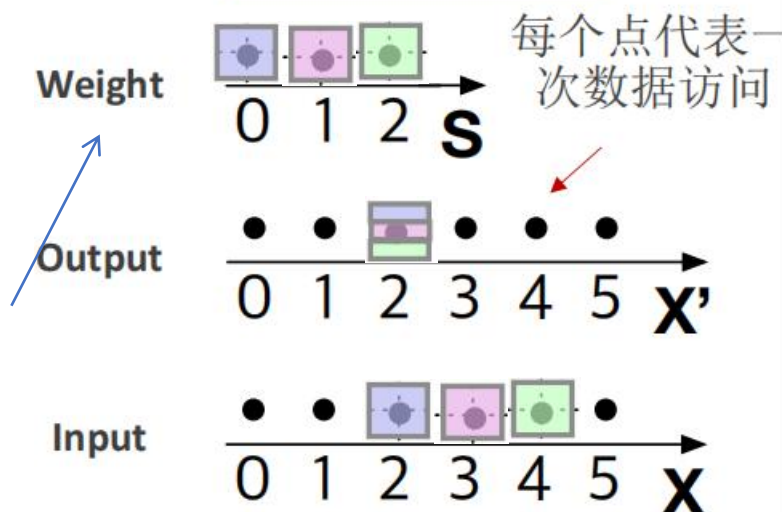
- Weight[s]
- Output[x']
- Input[x'+s]

Time = 2

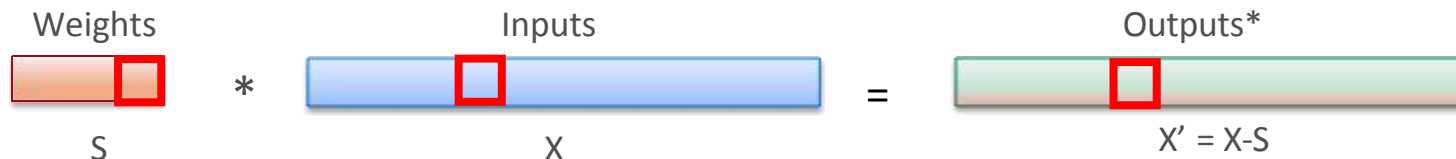
PE2
PE1
PE0



Data Space



权重固定的数据流



Computation

```
for(int s = 0; s < S; s++)
    for(int x = 0; x < X'; x++)
        Output[x] += Weight[s] * Input[x+s]
```

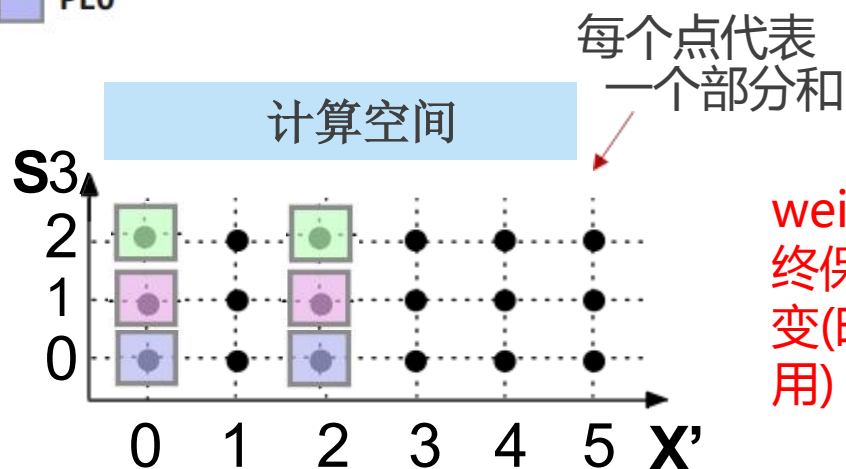
Data

PartialSum[X'] [S] needs to access:

- Weight[s]
- Output[x']
- Input[x'+s]

Time = 2

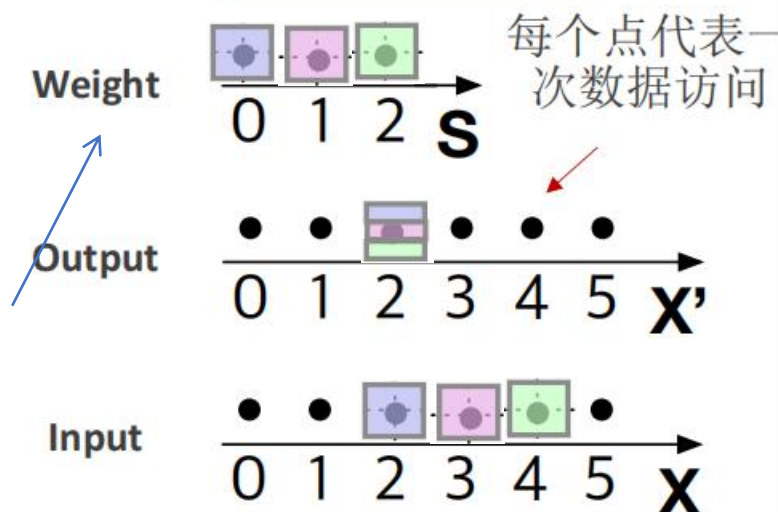
PE2
PE1
PE0



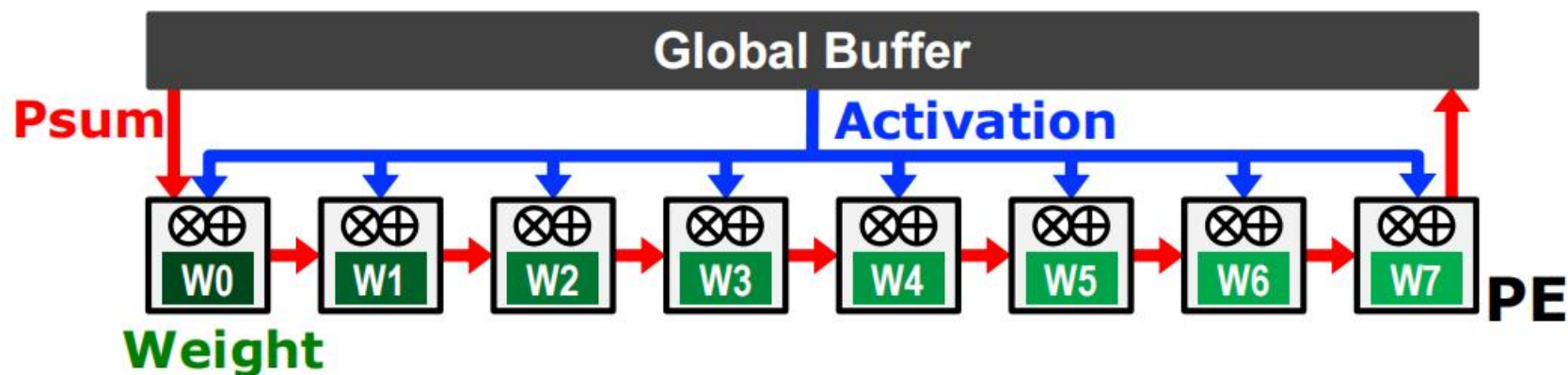
需要Output进行空间上规约

weight始终保持不变(时间重用)

Data Space



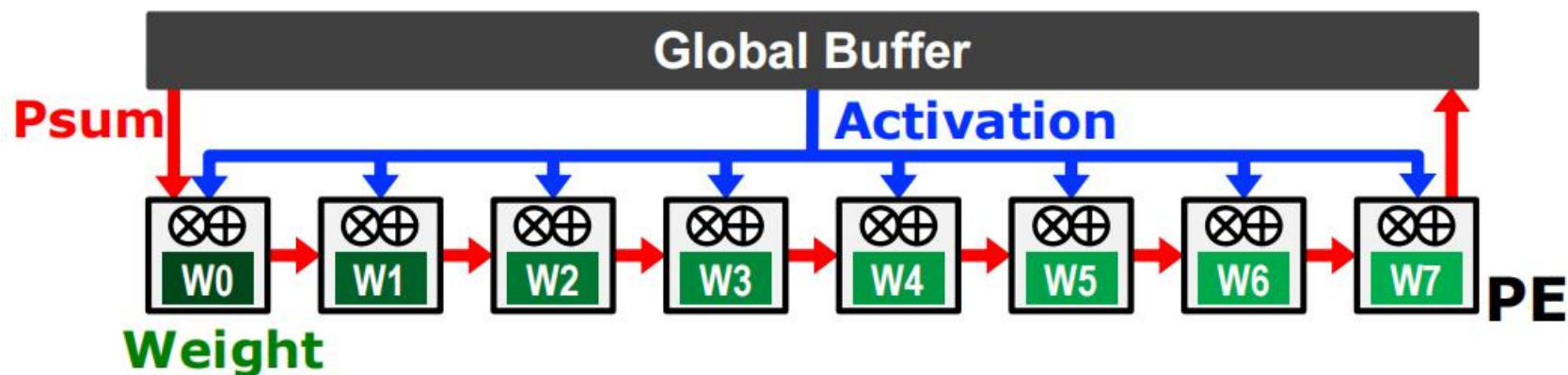
权重固定的数据流实现



第一轮：

1. 将weight分布在各个PE上, PE0- $\rightarrow$ W0, PE1- $\rightarrow$ W1,..., PE7- $\rightarrow$ W7
2. 广播input到各个PE上, PE0- $\rightarrow$ A[0], PE1- $\rightarrow$ A[1], ..., PE7- $\rightarrow$ A[7]
3. **运算**：每个PE得到Psum, PE0- $\rightarrow$ W0*A[0], PE1- $\rightarrow$ W[1]*A[1], ..., PE7- $\rightarrow$ W[1]*A[7]
4. **规约**：Psum按照PE传递累加

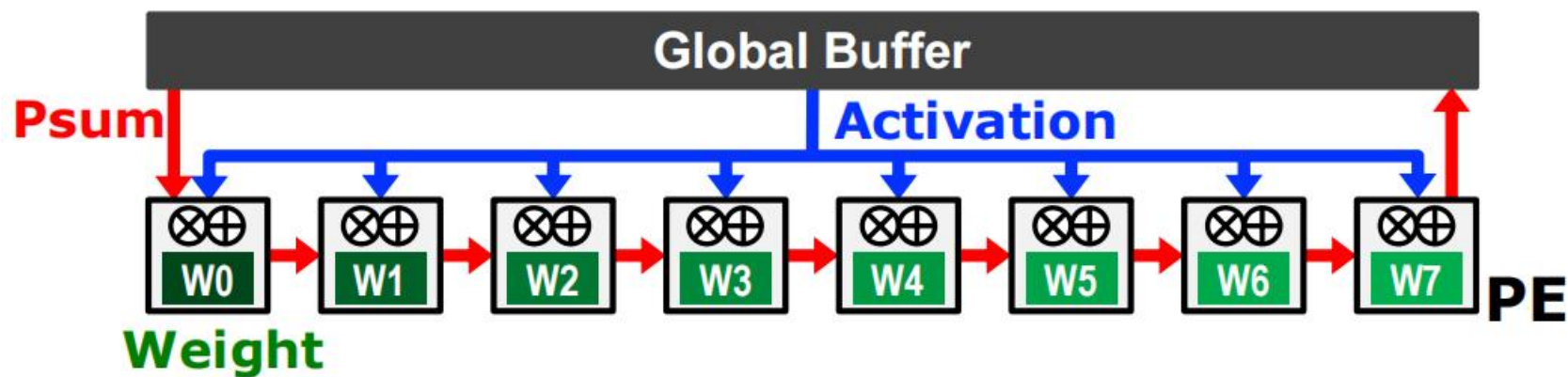
权重固定的数据流实现



第二轮：

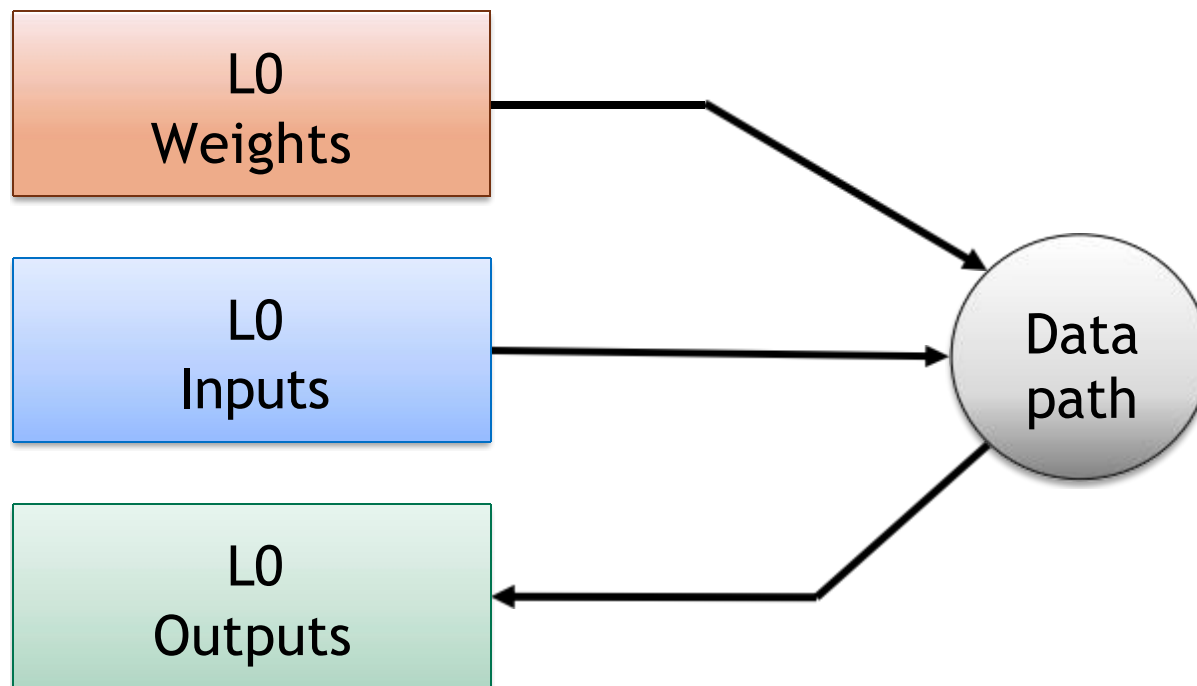
1. weight保持不变, PE0- $\rightarrow$ W0, PE1- $\rightarrow$ W1,..., PE7- $\rightarrow$ W7
2. input移动, 广播到各个PE上, PE0- $\rightarrow$ A[1], PE1- $\rightarrow$ A[2], ...
3. **运算**：每个PE得到Psum, PE0- $\rightarrow$ W0*A[1], PE1- $\rightarrow$ W[2]*A[1], ...
4. **规约**：Psum按照PE传递累加

权重固定的数据流实现

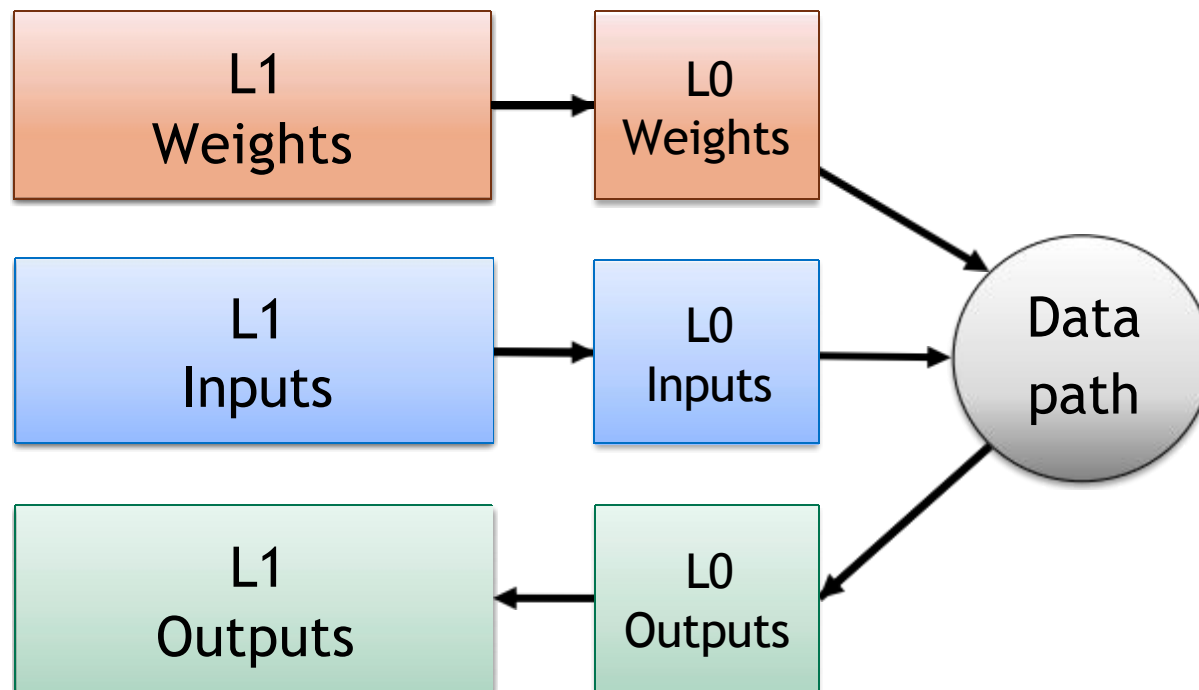


- 保持weight不变，减少了weight的访存次数
- 广播input，在PE之间传递累加Psum利用了空间重用

多层缓存

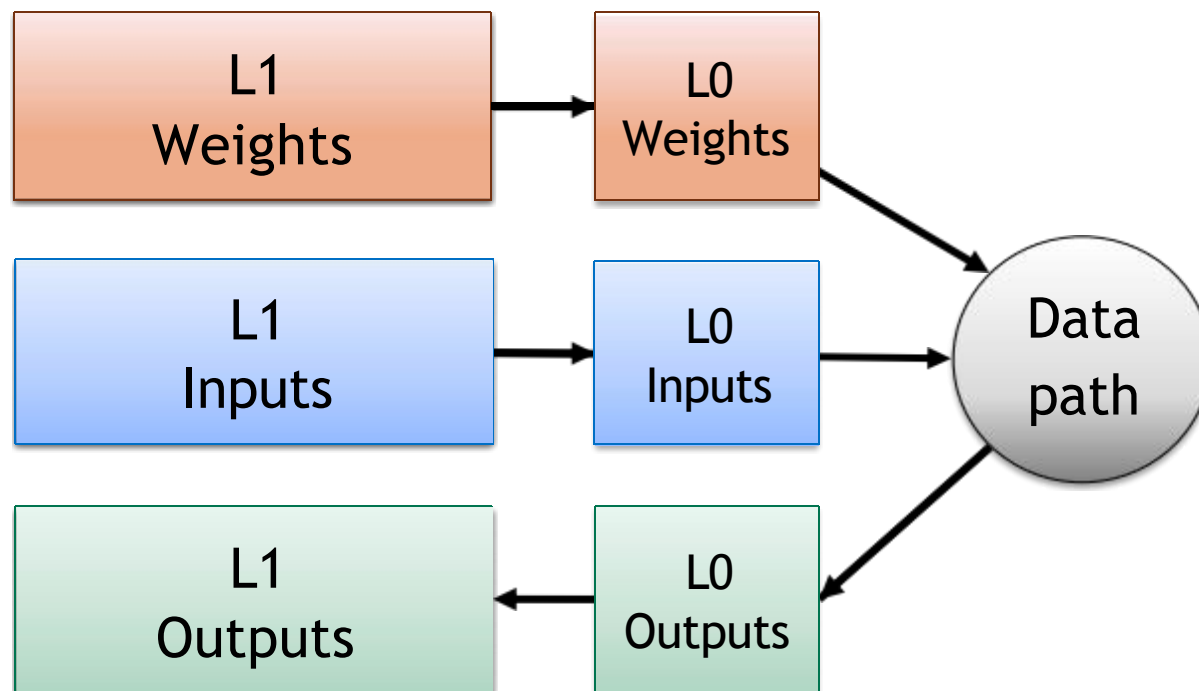


多层缓存



如何在循环嵌套中体现？

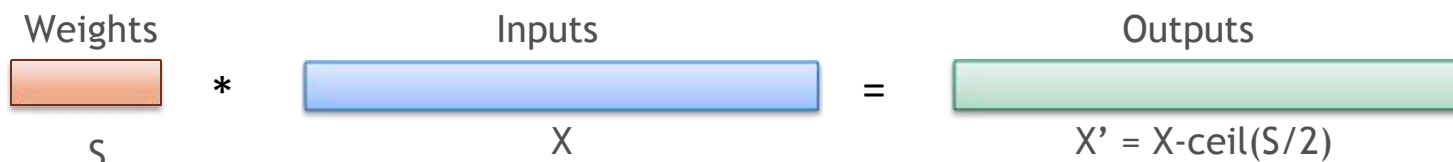
多层缓存



如何在循环嵌套中体现？

新的循环层级

一维卷积 – “Tiled”



```
int i[X];      # Input activations
int w[S];      # Filter Weights
int o[X'];     # Output activations

// Level 1
for (x1 = 0; x1 < X'1; x1++) {
    for (s1 = 0; s1 < S1; s1++) {
        // Level 0
        for (x0 = 0; x0 < X'0; x0++) {
            for (s0 = 0; s0 < S0; s0++) {
                x = x1 * X'0 + x0;
                s = s1 * S0 + s0;
                o[x] += i[x+s] * w[s];
            }
        }
    }
}
```

X' 和 S 被因式分解:

$$\begin{aligned} X'0 * X'1 &= X' \\ S0 * S1 &= S \end{aligned}$$

缓存大小

```
// Level 1
for (x1 = 0; x1 < X'1; x1++) {
  for (s1 = 0; s1 < S1; s1++) {
    // Level 0
    for (x0 = 0; x0 < X'0; x0++) {
      for (s0 = 0; s0 < S0; s0++) {
        o[x1*X'0+x0] += i[x1*X'0+x0 + s1*S0+s0] * w[s1*S0+s0];
      }
    }
  }
}
```



每次Level 1循环保持不变

Level 0的缓存大小是每次Level 1迭代内部存储数据所需要的空间

Level 1缓存大小是连续两次Level 1迭代之间保留数据需要的存储空间

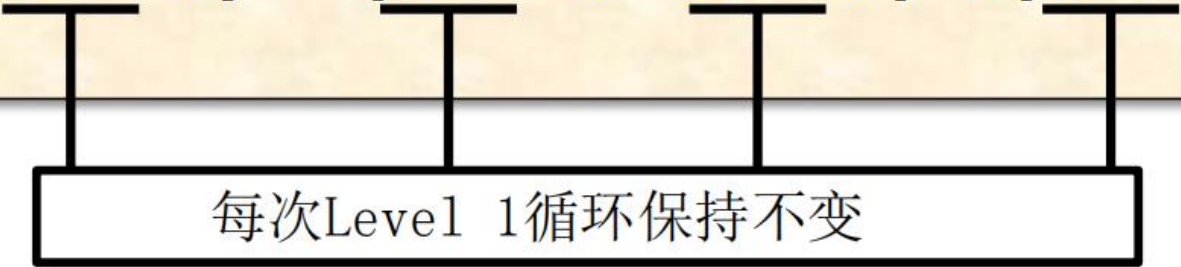
缓存大小

```
// Level 1
for (x1 = 0; x1 < X'1; x1++) {
  for (s1 = 0; s1 < S1; s1++) {
    // Level 0
    for (x0 = 0; x0 < X'0; x0++) {
      for (s0 = 0; s0 < S0; s0++) {
        o[x1*X'0+x0] += i[x1*X'0+x0 + s1*S0+s0] * w[s1*S0+s0];
      }
    }
  }
}
```

每次Level 1循环保持不变

	Level 0	Level 1
Weights	S0	S
Inputs	X'0+S0	S
Outputs	X'0	1

```
// Level 1
for (x1 = 0; x1 < X'1; x1++) {
  for (s1 = 0; s1 < S1; s1++) {
    // Level 0
    for (x0 = 0; x0 < X'0; x0++) {
      for (s0 = 0; s0 < S0; s0++) {
        o[x1*X'0+x0] += i[x1*X'0+x0 + s1*S0+s0] * w[s1*S0+s0];
      }
    }
  }
}
```



The diagram shows four vertical lines originating from the innermost loop's index expressions: $x1 \cdot X'0 + x0$, $s1 \cdot S0 + s0$, $s1 \cdot S0 + s0$, and $s1 \cdot S0 + s0$. These lines converge into a single horizontal line that connects to a rectangular box below the code.

每次Level 1循环保持不变

缓存的功耗与缓存大小呈函数关系

每个level的缓存功耗与缓存访问次数成正比

```
// Level 1
for (x1 = 0; x1 < X'1; x1++) {
  for (s1 = 0; s1 < S1; s1++) {
    // Level 0
    for (x0 = 0; x0 < X'0; x0++) {
      for (s0 = 0; s0 < S0; s0++) {
        o[x1*X'0+x0] += i[x1*X'0+x0 + s1*S0+s0] * w[s1*S0+s0];
      }
    }
  }
}
```

每次Level 1循环保持不变

对于level 0，就是所有数据传输到数据通路的能耗

对于level L，由三部分组成：

- 来自level L+1的数据
- 传输到level L-1的数据
- 从level L-1返回的数据

权重访问能耗

- Level 0 读操作
 - 每次 level 1 迭代 -> $X'_0 * S_0$ 次读取weight
 - 乘以 level 1 迭代次数 $X'_1 * S_1$
 - 总读取次数 = $(X'_0 * S_0) * (X'_1 * S_1) = (X'_0 * X'_1) * (S_0 * S_1) = SX'$
- Level 1 到 Level 0 数据传输
 - 每次 level 1 迭代 -> weights传输量为 S_0
 - 乘以 level 1 迭代次数 $X'_1 * S_1$
 - 总数据传输 -> $S_0 * (X'_1 * S_1) = X'_1 * (S_0 * S_1) = SX'_1$

输入特征访问能耗

- Level 0 读数据
 - 每次 level 1 迭代 -> inputs读次数为 $X'0+S0$
 - 乘以 level 1 迭代次数 $X'1*S1$
 - 总次数 = $X'1*S1*(X'0+S0) = ((X'1*X'0)*S1)+(X'1*(S1*S0)) = X'*S1+X'1*S$
- Level 1 到 level 0 数据传输
 - 对于 $s=0$, input数据传输量为 $X'0+S0$
 - 接下来的 $S1-1$ 次迭代, 每次数据传输量为 $S0$
 - 所以每次 $x1$ 迭代总传输量为: $X'0+S0*S1 = X'0+S$
 - 乘以 $x1$ 迭代次数 $X'1$
 - 总传输量为 = $X'1*(X'0+S) = (X'1*X'0)+X'1*S = X'+X'1*S$

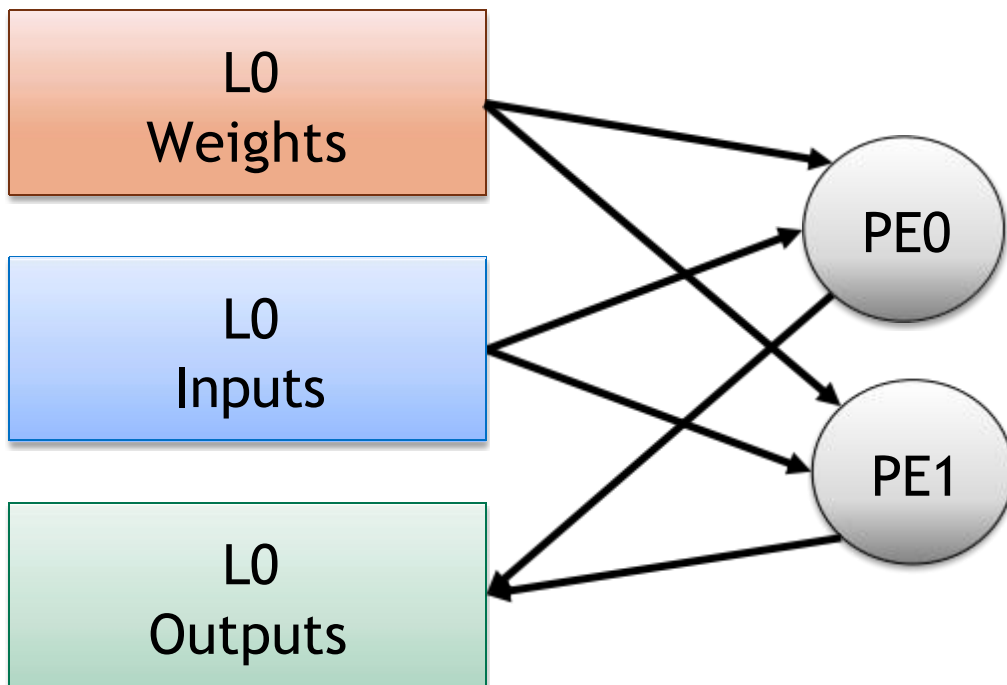
输出访问能耗

- Level 0 写操作

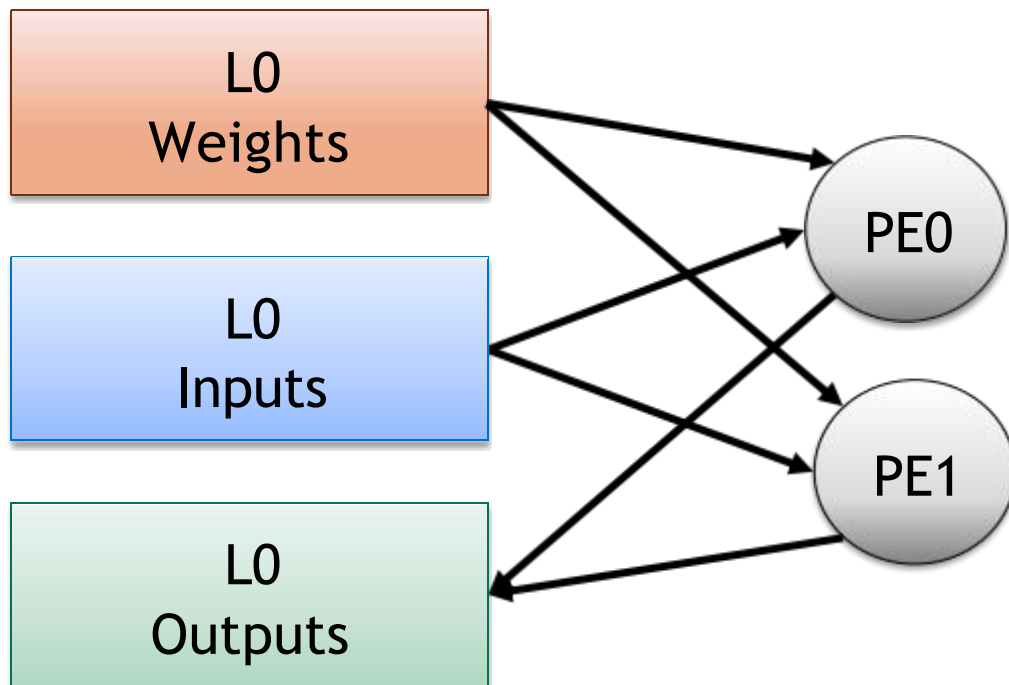
- 因为 level 0 是输出固定，每次level 1迭代只有 X'_0 次writes
- 乘以 level 1 $X'_1 * S_1$ 次迭代
- 总次数 = $X'_0 * (X'_1 * S_1) = (X'_0 * X'_1) * S_1 = X' * S_1$ writes

- Level 0 to 1 数据传输

- 每 S_1 次迭代之后，有 X'_0 个卷积结果(完整累加和)从level 0传回level 1
- 一共有 X'_1 次 S_1 次迭代
- 所以总数为 $X'_1 * X'_0 = X'$



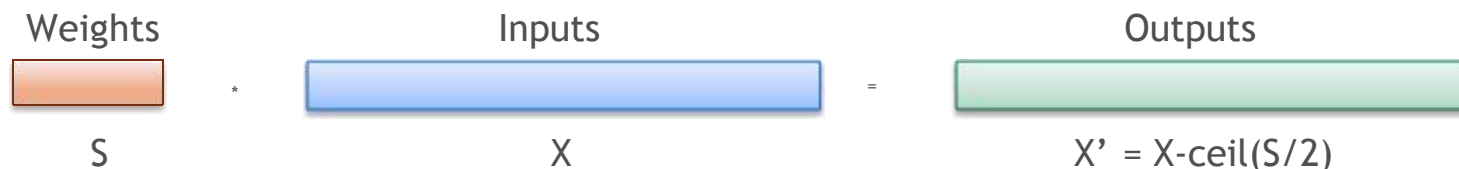
如何在迭代嵌套中反映出来？



如何在迭代嵌套中反映出来？

新的循环层次

一维卷积 – 输出划分



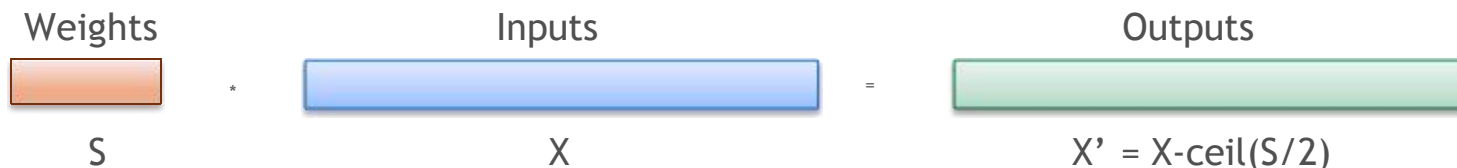
```
int i[X];      # Input activations
int w[S];      # Filter Weights
int o[X'];     # Output activations
```

Note:
 $X'0 * X'1 = X'$
 $S0 * S1 = S$

```
// Level 1
parallel-for (x1 = 0; x1 < X'1; x1++) {
    parallel-for (s1 = 0; s1 < S1; s1++) {
        // Level 0
        for (x0 = 0; x0 < X'0; x0++) {
            for (s0 = 0; s0 < S0; s0++) {
                o[x1*X'0+x0] += i[x1*X'0+x0 + s1*S0+s0]
                               * w[s1*S0+s0];
            }
        }
    }
}
```

每个PE负责处理一部分output (卷积操作)

一维卷积 – 输出划分



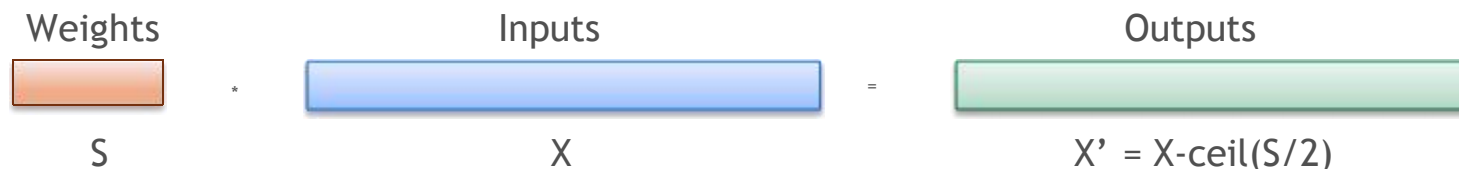
```
int i[X];      # Input activations
int w[S];      # Filter Weights
int o[X'];     # Output activations
```

Note:
 $X'_0 * X'_1 = X'$
 $S_0 * S_1 = S$

```
// Level 1
parallel-for (x1 = 0; x1 < X'1; x1++) {
    parallel-for (s1 = 0; s1 < S1; s1++) {
        // Level 0
        for (x0 = 0; x0 < X'0; x0++) {
            for (s0 = 0; s0 < S0; s0++) {
                o[x1*X'0+x0] += i[x1*X'0+x0 + s1
                                * w[s1*S0+s0];
            }
        }
    }
}
```

$X'_1 = 2$

一维卷积 – 输出划分



```
int i[X];      # Input activations
int w[S];      # Filter Weights
int o[X'];     # Output activations
```

Note:
 $X'0 * X'1 = X'$
 $S0 * S1 = S$

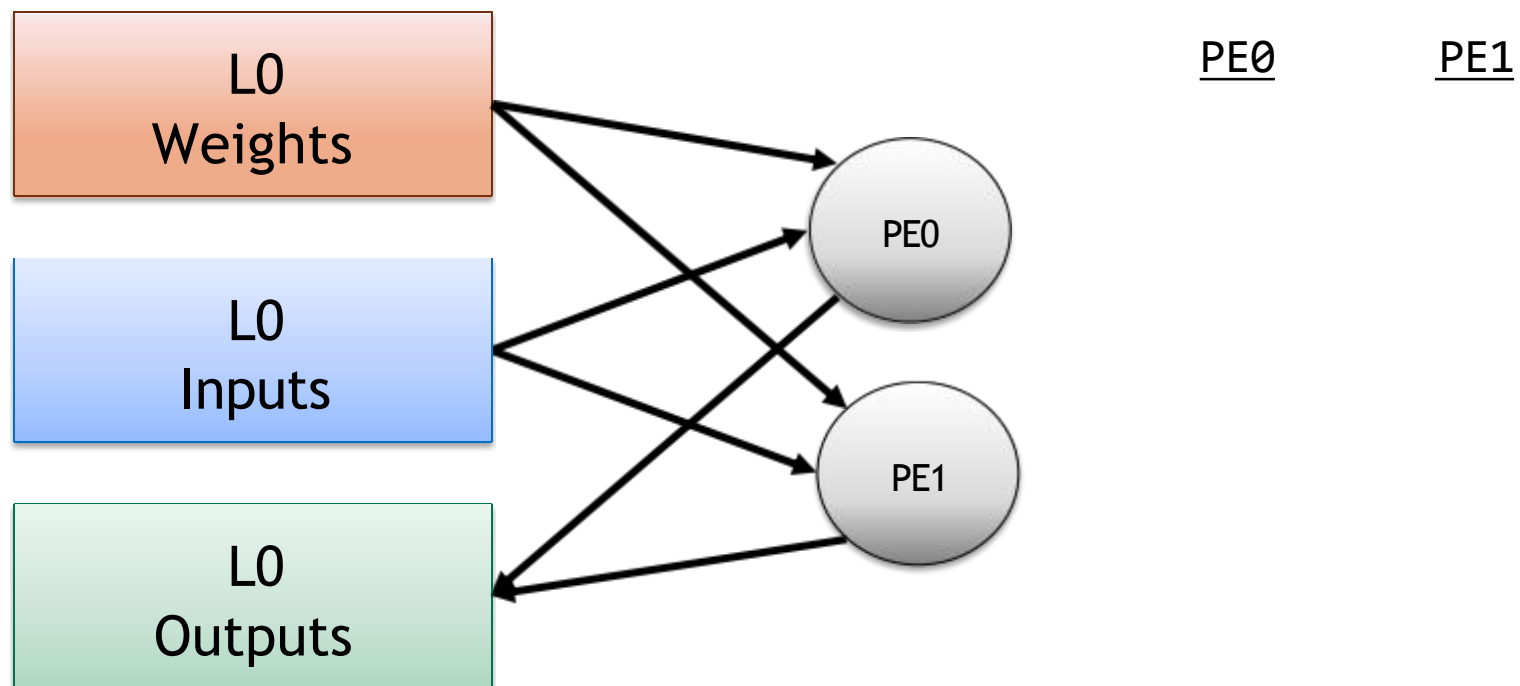
```
// Level 1
parallel-for (x1 = 0; x1 < X'1; x1++) {
    parallel-for (s1 = 0; s1 < S1; s1++) {
        // Level 0
        for (x0 = 0; x0 < X'0; x0++) {
            for (s0 = 0; s0 < S0; s0++) {
                o[x1*X'0+x0] += i[x1*X'0+x0 + s1*S0+s0]
                               * w[s1*S0+s0];
            }
        }
    }
}
```

$X'1 = 2$

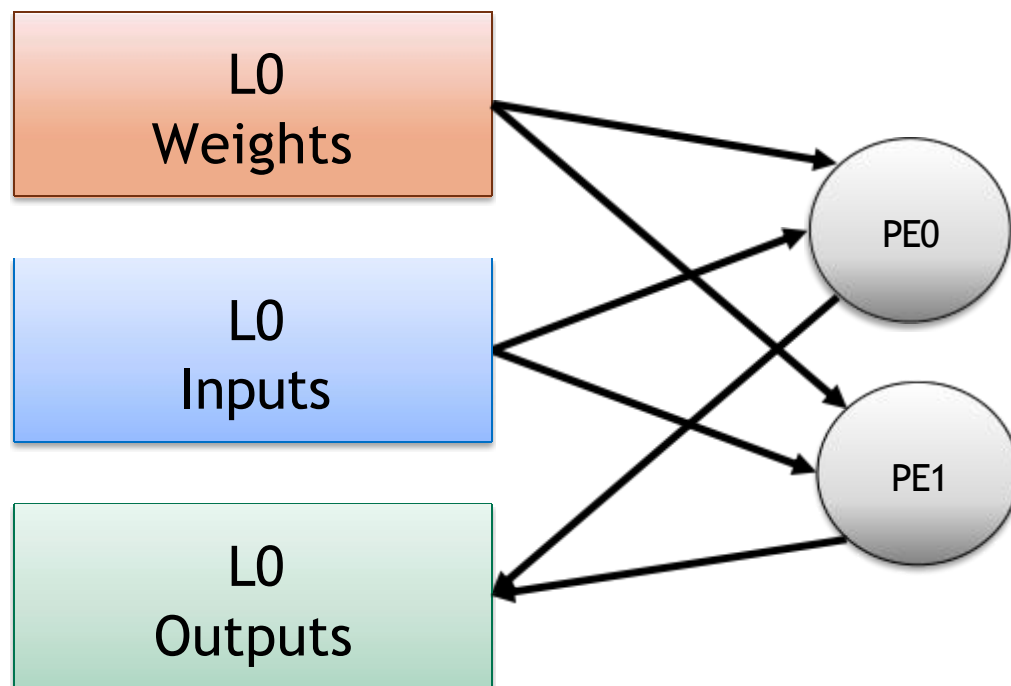
$S1 = 1 \Rightarrow s1 = 0$

level 1只剩下一层，每个PE各负责一半($X'1 = 0$ 由一个PE负责， $X'1=1$ 由另一个PE负责，并行进行计算)

多个PEs



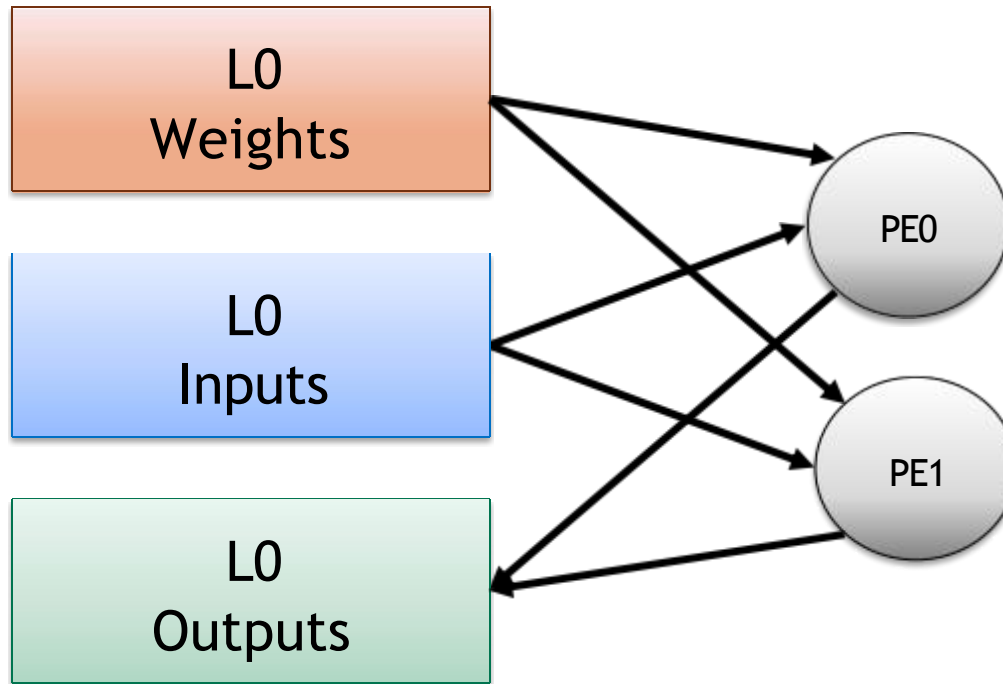
多个PEs



PE0
>>w[0]

PE1

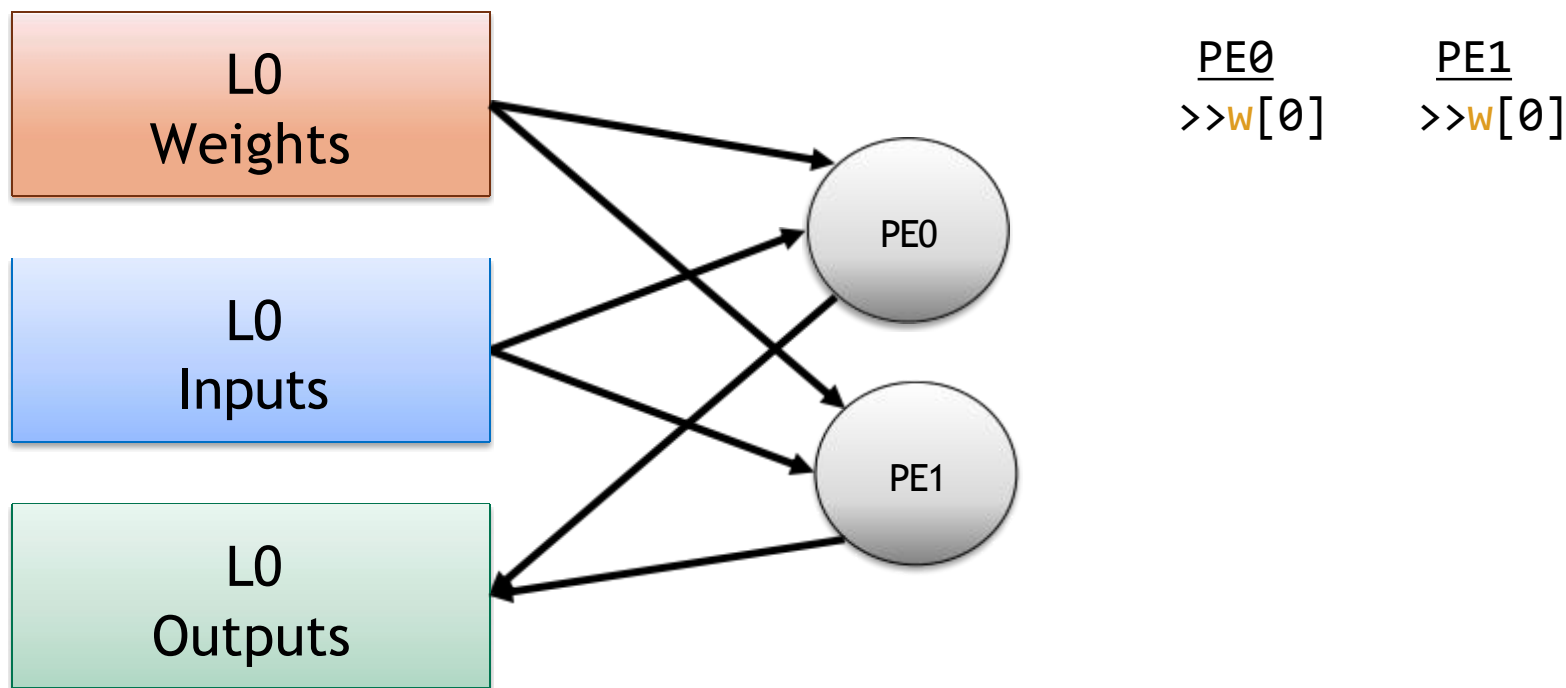
多个PEs



PE0
>> $w[0]$

PE1
>> $w[0]$

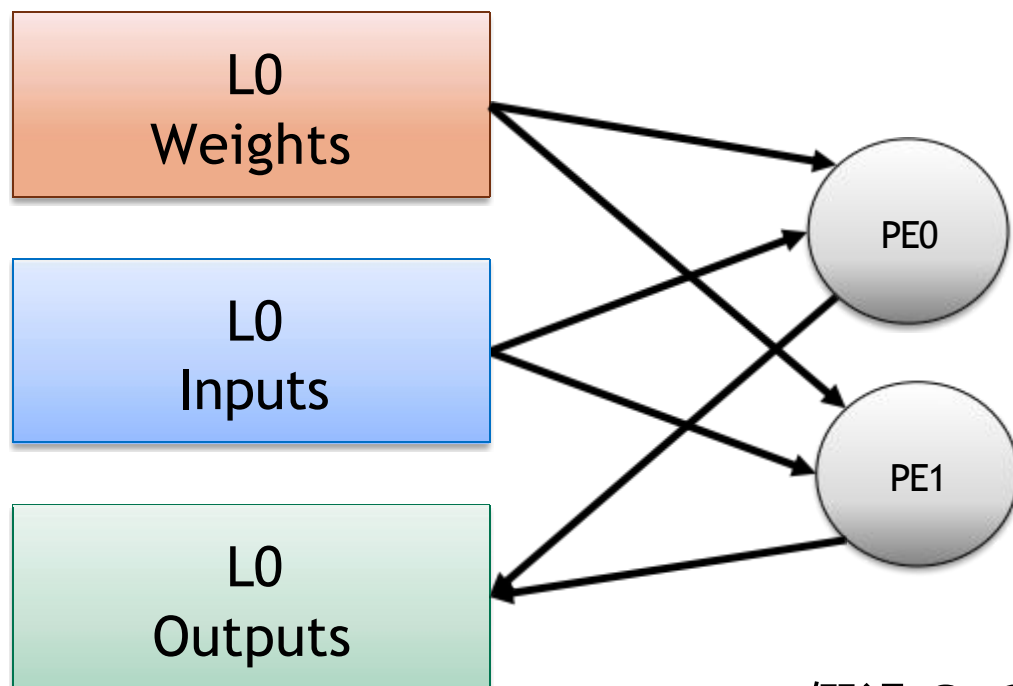
多个PEs



设计空间?

取一次 w ，广播给两个PE

多个PE: 输出划分

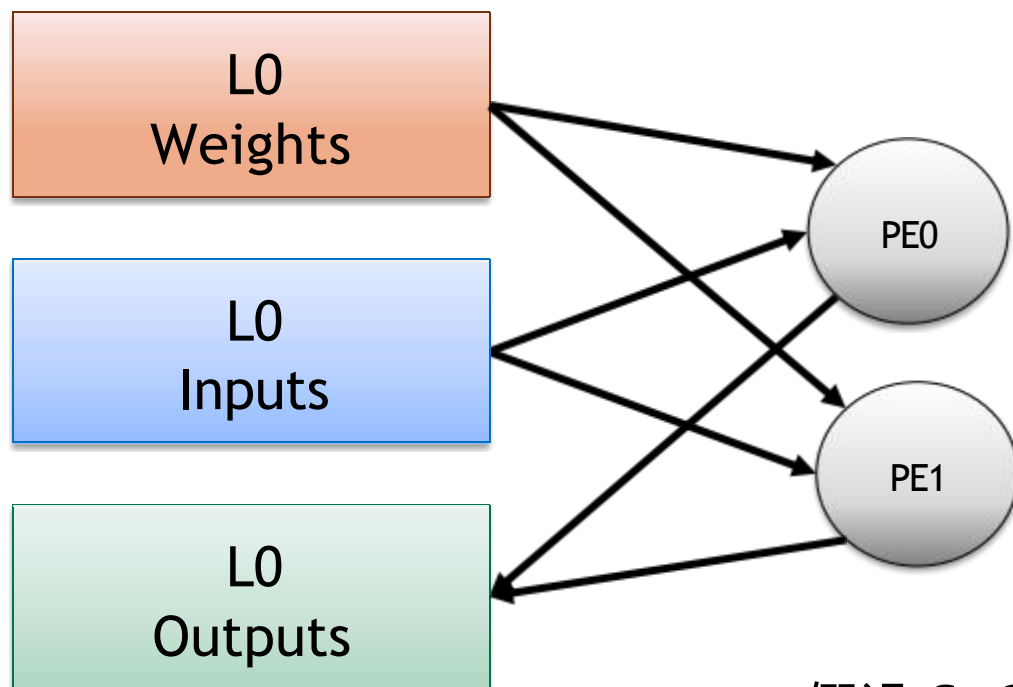


PE0
>>w[0]

PE1
>>w[0]

假设 $S=3$

多个PE: 输出划分

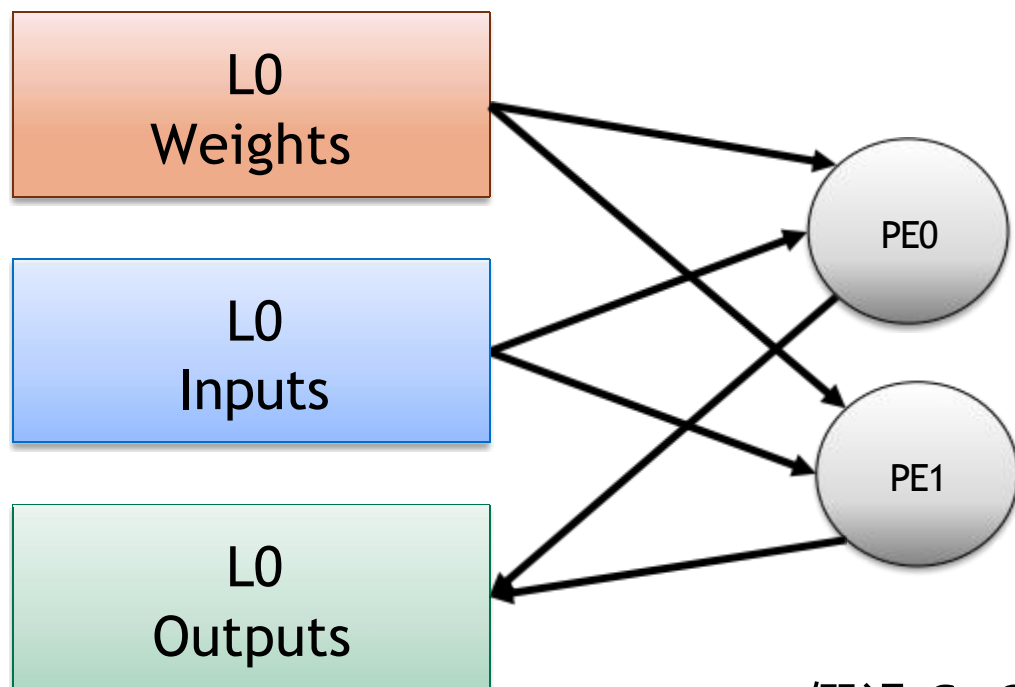


PE0
>>w[0]
>>i[0]

PE1
>>w[0]
>>i[X'0+0]

假设 $S=3$

多个PE: 输出划分



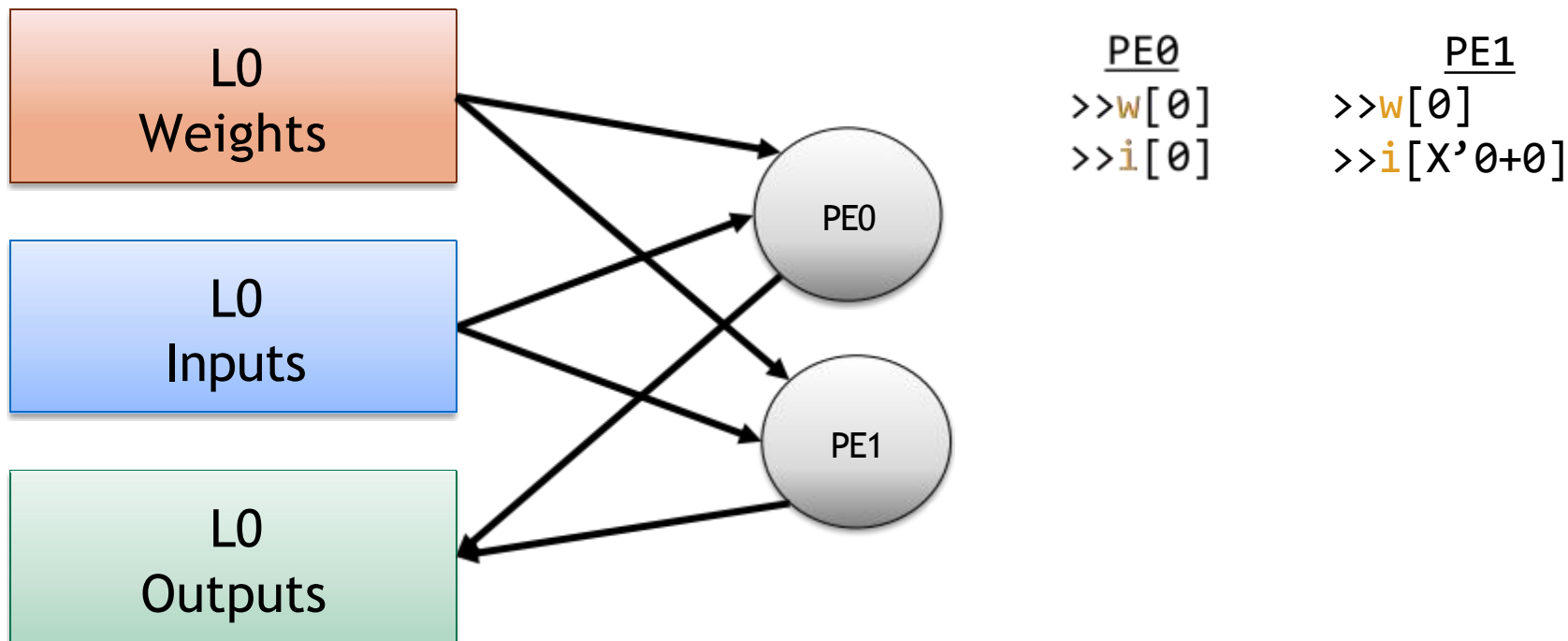
PE0
>>w[0]
>>i[0]

PE1
>>w[0]
>>i[X'0+0]

假设 $S=3$

设计空间?

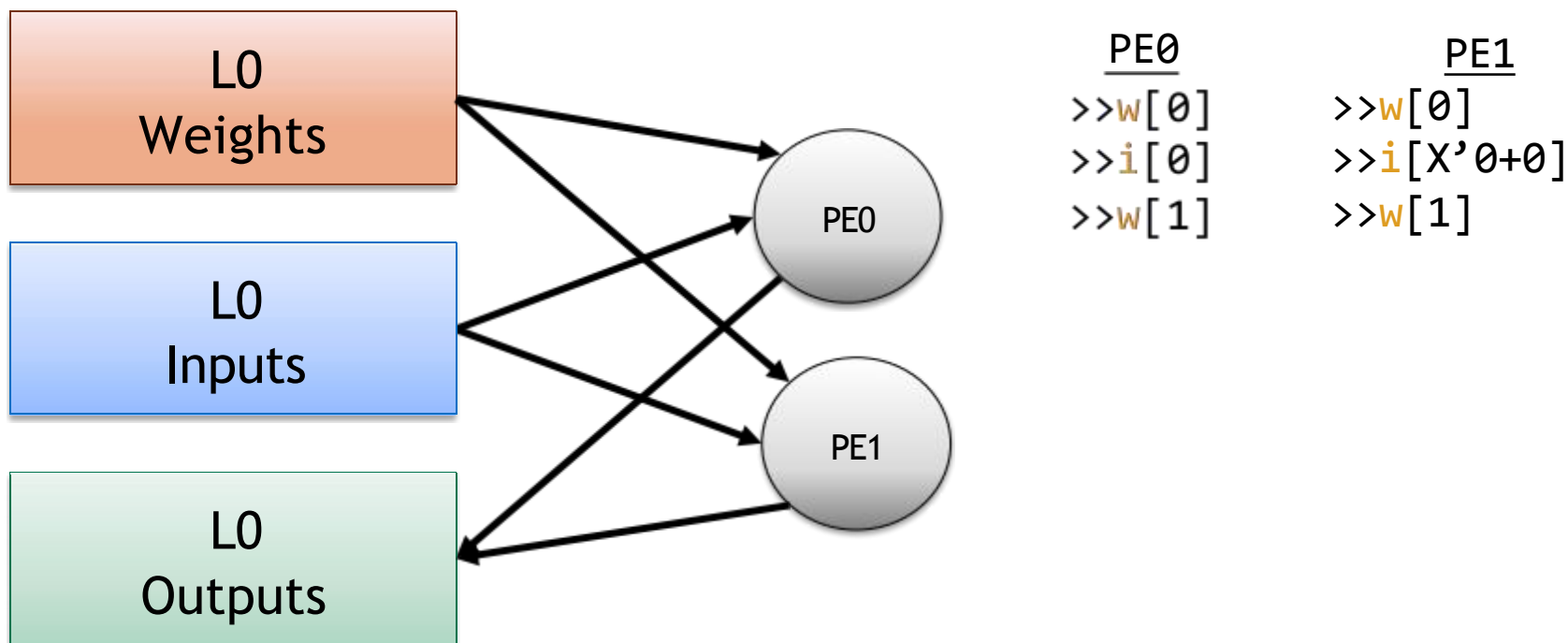
多个PE: 输出划分



设计空间?

两个PE并行取input

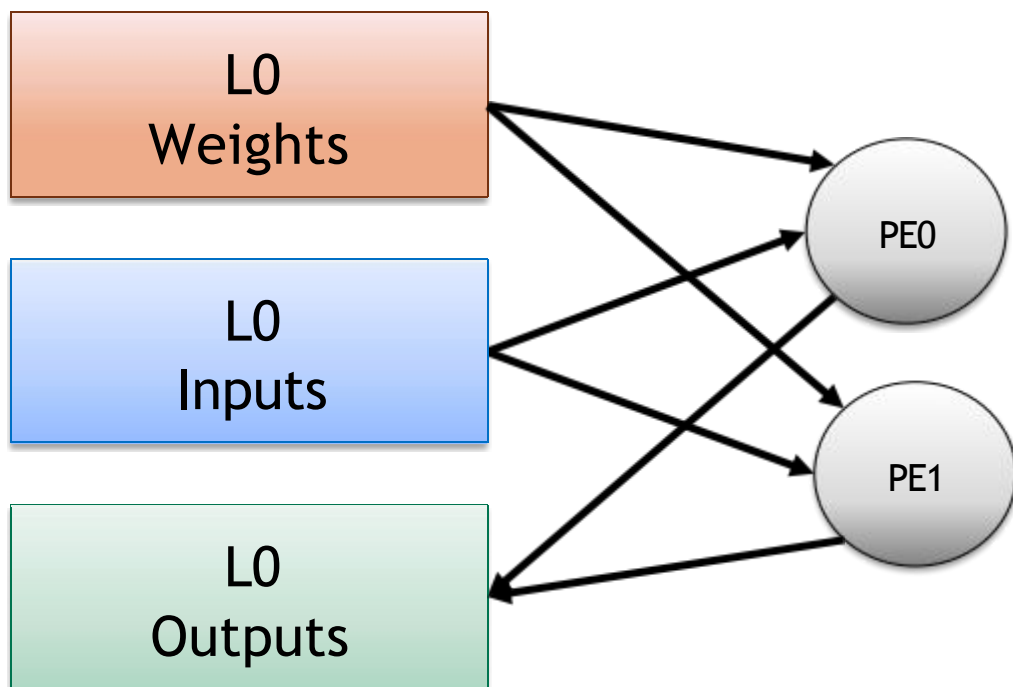
多个PE: 输出划分



设计空间?

两个PE并行取input

多个PE: 输出划分



设计空间?

两个PE并行取input

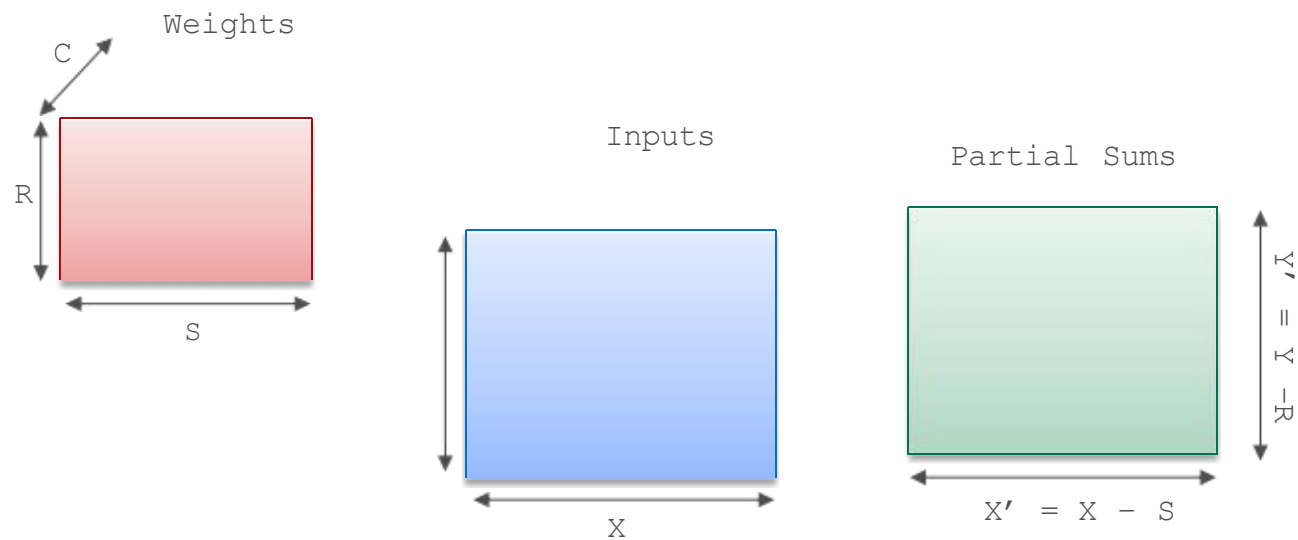
PE0
 >>w[0]
 >>i[0]
 >>w[1]
 >>i[1]
 >>w[2]
 >>i[2]
 <<o[0]

>>w[0]
 >>i[1]
 >>w[1]
 >>i[2]
 >>w[2]
 >>i[3]
 <<o[1]

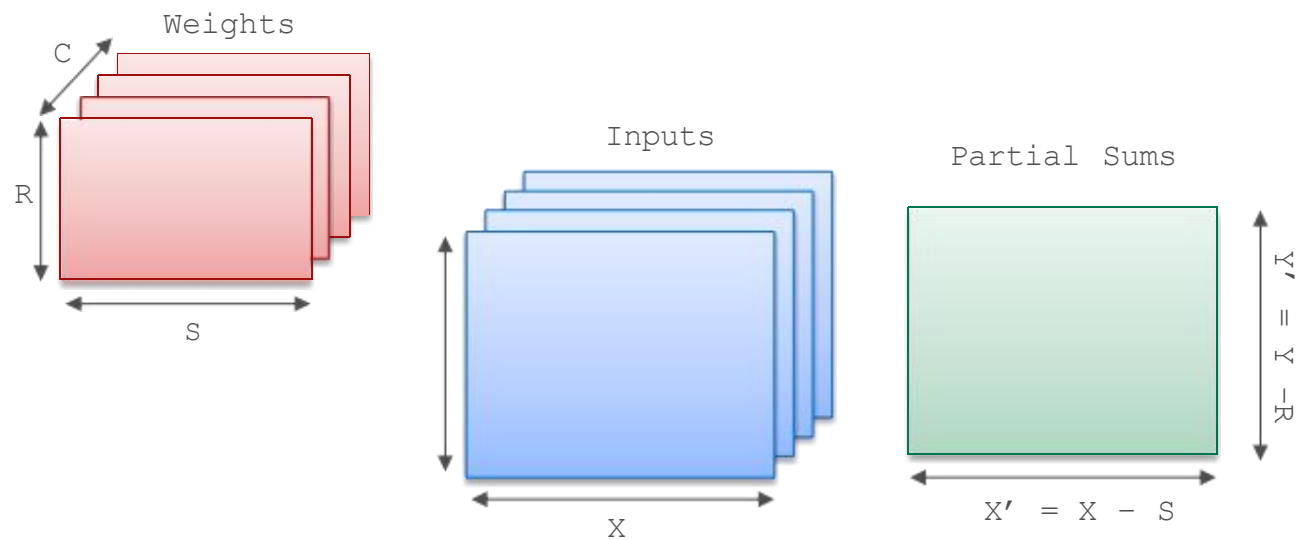
PE1
 >>w[0]
 >>i[X'0+0]
 >>w[1]
 >>i[X'0+1]
 >>w[2]
 >>i[X'0+2]
 <<o[X'0+0]

>>w[0]
 >>i[X'0+1]
 >>w[1]
 >>i[X'0+2]
 >>w[2]
 >>i[X'0+3]
 <<o[X'0+1]

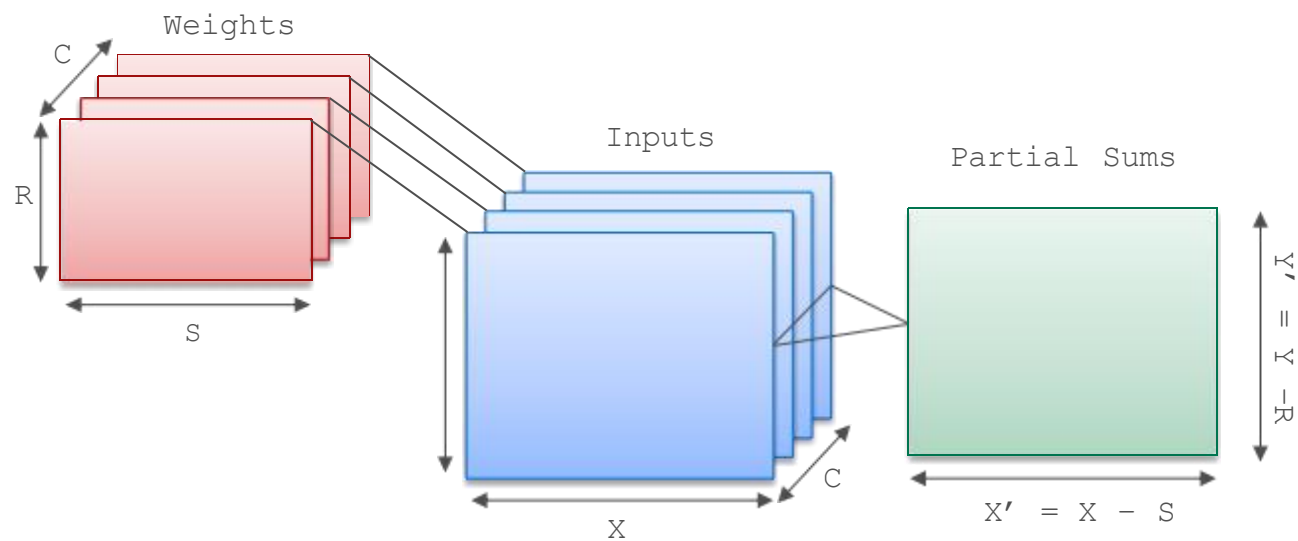
映射多维卷积



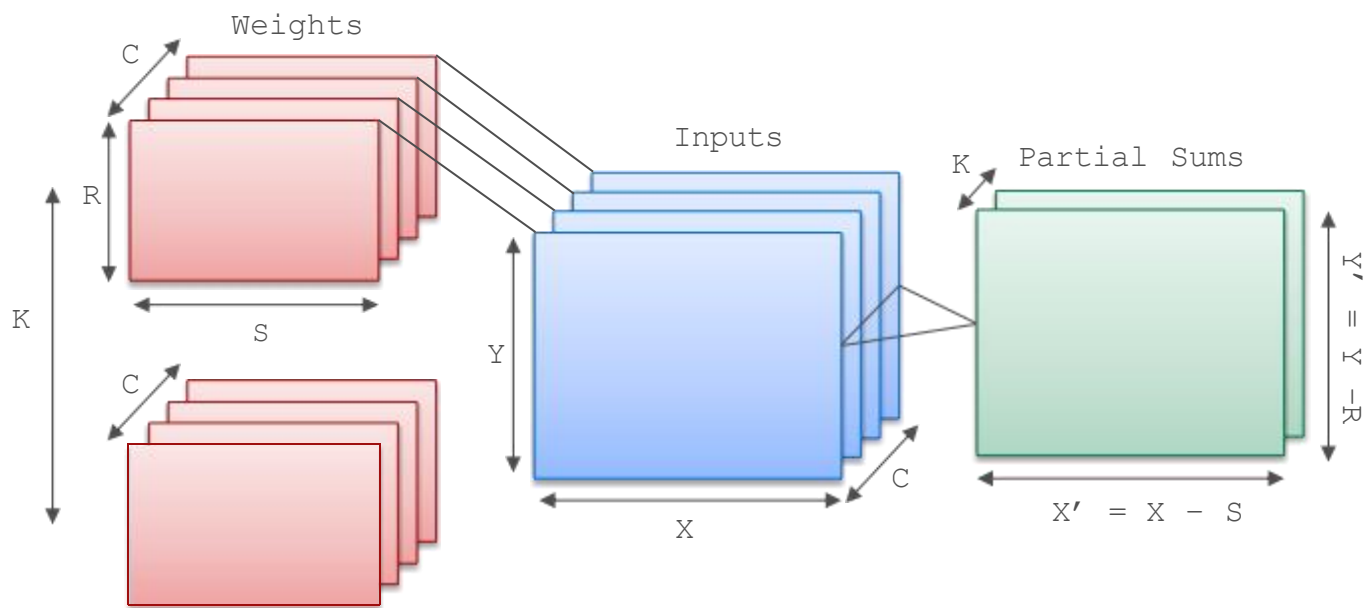
映射多维卷积



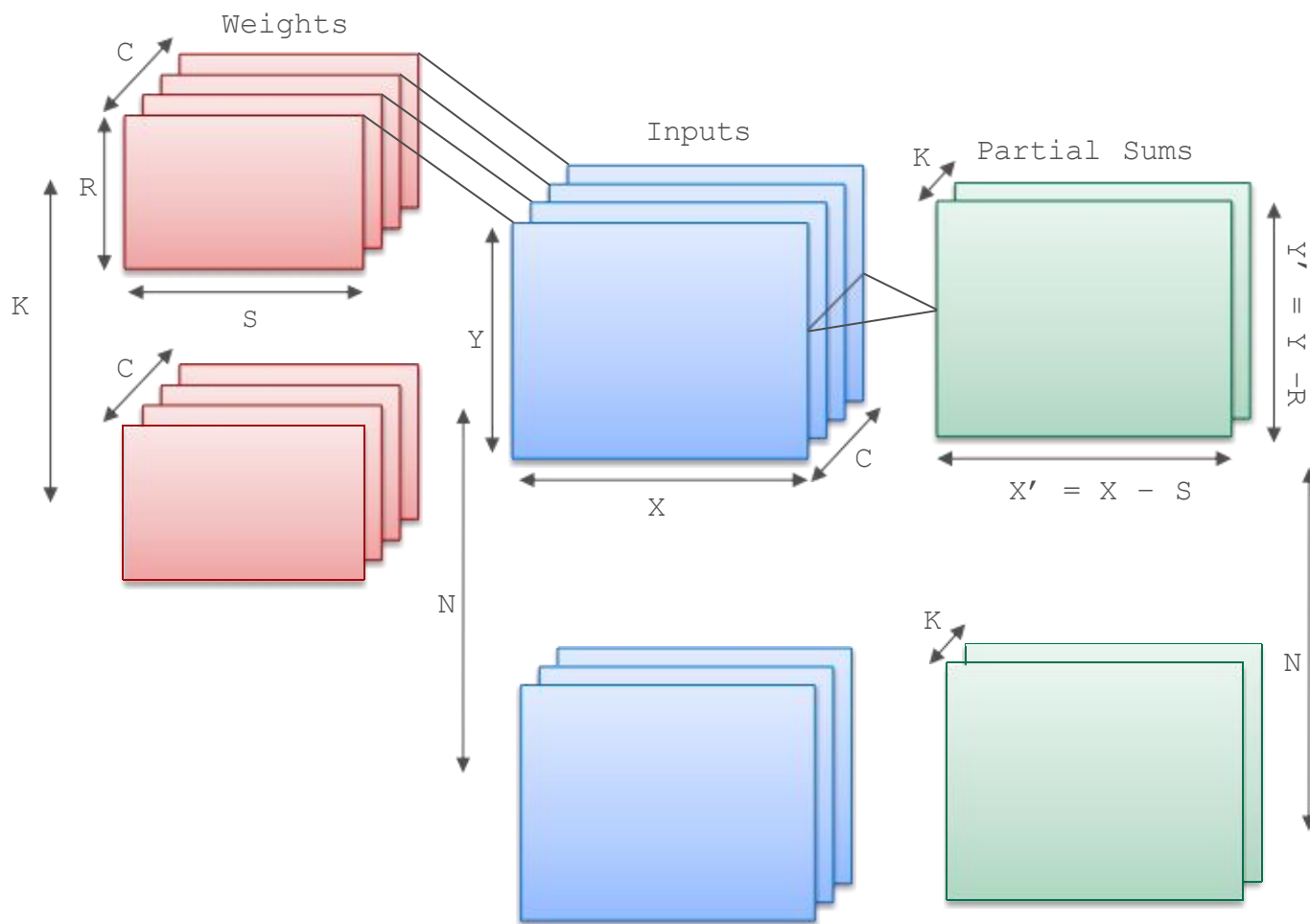
映射多维卷积



映射多维卷积



映射多维卷积



多维卷积

```
int i[C][Y][X];      # Input activation channels
int w[K][C][R][S];    # Filter weights (per channel pair)
int o[K][Y'][X'];     # Output activation channels

for (k = 0; k < K; k++) {
    for (y = 0; y < Y'; y++) {
        for (x = 0; x < X'; x++) {
            for (c = 0; c < C; c++) {
                for (r = 0; r < R; r++) {
                    for (s = 0; s < S; s++) {
                        o[k][y][x] += i[c][y+r][x+s]*w[k][c][r][s];
```

完成加速器描述

```
Input Fmaps:  I[G][N][C][H][W]
Filter Weights: W[G][M][C][R][S]
Output Fmaps: O[G][N][M][E][F]
```

```
// DRAM levels
for (g3=0; g3<G3; g3++) {
  for (n3=0; n3<N3; n3++) {
    for (m3=0; m3<M3; m3++) {
      for (f3=0; f3<F3; f3++) {
        // Global buffer levels
        for (g2=0; g2<G2; g2++) {
          for (n2=0; n2<N2; n2++) {
            for (m2=0; m2<M2; m2++) {
              for (f2=0; f2<F2; f2++) {
                for (c2=0; c2<C2; c2++) {
                  for (s2=0; s2<S2; s2++) {
                    // NoC levels
                    parallel-for (g1=0; g1<G1; g1++) {
                      parallel-for (n1=0; n1<N1; n1++) {
                        parallel-for (m1=0; m1<M1; m1++) {
                          parallel-for (f1=0; f1<F1; f1++) {
                            parallel-for (c1=0; c1<C1; c1++) {
                              parallel-for (s1=0; s1<S1; s1++) {
                                // SPad levels
                                for (f0=0; f0<F0; f0++) {
                                  for (n0=0; n0<N0; n0++) {
                                    for (e0=0; e0<E; e0++) {
                                      for (r0=0; r0<R; r0++) {
                                        for (c0=0; c0<C0; c0++) {
                                          for (m0=0; m0<M0; m0++) {
                                            O += I x W;
                                          }
                                        }
                                      }
                                    }
                                  }
                                }
                              }
                            }
                          }
                        }
                      }
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

DRAM to
Accelerator

Inter-PE Global
Buffer

Inter-PE
NoC

Intra-
PE

不同的level映射到不同的硬件资源

- **每个level需要决策:**

- 所有索引的排列顺序
- 每维索引在本level的因子(所有level因子乘积正好等于该维索引长度)
- 每维索引是否出现在本level

映射表示

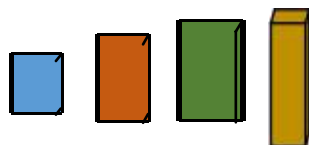
- 每个level需要决策:

- 所有索引的排列顺序 索引维度的阶乘
- 每维索引在本level的因子(所有level因子乘积正好等于该维索引长度)
- 每维索引是否出现在本level 2^N (N是索引维度)

决策空间巨大：(索引维度的阶乘)! * 2^N * 因子分解

探索映射方案

- 潜在循环顺序和因子分解的巨大空间
- 精确模拟实际维度和结构尺寸太慢
- Solution: 使用建模分析



```
int i[C][Y][X];    # Input activation channels
int w[K][C][R][S]; # Filter weights (per channel pair)
int o[K][Y'][X'];  # Output activation channels

for (k = 0; k < K; k++) {
  for (y = 0; y < Y'; y++) {
    for (x = 0; x < X'; x++) {
      for (c = 0; c < C; c++) {
        for (r = 0; r < R; r++) {
          for (s = 0; s < S; s++) {
            o[k][y][x] += i[c][y+r][x+s]*w[k][c][r][s];
          }
        }
      }
    }
  }
}
```

⋮

DL Operator

Mapping

HW Params

Analytical
Cost Model

Latency,
Throughput,
Energy, ...

e.g.,: Timeloop (ISPASS 2019),
MAESTRO (MICRO 2019), ..